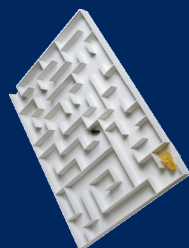


- **Полный перебор и его оптимизация**
- **Как свести решение к задаче существования**
- **Тождественные преобразования условий**

Эта книга о том, что делать с задачей, если её решение нельзя вычитать в учебнике. Иначе говоря, – эта книга о творчестве в программировании. В тексте вы не найдете готовых рецептов, скорее, это описание того, как искать путь в интеллектуальной неизвестности, как выстроить свое мышление, так чтобы, не зная готовых формул и теорем, все же получить достаточно приличное решение за оптимальное время.

Издание предназначено для широкого круга начинающих программистов – школьников, студентов, а также всех думающих разработчиков программного обеспечения.



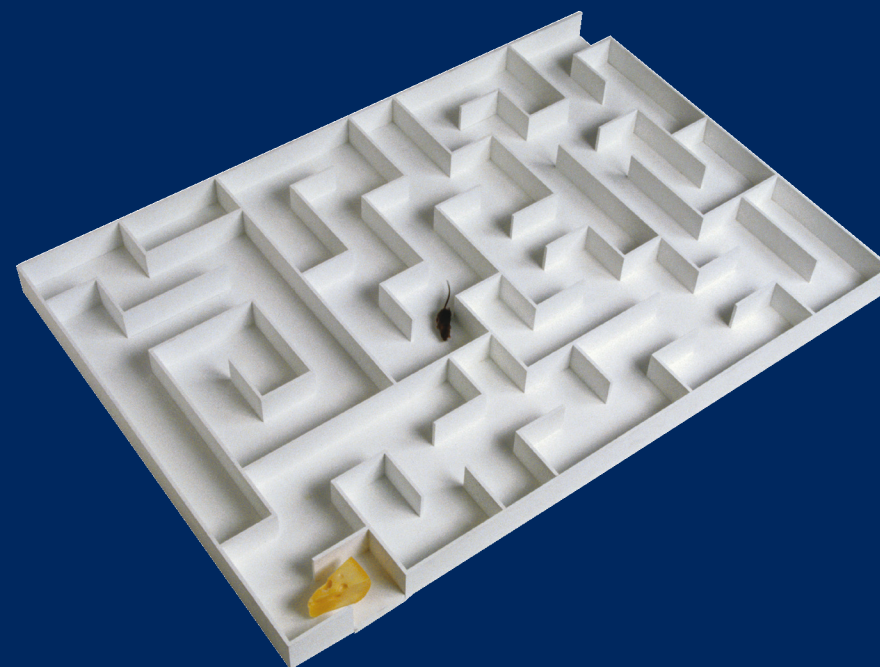
Книга является заключительной в авторской трилогии:
«Современное программирование с нуля»
«Искусство алгоритмизации»
«Искусство поиска решения в нестандартной задаче»



Internet-магазин:
www.dmkpress.com
Книга-почтой:
orders@alians-kniga.ru
Оптовая продажа:
«Альянс-книга». (499)782-3889
books@alians-kniga.ru



Искусство поиска решения в нестандартной задаче



Потопахин В. В.

Искусство поиска решения в нестандартной задаче



Москва, 2016

УДК 004.421
ББК 32.973.26-018
П64

П64 Потопахин В. В.

Искусство поиска решения в нестандартной задаче. – М.: ДМК Пресс, 2016. – 166 с.: ил.

ISBN 978-5-97060-198-3

Книга является заключительной в авторской трилогии книг после «Современное программирование с нуля» и «Искусство алгоритмизации».

Эта книга о том, что делать с задачей, если её решение нельзя вычитать в учебнике. Иначе говоря, – эта книга о творчестве в программировании. В тексте вы не найдете готовых рецептов, скорее, это описание того, как искать путь в интеллектуальной неизвестности, как выстроить свое мышление, так чтобы, не зная готовых формул и теорем, все же получить достаточно приличное решение за оптимальное время.

Издание предназначено для широкого круга начинающих программистов – школьников, студентов, а также всех думающих разработчиков программного обеспечения.

УДК 004.421
ББК 32.973.26-018

Все права защищены. Любая часть этой книги не может быть воспроизведена в какой бы то ни было форме и какими бы то ни было средствами без письменного разрешения владельцев авторских прав.

Материал, изложенный в данной книге, многократно проверен. Но, поскольку вероятность технических ошибок все равно существует, издательство не может гарантировать абсолютную точность и правильность приводимых сведений. В связи с этим издательство не несет ответственности за возможные ошибки, связанные с использованием книги.

ISBN 978-5-97060-198-3

© Потопахин В. В., 2016
© Оформление, издание, ДМК Пресс, 2016



ОГЛАВЛЕНИЕ

Введение	5
ГЛАВА 1. Как решается сложная задача	7
Пошаговое уточнение неопределенностей	9
Формализация задачи	13
Решение как построение цикла Дейкстры	17
Цикл Дейкстры	17
Алгоритмически конечная задача	18
Интересный пример	19
Еще одно важное обстоятельство – запись алгоритма	24
В заключение	27
ГЛАВА 2. Полный перебор и его оптимизация	28
Задачи, сводимые к перебору	29
Проблема комбинаторного взрыва	30
Главная мораль	47
ГЛАВА 3. Как свести решение к задаче существования	49
Главная идея	49
Задача поиска квадратного корня	50
Поиск отсутствующего числа	52
Решение уравнения Диофанта	58
ГЛАВА 4. Тождественные преобразования условий	64
Прежде всего необходимо убрать мусор из текста условия	64
Что делать после генеральной уборки	67
Задача о рекурсивной процедуре	67
Задача о бесконечном слове	69
Неопределенные уравнения	71
Расчет оптимального плана производства	73
Математическая модель задачи	74

Способ расчета выручки.....	74
Итак, где здесь геометрия?	75
Задача. Раскладывание колечек по штырькам	77
ГЛАВА 5. Моделирование физических процессов....	86
Модель движения системы тел в гравитационном поле	86
Задача о сложении прямого и отраженного колебаний.....	93
Задача о колебательном движении пружины	97
ГЛАВА 6. Несколько интересных задач	100
Задача Дейкстры.....	101
Задача о поиске пути с наибольшим весом.....	108
Задача о минимальном количестве заправок	113
Задача. Постфиксная и префиксная записи арифметического выражения	120
Прямая задача.....	120
Обратная задача	123
Задача. Самый длинный путь рубки	125
Задача. Одинокий путник с плохой памятью	131
Задача. Закраска односвязного контура.....	141
Обсудим некоторые алгоритмические идеи	142
Задача. Живая группа Го	152
Задача о черных пятнах на белой шкуре	156
В заключение	157
ГЛАВА 7. Практикум	158



ВВЕДЕНИЕ

Книга, введение к которой вы сейчас читаете, третья в моем авторском курсе. Две первые – «Современное программирование с нуля» и «Искусство алгоритмизации» – посвящены вхождению в творчество и науку программирования. Книга «Поиск решения» должна познакомить вас с одним из возможных ответов на вопрос «Что делать с нетривиальной задачей, если её решение нельзя прочесть в хорошей книжке?».

Сразу обращаю ваше внимание на одно важное обстоятельство. Программирование можно понимать как технологию сборки решения из более мелких задач. Такой подход создал индустрию программирования. И программирование можно понимать как искусство поиска решения логически сложной задачи, сведение которой к более простым не решает проблемы, потому как она либо просто не разбивается на хорошие задачи, либо эти более простые все равно представляют собой систему, связанную сложной логикой. Технологическому подходу будет посвящена четвертая книга моего курса. А сейчас мы стараемся посмотреть на программирование как на интеллектуальное искусство.

Главная идея книги – показать процесс мыслительной деятельности таким, какой он есть на самом деле, с ошибками, тупиковыми вариантами, рождением красивой идеи. И что, пожалуй, главное – показать возможность такой организации мышления, при которой поиск решения становится системной и плановой работой. Конечно, по прочтении книги у вас не будет рецепта построения решения. Единственный метод борьбы с творческими проблемами – развитый мыслительный аппарат, поэтому все главы книги – это описание процесса поиска решения реальных задач. И поиск этот не бессистемный, путь к решению в каждой задаче лежит не только через интуитивные прорывы и даже не столько через них, сколько через анализ накопленной информации.

Основной метод, действие которого проявляется на каждом шагу, – это метод борьбы с неопределенностями. Решение каждой за-

дачи представляется как последовательность вопросов «Что нам еще не ясно?» и «Как с этим бороться?». Возможно, некоторые из использованных задач искусственному «решателю» покажутся простыми, но цель максимально усложнить чтение не ставилась, поэтому задачи разноуровневые и поэтому книга может оказаться полезной для людей с самой различной степенью подготовки.

Базовый язык книги – Компонентный Паскаль, современная версия хорошо зарекомендовавшего себя языка, но некоторые решения записаны на псевдокоде. В принципе, базовый язык мог бы быть и другим, но все же Паскаль представляется наиболее удачным средством организации простого и в то же время достаточно строгого описания. Мой программистский и главным образом преподавательский, учительский опыт говорит, что императивный стиль письма более соответствует природе программистского мышления, чем функциональный или другие. Впрочем, даже если читатель не особенно приветствует императивные языки или язык Паскаль, то и такой читатель проблем с прочтением книги не испытает. Я уверен, что языковые особенности для большинства задач нашей предметной области не должны играть существенной роли.

P.S.

Обратите внимание на правила, выведенные в процессе решения, их немного, и они не составляют какой-либо завершенной теории. Создать систему правил можно, но это опасное занятие. Завершенная теория, скорее всего, привела бы не столько к повышению эффективности мыслительного процесса, сколько к ограничению ваших творческих возможностей. Поэтому правила, сформулированные во всех последующих главах, не носят характера предписаний, это скорее небольшие советы, корректирующие направление мышления.



ГЛАВА 1.

Как решается сложная задача

Существует один общий подход к поиску решения сложной задачи, независимо от того, из какой она области: математики, физики или программирования. Выражается он в трех простых предложениях:

1. Определим тип задачи.
2. Вспомним, какими методами нам или кому-нибудь другому доводилось решать задачи такого типа.
3. Попробуем применить эти методы к поставленной задаче.

Подход кажется логичным и разумным. Ведь большинство задач, с которыми мы сталкиваемся, кем-то уже решены, кто-то уже знает, как ответить на заинтересовавший вопрос, ответ уже есть в совокупной базе знаний человечества. Но накопленное общечеловеческое знание — это достаточно сложная штука. Оно не так доступно, как хотелось бы в силу своей огромности. Существуют и другие причины, по которым конкретный человек, сталкиваясь с реальной задачей, может не найти готового решения. Поэтому, несмотря на то что человечество в целом знает и умеет довольно много, отдельный человек часто встречается с ситуацией неопределенной и, следовательно, творческой. А в такой ситуации наш алгоритм из трех предложений уже не работает.

Кроме того, если взять действительно интересную задачу, то окажется, что определить, к какому типу она относится, довольно сложно. И часто задача будет относиться не к одному типу, а к нескольким. Например, это может быть задача комбинаторного характера с применением графов, или это может быть задача моделирования физических процессов с использованием графики и методов численной математики и т. д.

Если бы была возможна исчерпывающая классификация задач, с конечным количеством классов, четко отделимых друг от друга, то это бы

означало ограниченность программисткой науки. Программирование, как наука закончилось бы в момент написания последнего алгоритма на последний неразрешенный класс задач.

Вроде бы нет ничего страшного, просто в таких задачах мы имеем дело со сложными типами, и все равно можно действовать по той же схеме, то есть последовательно выполнять действия 1, 2, 3.

К сожалению, не получается. Во-первых, сложных типов можно сконструировать огромное количество, а чем типизация обширнее, тем сложнее выполнять пункт первый. Во-вторых, чем больше типов, тем сложнее в них ориентироваться, а в-третьих, тем сложнее отличить, где заканчивается один и начинается другой. Например, задача графического моделирования физического процесса, — это задача численной математики, физики или это графическая задача? В общем, при попытке составить какую-то классификацию мы столкнемся с таким количеством проблем, что невольно придет мысль поискать другой подход.

Попробуем подойти к сложным задачам с другой стороны. Начнем с небольшого, но очень важного замечания. Что бы мы не изобретали, все упирается в рождение идеи. А идея всегда рождается интуитивно, причем независимо от степени её гениальности и значимости. Происходит это так: вы некоторое время сосредоточенно размышляете на заданную тему, и вдруг приходит решение, причем не совсем понятно, откуда. Это называется интуицией. Существуют различные теории, объясняющие интуицию, откуда она берется, что она такое сама по себе, но для нас эти теории равным счетом ничего не значат, так как не дают метода мышления.

Все серьезные идеи рождаются интуитивно, что огорчает, так как совершенно не ясно, как интуицией управлять, но, с другой стороны, совершенно не подготовленный человек вряд ли сможет сформулировать красивую идею, что вселяет надежду. Ведь если для рождения эффективной идеи нужна подготовка, то, значит, интуиция где-то в своей основе содержит систему, какой-то метод, а методу можно научиться.

Что такой метод может из себя представлять? Можно ли описать его точно? Конечно, нельзя ожидать, что общий метод решения творческих задач будет иметь алгоритмическую ясность. Невозможно себе представить, что такой метод будет описанием последовательности действий. Хотя, конечно, многие люди, когда речь идет о методе, представляют некую последовательность инструкций, выполнив которые, можно получить верный результат. Но это глубокая, принципиальная ошибка.

Поэтому мы сразу и навсегда откажемся от идеи разработать такой всеобъемлющий алгоритм. Кроме того, мы решительно откажемся от попыток до конца понять тайну творчества. Это слишком невероятно, чтобы интуиция, творческий инсайт подлежал исчерпывающему логическому анализу. Это то, чего мы не будем делать. А попробуем разработать подход, помогающий удержать направление исследования и превращающий хаос творчества в осмысленный процесс.

Итак, мы ищем не методы решения задач, а методы организации мыслительной деятельности. Такова наша цель. И, чтобы её достичь, не будем строить теорию, а получим умения из практики. Решая задачи и отмечая то, что помогло получить решение, как-то обобщать свои наблюдения и, если получится, выводить общие правила.

Общие правила не должны иметь форму запретов, но надо четко понимать, что неограниченное творчество – это, по сути, хаос, из которого не появляется ничего продуктивного. Творчество творчеством, а любому исследователю необходимо получить приемлемое решение за ограниченное время. Поэтому дисциплина мышления должна ставить перед собой задачу, не создавая жестких запретов, тем не менее упорядочивать мыслительный процесс, нацеливая его на результат.

В этой книге большая часть глав посвящена конкретным проблемам, но есть несколько текстов общего значения, как, например, данная глава. А ниже мы рассмотрим подход, который, возможно, отражает самую общую идею поиска программистского решения и учитывает именно программистскую специфику такой работы.

Пошаговое уточнение неопределенностей

Рассмотрим пример несложной задачи. Пусть требуется разработать алгоритм расчета всех простых чисел, не превосходящих заданное число N . Если для решения не требуется особой эффективности, например верхняя граница не слишком велика, то можно воспользоваться самой очевидной идеей: алгоритм решения – это цикл, в теле которого для каждого очередного числа выясняется, простое оно или нет, и если простое, то число печатается как результат.

Для того чтобы принять решение относительно простоты очередного числа, необходимо проверить все числа, могущие быть его делителями, и если хотя бы один делитель есть, то проверяемое очередное не простое, иначе все-таки простое.

Идея вполне понятна, но все же вчитаемся в текст более внимательно, все ли здесь действительно хорошо. Один неясный пункт есть, это фраза, требующая проверить все числа, могущие быть делителями. Я из своего личного преподавательского опыта знаю, что ученики, недостаточно искушенные в задачах математической природы, о такие вещи спотыкаются довольно часто. «Как проверить?» – это и есть уточняющий вопрос, ответив на который, мы сможем записать решение в завершенной форме.

Заметим, что любое исследование начинается с искусства вопроса. Правильно заданный вопрос – это, образно говоря, половина ответа.

Рассмотрим более сложный пример. Есть группа людей со вполне определенными симпатиями и антипатиями друг к другу. Необходимо расставить их по рабочим позициям так, чтобы коллектив оказался психологически наиболее устойчивым. Возможно, психология дает какие-то методы решения подобных задач, но мы психологической наукой не владеем. А из соображений здравого смысла ясно, что необходимо организовать перебор всех возможных вариантов и выбрать из них вариант с наибольшим значением критерия устойчивости.

Это своего рода макроидея, или решение в первом приближении. Здесь сразу видны следующие неопределенности. Во-первых, что значит перебирать варианты? Ясно, что с людьми этого делать не получится, ни в одном языке программирования нет такого термина – «люди». Эту неопределенность мы устраняем легко, пронумеровав группу. Тогда перебор вариантов расстановки людей сводится к перебору всех возможных перестановок их номеров. Кстати, появившийся термин «перестановка» наводит на мысль поискать какой-нибудь уже известный алгоритм построения всех перестановок. Но здесь возникает новая неопределенность. Совершенно не факт, что наше понимание термина перестановка совпадает с математическим пониманием. В этом надо убедиться.

Следующая неопределенность сложнее. Это критерий устойчивости. Ясно, что для каждой полученной перестановки необходимо что-то считать, и это что-то должно характеризовать устойчивость построенного коллектива. Мы должны определиться с математической формой этого критерия.

Как известно, математика – это язык для точной записи знания, поэтому выше не зря появилось упоминание о математической форме записи кри-

терия. Любая работа по устранению неопределенностей в конечном итоге должна привести к математически строгим формулировкам.

Таким образом, данная итерация мыслительного процесса должна нам дать формулу расчета критерия и алгоритм, генерирующий перестановки. В общем-то можно начинать писать код. Но на этапе разработки алгоритма есть еще одна фундаментальная неопределенность – это требование к скорости работы будущей программы или требование к скорости работы алгоритма. Конечно, скорость программы и скорость алгоритма – несколько разные вещи. Хороший алгоритм можно испортить плохой программой, а вот плохой алгоритм хорошей программой не исправишь, поэтому вопросы эффективности – это прежде всего вопросы алгоритмизации.

В отношении поставленной задачи вопрос скорости очень актуален, мы пришли к необходимости использовать математическую конструкцию перестановок, а, как известно, для N элементов можно построить $N!$ перестановок. Это очень большая величина, и если коллектив содержит более 10 человек, чисто переборный вариант решения нас вряд ли устроит, и есть смысл подумать о более быстром алгоритме, использующем какие-то особенности формулировки задачи, или обдумать возможность отказа от идеального решения. То есть следующая итерация борьбы с неопределенностью должна дать ответ на вопрос о желаемой эффективности алгоритма.

Таким образом, процесс поиска решения заключается в записи решения, так как оно уже понято, выявлении неопределенностей и формулировке вопросов, ответы на которые устранят неопределенности. И весь процесс можно разбить на ряд итераций, для каждой из которых выполняется очередная запись решения, выявляются очередные неопределенности, формулируются очередные вопросы.

Заметим, что метод ничего не говорит о том, как будет организован поиск ответов на поставленные вопросы, это уже несколько иная проблема. Метод пошагового уточнения, – это общая форма организации мыслительного процесса, еще не гарантирующая успешности в частном деле поиска ответов на содержательные вопросы.

Важно заметить, что разбиение мыслительного процесса на фиксированные итерации довольно условно. Конкретная траектория поиска решения зависит от имеющейся системы знаний решателя, от степени развитости интуиции, от банальной удачи. Метод предлагает в общем-то простую вещь. Разбить процесс решения на этапы, в конце

каждого решателя должен провести анализ и четко понять, что уже ясно и какие проблемы перед ним стоят.

Таким образом, описываемый процесс уточнения самым прямым образом требует навыков рефлексии (анализа собственного состояния). Вы должны научиться постоянному контролю своего интеллектуального состояния. Внутри вашего мыслительного процесса должен постоянно работать маячок, фиксирующий обнаружение проблемы.

Набор проблем, – кстати, величина не постоянная. Появляющиеся идеи приближают окончательное решение не в арифметическом смысле, с каждым шагом, все теплее и теплее. Полученное промежуточное решение может поставить новую проблему. Например, в рассмотренном примере вывод о необходимости алгоритма построения перестановок ставит перед решателем сложнейшую проблему комбинаторного взрыва.

Усложнение решения – процесс неизбежный. Необходимо понимать, что решатель, приступая к задаче, скорее всего, не до конца понимает её характер и природу. Процесс решения приводит к более точному пониманию, а следовательно, к проявлению проблем, которые существовали, но не были видны решателю.

Еще одно важное замечание о порядке выбора подзадач. Предположим, что после очередной итерации мы имеем ряд проблем: *A*, *B*, *C*... и т. д. Какую из них выбрать для следующей итерации? Простая логика говорит, что ключевую, то есть такую, которая укажет магистральный путь для решения всей задачи. Наверное, это справедливо. Вопрос только в том, что, не решив задачу, нельзя сказать, какая проблема была главной, хотя опыт говорит, что интуитивно это как раз почти всегда ясно – ключевая проблема выглядит наиболее сложно.

Однако в этом вопросе есть важный психологический нюанс. Для успешной работы в задачу нужно погрузиться. Погружение, особенно для не вполне опытного исследователя, подразумевает хотя бы небольшой, локальный, но успех. Теперь представьте себе ситуацию. Вы приступаете к задаче и четко видите, что нужен ответ на сложный вопрос *A* и относительно простой вопрос *B*. Вы не сомневаетесь, что ответ на вопрос *A* будет ключом к решению, ответ на вопрос *B* имеет чисто технический характер. Простая логика говорит, что необходимо заняться вопросом *A*. Но, быть может, лучше решить технический вопрос *B*?! Вполне возможно, что его решение не очень важно, может быть, даже позже, получив ответ на вопрос *A*, вы поймете, что ответ на

вопрос *B* был не вполне удачным, но все-таки работа над вопросом *B* может дать вам именно то необходимое погружение в задачу, без которого не заработает ваш мыслительный аппарат. Это как разминка перед забегом. Она не приносит победы, но готовит мускулатуру к необходимому рывку.

Сказанное означает, что процесс поиска решения не сводится к чисто интеллектуальному действию, набору каких-то логических умозаключений. Решение ищет человек с определенными психологическими возможностями и особенностями, и это должно быть учтено.

Наконец, последнее, но не в последнюю очередь. Говоря о программировании, традиционно принято рассказывать о структурном программировании и нисходящем проектировании. Во второй книге моего курса, называемой «Искусство алгоритмизации», этим понятиям посвящена целая глава. Здесь же мы говорим о несколько другом. Обсудим различие. Структурное программирование – это парадигма, описывающая технически грамотное построение программы, мы же сейчас обсуждаем не программирование, а процесс поиска решения. Термин «Нисходящее проектирование» несколько ближе. Эта парадигма предписывает разбивать задачу на подзадачи, из решений которых потом будет конструироваться решение исходной задачи. Это ближе, но все же не вполне о нашей проблеме.

Принцип борьбы с неопределенностями предписывает разбивать на части не задачу, а процесс поиска решения, то есть речь, по большому счету, идет не о статической структуре задачи, а о динамичном мыслительном процессе.

Все последующие главы посвящены построению динамичного мыслительного процесса. Описанный выше подход, – это основа, фундамент. Далее мы постараемся выделить некий набор правил, помогающих определить интеллектуальную ситуацию и то, какими действиями её изменять в сторону большей определенности. Но это дальше, а пока глава еще не завершена, обсудим еще некоторые важные вещи.

Формализация задачи

Процесс формализации также можно рассматривать как этап уточнения задачи, но процедура формализации имеет особое значение и особый смысл подготовительного этапа к поиску решения. Пробле-

ма любой реальной задачи – в неопределенности терминологии как минимум, а иногда и в упущении некоторых условий. Строгое определение всех объектов, участвующих в условии задачи, обязательно уберет неточность терминов и, возможно, поможет выявить пропущенные условия.

Что мы имеем в виду, когда говорим о возможных неточностях? Проиллюстрируем проблему примерами. Например, в следующей формулировке «Дано множество объектов, найти такую их комбинацию, что....» скрывается очень грубая неопределенность, делающая условие полностью неясным. Комбинация – это термин, не имеющий однозначного смысла. Это могут быть перестановки, это могут быть размещения, сочетания. Процесс перебора комбинаций самым серьезным образом зависит от типа комбинации, поэтому в условии задачи необходимо прояснить, о каком типе комбинации идет речь, и переписать условие соответствующим образом, например так: «Дано множество объектов, найти такое их размещение, что....». Здесь формулировка уже более строгая, и при этом мы получили значительно больше, нежели просто понимание, мы получили метод решения, свойственный к алгоритму построения размещений.

Таким образом, формализация выполняет функцию уточнения условия, устраняя избыточность терминологии и создавая предпосылки поиска решения.

Далее, решение задачи – это запись некоторым набором терминов. Условие задачи – это также запись некоторым набором терминов. Для того чтобы работать с заданным условием, необходимо терминологическое соответствие между текстом условия и будущим решением. Известно, что любой язык программирования отличается очень небогатым набором выразительных средств в сравнении с языком естественным, что порождает ситуации очень серьезного смыслового разрыва, в результате которого задача, записанная на естественном языке, фактически не решается, пока мы не запишем её в терминах языка программирования.

В такую ситуацию постоянно попадают участники программистских олимпиад. Приведем несколько простых примеров.

Пример 1. В классе 30 учеников, между которыми есть как симпатии, так и антипатии, учитель поставил перед собой задачу рассадить учащихся так, чтобы создать в классе максимально комфортную психологическую обстановку.

У нас нет понятия «максимально комфортная психологическая обстановка», поэтому сразу договоримся, что речь идет о допустимой или недопустимой рассадке учеников в классе. Далее заметим, что ученические места в классе представляют собой красивый прямоугольник. Это означает, что неопределенный термин «рассадка учеников в классе» можно заменить более формальным «заполнение числовыми значениями прямоугольной матрицы». Термин «множество учеников» можно заменить термином «массив чисел», в котором числа могут играть роль номеров учащихся.

Пример 2. Морской берег имеет сложную конфигурацию, представляющую собой ломаную линию...

Не приводим формулировку полностью, она довольно длинная. Сказанного достаточно для иллюстрации проблемы. Здесь прозвучали два естественных термина: «морской берег», «сложная конфигурация». Если их заменить языковыми терминами, то может получиться, например, такая формулировка:

«Дан массив записей, содержащих два целочисленных поля, представляющих собой координаты вершин ломаной линии...»

В вышеприведенных примерах вопросы формализации решились достаточно легко. Мы просто подобрали термин алгоритмического языка, один в один подходящий для замены термина естественного. Но такой подход работает не всегда.

Пример 3. Дано число вида A^N ...

Не так важно, что требуется в этой задаче. Угрозы от прямой замены терминов видны невооруженным глазом. Если N достаточно велико, то целочисленных типов может оказаться недостаточно, а использование действительных типов повлечет за собой неточность в представлении числа, что может оказаться фатальным для решения. В этой задаче, прежде чем решать вопрос о формализации представления данных, необходимо проанализировать, какие должны быть выполнены операции.

Если речь идет о выполнении арифметических целочисленных операций, то может быть разумно представить число в виде массива или связного списка. Если речь идет об исследовании вопросов делимости, то, может быть, можно вообще обойтись без представления конечного числа, а лучше подумать, как связать свойства основания « A » со свойствами операции возведения в степень? В этом случае привычный целый тип окажется достаточным, а задача окажется более математической, нежели программистской.

Формализация исходных объектов, описанных в условии задачи, самым прямым образом связана с потребностями операций, которые предполагается выполнять над объектами.

Пример 4. Лиса на круглом закрытом поле пытается поймать зайца. Скорость зайца равна скорости лисы. Определить оптимальную стратегию для обоих.

Сходу ясно только то, что и лиса должна постоянно находиться в движении, и заяц может остановиться только в том случае, когда лиса не движется. Если копнуть глубже, то мы сразу упираемся в термин «стратегия». Что это такое и как это описать?

Наверное, стратегия, – это описание действий зайца и лисы, которых, кстати, легко формализовать парой координат на поле. Тогда стратегию поведения зайца можно описать кривой линией, при условии что траектория лисы известна. Но траектория лисы, по соображениям здравого смысла, зависит от траектории зайца. Выходит так, что траекторию зайца нельзя определить без траектории лисы, и наоборот, и получаем тупиковую ситуацию. Здесь есть два выхода из положения.

Выход первый. Описание поведения объектов в некоторой небольшой пространственной или временной окрестности.

Таким способом часто решают свои проблемы физики и математики. Например, зная значение производной от функции в точке, можно интегрированием восстановить внешний вид функции. Зная локальные характеристики газа в небольшом объеме, можно представить его поведение в большом сосуде. Если мы знаем скорость тела на малом участке пути и знаем, что никакие силы на него не действуют, то мы знаем, как оно будет двигаться дальше. Это намек на то, что, зная общее правило для принятия решения лисой и зайцем в конкретный момент времени, мы можем выстроить и траекторию. Вполне возможно, что у неё окажется какой-то очень простой вид. В этом случае решение задачи сводится к поиску такого точечного правила.

Выход второй. Все же возможно решение макрозадачи.

В условии было сказано, что для обоих необходимо построить оптимальную стратегию. Мы уже решили, что термин «стратегия» удачно заменяется термином «траектория». Далее, можно вполне обоснованно выдвинуть гипотезу, что при заданной форме поля существует только одна оптимальная стратегия, а следовательно, и только одна траектория. Справедливость этой гипотезы будет означать, что исходную задачу можно переформулировать так:

«Построить для преследуемого и преследователя такие траектории, что отступление от оптимума со стороны преследователя будет означать увеличение расстояния, а отступление со стороны преследуемого будет означать сокращение расстояния».

В некотором смысле можно сказать, что формализация задачи – это ответ на вопрос, что же на самом деле требуется получить в решении и как это должно выглядеть.

Еще можно сказать, что формализация задачи – это запись условия и процесса решения в виде, не допускающем многозначного толкования, что, в свою очередь, дает еще одно определение формализации, как запись, на формальном языке. В таком коротком параграфе дано сразу несколько пониманий термина «формализация», но, по большому счету, таким образом всего лишь показаны разные стороны одной вещи.

Решение как построение цикла Дейкстры

А сейчас позвольте сформулировать гипотезу относительно формы очень многих программистских решений. Сразу хочу заметить, что это не более чем моя личная гипотеза, поэтому не судите слишком строго, а звучит она так:

«Решение алгоритмически конечной задачи сводимо к построению цикла Дейкстры».

Здесь в соответствии с нашей же технологией рассуждений необходимо убрать две неопределенности. Во-первых, надо пояснить, что такое алгоритмически конечная задача, и, во-вторых, возможно, не все знают, что такое цикл Дейкстры.

Цикл Дейкстры

Э. Дейкстра в теории систематического вывода императивных программ ввел многоветочную форму цикла `while`, представляющую собой описание циклического процесса, управляемого несколькими условными операторами. В языке Компонентный Паскаль цикл Дейкстры можно реализовать безусловным циклом `loop`.

Пример:

```

LOOP
  IF условие THEN EXIT
  {ELSIF условие THEN группа операторов}
  .....
END;
END;

```

Фигурные скобки, ограничивающие условный оператор, означают возможность многократного повторения этой части. Цикл Дейкстры позволяет исключить необходимость вложенных циклов.

Таблица 1.1. Представление циклом Дейкстры вложенного цикла

Структура с вложенными циклами	Идентичная структура с циклом Дейкстры
<pre> FOR x:=1 TO N DO FOR y:=1 TO M DO Группа операторов оператор y:=y+1; END; END; </pre>	<pre> x:=1;y:=1; LOOP IF x>N THEN EXIT ELSIF y>M THEN x:=x+1; y:=1; ELSE Группа операторов оператор y:=y+1; END; END; </pre>

Указанное свойство цикла Дейкстры позволяет утверждать, что имеет место следующая гипотеза: один процесс (даже сложный) равнозначен одному циклу Дейкстры.

Алгоритмически конечная задача

Назовем таковой задачу, для которой можно точно описать исходный набор данных и конечный результат, также в виде некоторого набора данных. Заметим сразу, что системы управления чем-либо, системы обработки баз данных под данное определение не подходят, там важен процесс, начало которого, по большому счету исходными данными не определяется, а скорее важна временная точка, в которой программа начала работать. Нет и завершения, есть только некоторые промежуточные состояния.

Под наше определение попадают расчетные задачи, наверное, любого характера, а также задачи, цель которых – ответить на вопрос, справедливо некоторое утверждение или нет.

То есть алгоритмически чистая задача реализуется процессом, имеющим начало и завершение, процесс инициируется некоторым набом

ром данных и завершается некоторым набором. Любой процесс, как известно, состоит из ряда итераций, возможно имеющих сложную структуру, но выше мы выяснили, что сложная структура операций сводится к одному циклу Дейкстры, что, собственно, и требовалось показать.

Конечно, данный текст нельзя воспринимать как строгое доказательство, но такой цели и не ставилось, здесь изложена не более чем гипотеза, надеюсь, что правдоподобная.

Интересный пример

Конечно, очень сложно представить реальную задачу, укладывающуюся в какой-либо точно сформулированный принцип, но тем не менее хотелось бы показать более-менее содержательный пример, демонстрирующий сказанное. Такие примеры, без сомнения, есть, и ниже рассмотрим один из них. Это одна из моих любимых задач. Относительно простая, но с изюминкой, кроме того, представляющая широкий класс совершенно реальной проблемы построения фракталов. Попробуем написать программу, рисующую графический объект, изображенный на картинке ниже:

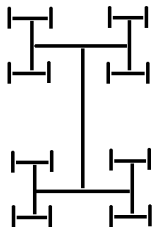


Рис. 1.1. Фрактальная снежинка

Простое наблюдение говорит о том, что здесь имеет место рекурсивный процесс, в ходе которого каждый нарисованный отрезок порождает еще два. Начинается процесс с некоторого исходного отрезка, на нашем рисунке с вертикальной линии определенной длины. Сказанное наводит на мысль о простейшем построении программы:

- рисуем отрезок;
- вызываем рекурсивную процедуру для нового отрезка;
- вызываем рекурсивную процедуру для нового отрезка.

Два совершенно одинаковых вызова – это не описка, а отражение того факта, что каждый отрезок порождает два, а отличий нет по при-

чине отсутствия какой-либо информации, позволяющей нам отделить один вызов от другого. Немного упростим запись:

```
Отрезок (x1, y1, x2, y2) ;  
Рекурсия () ;  
Рекурсия () ;
```

Два вызова обязаны отличаться списком передаваемых в них параметров, этот список и есть неопределенность, с которой сейчас начнем бороться. Проведя ряд рассуждений, построим алгоритм, решая на каждом шаге очередную небольшую проблему.

Рассуждение первое:

Очередной отрезок строится от конца предыдущего. Поэтому, даже не зная, как он будет строиться, мы должны прийти к заключению, что информация о координатах конца необходима. И получаем следующее уточнение:

```
Отрезок (x1, y1, x2, y2) ;  
Рекурсия (x1, y1) ;  
Рекурсия (x2, y2) ;
```

Рассуждение второе:

Очередной отрезок имеет длину меньшую, чем предыдущий. Это наводит на мысль, что необходимо передать длину предыдущего (иначе не вычислить меньший):

```
Отрезок (x1, y1, x2, y2) ;  
Рекурсия (x1, y1, y2-y1) ;  
Рекурсия (x2, y2, y2-y1) ;
```

Рассуждение третье:

Еще мы знаем, что отрезки на каждом шаге меняют свою ориентацию. Вертикальный отрезок порождает два горизонтальных, горизонтальный порождает два вертикальных. Весьма существенная информация для рекурсивной процедуры, рисующей отрезки, и, видимо, эту информацию необходимо передать. Договоримся, что единица будет означать вертикальную ориентацию, а двойка – горизонтальную, и получим очередное уточнение:

```
Отрезок (x1, y1, x2, y2) ;  
Рекурсия (x1, y1, y2-y1, 1) ;  
Рекурсия (x2, y2, y2-y1, 1) ;
```

Рассуждение четвертое:

Похоже, что информация о собственно отрезках исчерпана, но так как мы строим рекурсивную процедуру, то необходимо обдумать свойства рекурсивного процесса, хотя бы самые общие. Хотя бы то, что процесс должен иметь начало и завершение. Это справедливо для любого вычислительного процесса, но рекурсивный устроен достаточно нетривиально, и если его начало очевидно, то о завершении зачастую следует позаботиться.

В общем случае в теле процедуры должен отрабатываться какой-то критерий, позволяющий принимать решение о возможности нового вызова. Мы пока не имеем желания думать о механизме принятия этого решения, минимальная цель анализа – выяснить, какая необходима информация о рекурсивном процессе, для принятия такого решения. Самое простое – это строить критерий на какой-то величине, изменяемой от вызова к вызову. Таких величин две. Это номер вызова и длина отрезка. Таким образом, вызовы можно прекращать, когда номер вызова станет больше некоторого критического значения или длина отрезка станет меньше некоторого критического значения. Если договоримся, что критерий будет использовать номер вызова, то последнее уточнение приобретет следующий вид:

```
Отрезок (x1, y1, x2, y2) ;  
Рекурсия (x1, y1, y2-y1, 1, 0) ;  
Рекурсия (x2, y2, y2-y1, 1, 0) ;
```

Это стартовый код, который будет по-разному реализован в разных языках, без изменения смысла. Все содержательные проблемы здесь решены. Обратите внимание, что законченный фрагмент текста получен без формулировки какой-либо законченной идеи решения. Мы просто на каждом шаге формулировали и устраняли очередную неопределенность. И сейчас так же попробуем построить и процедуру «Рекурсия».

Рассуждение первое:

Очевидно, что её изначальный вид таков:

```
Рекурсия (x, y, длина, тип, номер)  
Начало  
Конец
```

А далее необходимо развернуть информацию, заключенную в передаваемых данных. Начнем с последнего, с номера вызова. Эта величина говорит о том, что некие операции (пока не знаем, какие) будут

выполняться, только если текущий номер вызова не превысит критического значения, и получаем первое уточнение:

```
Рекурсия (х, у, длина, тип, номер)
Начало
    Если номер < критического То
        Последовательность каких-то операций
Конец
```

Рассуждение второе:

Далее идет информация о типе. Помним, что типов отрезка два, это можно в коде отразить так:

```
Рекурсия (х, у, длина, тип, номер)
Начало
    Если номер < критического То
        Если тип = 1 То
            Последовательность операций
        Если тип = 2 То
            Последовательность операций
Конец
```

Рассуждение третье:

Следующая величина несет в себе информацию о длине отрезка. Мы знаем, что длина должна уменьшаться. Пусть она уменьшается вдвое, что можно записать так:

```
Рекурсия (х, у, длина, тип, номер)
Начало
    Если номер < критического То
        Если тип = 1 То
            Длина = Длина / 2
            Последовательность операций
        Если тип = 2 То
            Длина = Длина / 2
            Последовательность операций
Конец
```

Здесь сразу заметим, что в программе появился момент неоптимального построения вычислительного процесса. А именно длина вычисляется дважды. Наверное, этот оператор разумно вынести наверх, но необязательно сейчас. Необязательно строить идеальную программу за один проход. Кое-что допустимо оставить на этап оптимизации.

Рассуждение четвертое:

Осталась информация о координатах точки, от которой прорисовывается отрезок. Следовательно, можно теперь прорисовать отрезок с учетом того, вертикальная линия необходима или горизонтальная:

```

Рекурсия (х, у, длина, тип, номер)
Начало
    Если номер < критического То
        Если тип = 1 То
            Длина = Длина / 2
            Отрезок (х-длина, у, х+длина, у2)
            Последовательность операций
        Если тип = 2 То
            Длина = Длина / 2
            Отрезок (х, у-длина, х, у+длина)
            Последовательность операций
Конец

```

Рассуждение пятое:

А теперь вернемся к началу. Мы в первых строках выяснили, что каждый нарисованный отрезок порождает два вызова процедуры, и далее определили параметры для таких вызовов. Поэтому наш последний шаг:

```

Рекурсия (х, у, длина, тип, номер)
Начало
    Если номер < критического То
        Если тип = 1 То
            Длина = Длина / 2
            Отрезок (х-длина, у, х+длина, у2)
            Рекурсия (х-длина, у, длина, 2, номер+1)
            Рекурсия (х+длина, у, длина, 2, номер+1)
        Если тип = 2 То
            Длина = Длина / 2
            Отрезок (х, у-длина, х, у+длина)
            Рекурсия (х, у-длина, длина, 1, номер+1)
            Рекурсия (х, у+длина, длина, 1, номер+1)
Конец

```

Построение программы (на псевдокоде) полностью завершено. Обратите внимание, что мы выполнили всю работу, пошагово формулируя и устраняя неопределенности, не имея общего плана алгоритма. Такой подход, видимо, будет работать, если необходимо построить вычислительный процесс, управляемый небольшим количеством просто определяемых числовых параметров.

Еще одно важное обстоятельство – запись алгоритма

Любая задача – это, во-первых, содержательная проблема, а во-вторых проблема записи. Как записать решение, не все равно в любой предметной области, но в программировании задача записи приобретает особый смысл и значение, настолько особый, что для программистских решений приходится изобретать особые языки – посредники между машиной и программистом. Слово «посредник» по смыслу означает быть посередине. Но языки программирования на самом деле настолько содержательно бедны, по сравнению с языками естественными, что считать их посредниками можно только с некоторой натяжкой. Собственно, даже для профессиональных программистов зачастую требуется еще один уровень посредничества в виде блок-схем, различных вариантов псевдокода и т. д. Тем более проблема записи алгоритма в виде, более понятном, нежели запись на языке программирования, актуальна в деле обучения.

Чуть выше уже был детально разобран достаточно интересный пример с записью на псевдокоде. Заметим сразу, что сила псевдокода – отнюдь не в записи команд словами русского языка, а в более слабом формализме, что делает псевдокод более понятным для человека. Мы будем пользоваться псевдокодом и далее, поэтому было бы полезно если не дать его полное описание, то хотя бы обозначить некоторые основные идеи.

Величина. В любом языке программирования для величины есть понятие типа, необходимого для определения требуемого объема памяти и набора допустимых операций и имени величины. Так как псевдокод все равно не транслируется для исполнения на машине, то понятие типа можно опустить и предположить, что тип величины ясен из контекста её использования. Имя величины необходимо подбирать так, чтобы наиболее полно выразить её функциональное назначение. Разумно, при таком соображении, не жалеть символов и между коротким именем и более содержательным выбирать содержательное.

Процедуры. Понятие процедуры в псевдокоде существенно расширим. Мы справедливо полагаем, что промежуточная запись необходима для выделения смысловых блоков, расписать детально которые уже можно на конкретном языке программирования. Поэтому на

псевдокоде мы структурируем алгоритм до смысловых блоков, функция которых вполне ясна. Вот такие блоки и будут основной единицей записи. А её обозначением может стать словосочетание или даже короткое предложение, поясняющее функции блока. Возможно, эта смысловая единица все же нуждается в дополнительном описании, в этом случае её имя превращается в имя вспомогательного алгоритма, расположенного где-то в другом месте. При этом вполне можно описать в том или ином виде набор входных или результирующих данных. Например, так:

Расчет характеристики движения (координаты (x, y) , скорость, масса)

Из такой записи должно быть ясно, что где-то есть запись набора операций, вычисляющих некоторую характеристику пути, какую именно, должно быть ясно из текста алгоритма или условия задачи, а в скобочной записи разъясняется смысл передаваемых данных.

Циклические конструкции. В этом пункте также уйдем от излишнего формализма. От языка к языку синтаксис описания циклов может существенно изменяться, но для нас важны лишь два обстоятельства. Любой цикл задает последовательность итераций через указание множества, которое необходимо перебрать. Это множество может быть задано, например, арифметическим выражением, изменяющим величину — параметр от начального до конечного значения, причем эти значения могут быть как жестко заданной величиной, так и вычисляемым условием. И наиболее общая форма цикла будет выглядеть так:

Для всех значений A из множества B выполнить
Последовательность операций

Конструкция выбора. Здесь также возможны варианты синтаксиса, но независимо от формы суть конструкции — в выборе последовательности операций в зависимости от проверки условия, поэтому остановимся на простейшей форме, договорившись, что условные конструкции могут вкладываться друг в друга.

Если условие То первая последовательность операций
Иначе вторая последовательность операций

Заметим, что краткость и неформальность определения псевдокода очень важна. Если поставить перед собой задачу построить мощный инструмент описания, то по мере его детализации такой инструмент будет все меньше и меньше отличаться от языка программирования и

все больше и больше будет уходить главная функция – посредничество в передаче смысла между программистом и строгим языком.

Кроме того, у такого рода способов неформальной записи есть еще одна важная нетривиальная функция. А именно псевдокод важен не только как средство записи уже созданного алгоритма, но и как средство разработки, позволяющее записывать промежуточные идеи, структурировать запись того, что уже ясно. А это, в свою очередь, позволяет направлять мысль на дальнейшую работу. Грамотная запись помогает увидеть ошибки, дает возможность изменить план исследований, если текущие проблемы имеют критический характер. Таким образом, запись алгоритма на псевдокоде – это механизм управления мыслительным процессом, и, как полагается хорошему управленцу, он создает возможность для программиста все проблемы и узкие места алгоритма расшить до того момента, как он приступит к самому трудоемкому этапу своей работы – написанию программы, а как известно, ошибки этого этапа стоят наиболее дорого. Приведем пример разработки с записью на псевдокоде. Дано выражение:

$$f = \sum_{k=0}^n a_k x^k.$$

Вычислить его значение при заданных значениях k и x . Начнем с того, что это выражение вычисляется суммированием более простых выражений от 0 до n , что можно записать следующим образом:

```
Ввести все значения  $a_k$ 
Ввести значение  $x$  и  $n$ 
Сумма = 0
Для всех  $k$  от 0 до  $n$  выполнить
    Сумма = Сумма + Результат расчета очередного выражения
Печать значения Сумма
```

Из записи видно, что «Результат расчета очередного выражения» можно воспринимать как возврат значения функции «Расчет очередного выражения». Разработку текста этой функции можно вынести в отдельную запись. Сделаем это, учтя сразу, что на вход функция получает значение индекса k . Еще заметим, что фразу «Ввести все значения a_k » мы расшифровывать не будем. Это действительно сложная операция, включающая в себя циклическую конструкцию, но её смысл предельно ясен уже на этом этапе, и такую мелочь можно вполне оставить на этап записи на языке программирования, не загромождая псевдокода.

```
Расчет очередного выражения
  Вход: целое  $k$ 
Начало
  Расчет степени  $x^k$ 
  Возврат значения степени, умноженной на коэффициент  $a_k$ 
Конец
```

И наконец, распишем смысловой блок, обозначенный фразой «Расчет степени». Его уже не обязательно выделять как отдельную функцию.

```
Степень = 1
Для  $i$  от 1 до  $k$  выполнить
  Степень = Степень *  $i$ 
```

Конечно, это простой пример. Такого рода задачи большинство программистов сразу будет записывать на языке программирования, но это только пример. Вообще, что и насколько расписывать, какие фрагменты оставлять на этап программы, какие оформлять на псевдокоде, что описывать как смысловой блок, а что выносить в отдельную запись, решается каждым разработчиком индивидуально в зависимости от уровня личной квалификации, вкусов и привычного стиля.

В заключение

Таким образом, получив формулировку задачи, первое, что необходимо сделать, — это привести её в соответствии с имеющимся интеллектуальным инструментарием, записать в общепринятых математических терминах или терминах языка программирования и в процессе формализации точно понять, что от нас требуется на самом деле.

Затем решатель начинает процесс снятия неопределенностей в своем знании о задаче. Он исследует условие, выясняет, что ему в задаче ясно, что нет. На основе полученных неопределенностей формулирует вопросы, становящиеся отправными пунктами последующего исследования. И так до получения решения в окончательном виде.

Конечно, все сказанное выше — это не более чем каркас, на который навешиваются более тонкие и более содержательные методы и приемы. Их исследование и будет предметом всех последующих глав этой книги.



ГЛАВА 2.

Полный перебор и его оптимизация

В этой главе мы займемся классом проблем, требующих большого объема вычислительных ресурсов. Это комбинаторные задачи. Их решение заключается в переборе всех возможных комбинаций какого-то множества объектов и поиске их расположения, удовлетворяющего некоторым условиям. В принципе, перебор – процедура понятная, даже для не очень опытного решателя. И рассуждать здесь было бы не о чем, если бы не одно решающее обстоятельство. Зачастую этих возможных комбинаций настолько много, что даже простая по формулировке задача может потребовать столько вычислительных усилий, что её можно признать нерешаемой, если бы не вычислительная техника.

Поэтому очень интересно посмотреть, что возможно в такой ситуации. Вообще, необходимо заметить, что полный перебор – это самый простой подход к поиску решения и, наверное, самый естественный для человеческого ума. Задач, требующих перебора в той или иной форме, настолько много, что игнорировать метод нельзя. А сущность его в следующем: есть множество каких-то элементов, и необходимо найти их комбинацию, удовлетворяющую определенным условиям. Для чего требуются две вещи. Во-первых, необходимо определить правило построения комбинаций, и, во-вторых, нужен критерий, позволяющий установить соответствие комбинации условиям задачи.

Для построения правила требуется определиться с типом комбинации. Может быть, повезет и это будут либо сочетания, либо размещения, либо перестановки, то есть стандартные комбинации, для получения которых существуют готовые алгоритмы. Если не повезет и ситуация окажется несколько хитрее, то, скорее всего, сложности будут технического плана и комбинацию возможно построить из известных. Довольно часто в задачах перебора не требуются понятия комбинаторики, несложно привести пример, в котором достаточно

последовательно перебрать пронумерованное множество. Такой является задача о поиске наибольшего во множестве элементов, но это тривиальные, неинтересные случаи, и они не попадают в сферу нашего внимания. Наша цель – рассмотрение задач, для которых необходим комбинаторный перебор в различных вариантах. Рассмотрим несколько таких примеров.

Задачи, сводимые к перебору

Задача 1. Симбиоз на космическом корабле. Космический корабль должен перевести N организмов на Землю. Эти организмы относятся к M -видам. И так они устроены, что некоторые виды не могут находиться в непосредственной близости друг с другом. То есть задано множество пар вида (A, B) , где A и B – это организмы соответственно вида A и вида B , не могущие находиться рядом. Посадочные места на корабле представляют собой линейное множество. Необходимо расположить организмы таким образом, чтобы несовместимые организмы не оказались рядом, и если это невозможно, то выдать соответствующее сообщение. Организмов, естественно, не больше, чем посадочных мест, но может быть и меньше.

Решение. Пусть посадочных мест L , и предположим для общности, что $N < L$. Придумаем дополнительный нейтральный организм, способный ужиться с любым настоящим, и прибавим столько добавочных организмов, чтобы их общее количество стало равно количеству посадочных мест. После такой операции задача сводится к задаче получения всех перестановок на множестве из L мест и проверке каждой перестановки на «выживаемость организмов».

Задача 2. Задача о рюкзаке. Дан набор предметов различного веса и различной ценности (ценность в денежном выражении). Необходимо выбрать набор, такой что рюкзак сможет его выдержать, а в рюкзак можно уложить только вполне определенный вес и не больше, при этом набор должен иметь максимальную возможную ценность.

Решение. Так как речь идет о комбинации из части предметов, и при этом каждый предмет может участвовать в комбинации не более одного раза, то, очевидно, задача сводится к получению всех возможных сочетаний и, к проверке каждого из них на вместимость в рюкзак и сравнению стоимости нового сочетания с уже имеющимся наилучшим.

Проблема комбинаторного взрыва

К сожалению, для задач комбинаторного типа существует серьезная проблема, называемая проблемой комбинаторного взрыва. Рассмотрим для примера задачу о двух кучах камней.

Задача 3. О двух кучах камней. Дана большая куча камней, вес которых может быть самым разным. Требуется раскидать её на две кучи, так чтобы разность их весов была минимальной.

Решение. Здесь требуемая комбинация, – это сочетание. Действительно, одну из куч можно рассматривать как произвольное сочетание, тогда вторая – это то, что осталось. Критерий проверки очевиден, алгоритм получения всех возможных сочетаний известен. Но насколько реально перебрать все сочетания?

Количество сочетаний из n элементов по m экземпляров вычисляется по следующей формуле:

$$C_n^m = \frac{n!}{m!(n-m)!}.$$

Число достаточно большое. А общее количество всех сочетаний из n элементов равно 2^n . Количества иных комбинаторных конструкций выражаются не меньшими значениями. Например, количество перестановок из N элементов равно факториалу числа N . И это лишает нас возможности написать программу, запустить которую на выполнение имеет смысл. Работать такая программа будет, но шансов дождаться результата даже для небольшого набора данных немного. Если, например, в задаче о двух кучах дано всего лишь 100 камней, что очень немного, то общее количество сочетаний выражается астрономическим числом 2^{100} . Но совершенно безвыходные положения встречаются не слишком часто. Если речь идет о реальной прикладной задаче, то заметим, что:

Не всегда требуется идеальное решение. Довольно часто возможен компромисс в отношении качеств программы. А если согласиться на некоторую потерю качества, то проблему комбинаторного взрыва можно обойти.

Если в задаче о двух кучах камней мы готовы принять не наилучшее решение, а вполне разумное, то становится возможен следующий алгоритм:

Упорядочим камни исходной кучи в порядке возрастания
Пока в исходной куче есть камни
 Возьмем самый тяжелый (Очередной)
 Если Левая куча тяжелее Правой, то Очередной положим в Правую
 Иначе в Левую кучу.

Алгоритм удивительно простой. К сожалению, он не дает идеального решения во всех возможных ситуациях, но, что интересно, есть класс наборов данных, на котором все же получается идеальное решение, и во многих иных ситуациях получаемое решение не такое уж плохое.

Проблема для самостоятельного исследования

Постройте пример, для которого алгоритм находит идеальное решение. Затем постройте пример, для которого алгоритм не дает идеального решения, и затем попробуйте сформулировать определение класса ситуаций, для которых алгоритм срабатывает.

Подобный подход к решению задачи называется эвристическим. Закрепим это правилом:

Эвристика – это основанное на опыте правило, существенно ограничивающее поиск решения в сложной задаче. Эвристика не гарантирует оптимальность полученного решения, полезная эвристика предлагает варианты, которые с высокой степенью вероятности оказываются хорошими или, точнее, достаточно хорошими. Соответственно, эвристический алгоритм – это алгоритм, использующий эвристику.

Мы и далее будем исследовать эвристические алгоритмы, а сейчас рассмотрим еще одну возможность ограничения перебора. Вопрос: если некоторое количество комбинаций уже проанализировано, то нельзя ли, используя полученную информацию, некоторые из последующих комбинаций исключить из рассмотрения?

Вполне здравая идея. Действительно, метод полного перебора исходит из того соображения, что нет никакой информации, позволяющей сделать вывод о будущей еще не построенной комбинации, поэтому то и надо перебирать все. А если информация есть, то, наверное, это что-то меняет. Подумаем, что именно.

Для того чтобы избавиться от процедуры перебора, нужна закономерность. Например, для того чтобы выяснить, присутствует ли заданное число A в некоторой арифметической прогрессии, нет необходимости перебирать все элементы прогрессии, достаточно установить, можно ли число A представить в виде: $A = a_1 + d(n-1)$. Указанная формула и есть искомая закономерность. Поэтому в борьбе с перебором проведите анализ вашей

структуры данных и посмотрите, можно ли в отношении этой структуры сформулировать какие-нибудь точные утверждения. Если да, то, может быть, эти точные утверждения позволят сократить перебор.

Попробуем развернуть идею. Для начала совсем тривиальный пример. Пусть дано ведро яблок и требуется выяснить, есть ли среди них одно гнилое. Можно поступить так: выложим все ведро и просмотрим каждое яблоко. Таким образом, ответ на вопрос будет получен гарантированно, но придется выполнить массу лишней работы. Можно поступить несколько иначе. Будем выбирать из ведра яблоки по одному, до тех пор, пока не встретим хотя бы одно гнилое. После этого даем утвердительный ответ и работу прекращаем. Отрицательный ответ, естественно, возможен только после перебора всего ведра. Сколько при таком подходе будет выполнено операций? Как минимум одна, если уже первое яблоко окажется гнилым, но это маловероятно. Как максимум придется перебрать все ведро, но интуитивно ясно, что, скорее всего, придется перебрать половину ведра (если гнилое только одно). Таким образом, с большой долей вероятности половину операций можно будет отбросить.

Описанная идея имеет неэвристический характер. Ясно, что ни одна из комбинаций не отбрасывается в действительности. Просто перебор прекращается после того, как некоторый критерий сообщит, что решение уже получено. Идея допускает усиление. Любой перебор можно представить как дерево вариантов. В виде такого дерева можно представить и перебор вариантов в задаче о рюкзаке. Для рюкзака каждый узел содержит информацию об уже уложенных предметах. Переход к новому узлу – это укладка в рюкзак еще одного предмета. Тогда ясен и критерий обрезки ветвей дерева. Если в некотором узле достигнут максимум веса, то дальше строить дерево нет смысла, и можно выполнить подъем на узел вверх.

Для задачи о двух кучах камней узел дерева – это три кучи: левая, правая и оставшиеся камни. Критерий обрезки ветвей будет такой: спуск вниз по дереву не имеет смысла, если сумма весов одной из куч (левой или правой) и оставшихся камней меньше либо равна весу другой кучи (правой или левой). Если такое случилось, то все оставшиеся камни надо просто положить в меньшую кучу, и в этом направлении построение дерева завершено. Естественно ожидать, что для реализации такого подхода не все равно, как строятся комбинации. Например, в задаче о рюкзаке необходимо, чтобы дочернее сочетание (стоящее по ветке ниже) было тяжелее родительского. Требова-

ние утяжеления дочернего сочетания означает, что каждое сочетание должно строиться из предыдущего добавлением одного элемента из оставшихся. Пожалуй, на задаче об укладке рюкзака мы и покажем метод в деталях.

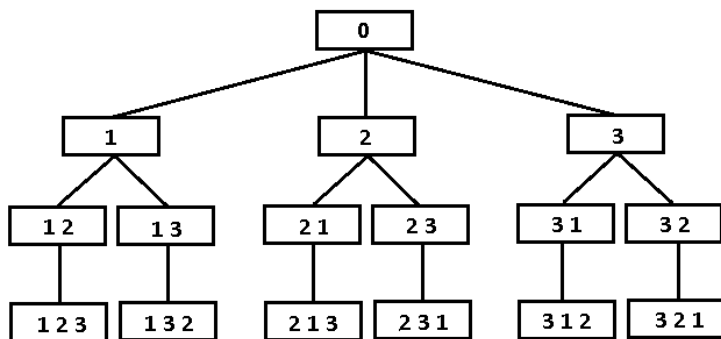


Рис 2.1. Дерево перебора для задачи о рюкзаке

Начнем построение реализации задачи о рюкзаке. Для начала определимся со структурой данных, описывающей складываемые вещи:

```
ТипВещь=RECORD
    вес, цена: INTEGER;
END;
```

С помощью описанного типа мы создадим массив вещей:

```
Вещь: ARRAY 10 OF ТипВещь;
```

Общая идея. Запустим процесс построения сочетаний из вещей. Пусть очередное сочетание – это и есть набор, складываемый в рюкзак. Очередное сочетание является допустимым, если суммарный вес сочетания не превышает допустимого. Величину допустимого веса вводим из потока. Если сочетание является допустимым, то выясняем стоимость сочетания и сравниваем полученную величину с уже найденным максимальным значением. Если новая сумма больше имеющегося максимального, то, соответственно, значение максимума стоимости изменяется на новое. Таким образом, главный цикл – это цикл построения сочетаний.

Еще раз вспомним очень важный момент. Сочетание необходимо строить таким образом, чтобы новое сочетание получалось утяжелением уже построенного (родительского). Таким образом, каждое

сочетание определяется через сочетание меньшей длины, а это рекуррентное определение. Отсюда следует, что разумно было бы строить процесс рекурсивно, начиная с нулевой длины, перебирая на каждом вызове все возможные варианты утяжеления. Главная процедура может выглядеть следующим образом:

```
BEGIN
    StdLog.Clear;
    In.Open;
    /Вводим набор предметов./
    In.Int(Количество);
    FOR k:=0 TO Количество-1 DO
        In.Int(Вещь[k].вес); In.Int(Вещь[k].цена);
        Набор[k]:=0;
    END;
    In.Int(Вес); /Максимально допустимый вес./
    /Максимальная сумма уложенных предметов до начала процесса./
    Сумма:=0;
    Рюкзак(0); /Запуск рекурсивного процесса./
    /Распечатка результатов./
    StdLog.String('Сумма = '); StdLog.Int(Сумма); StdLog.Ln;
    StdLog.String('Список : '); StdLog.Ln;
    FOR k:=0 TO Количество-1 DO
        IF НаборРезультат[k]=1 THEN
            StdLog.Int(Вещь[k].вес); StdLog.Int(Вещь[k].цена);
            StdLog.Ln;
        END;
    END;
END;
END Главная;
```

Еще немного поясним структуры данных. В нашей реализации предполагается несколько глобальных структур. Это ограничение по весу (переменная *Вес*), найденная максимальная стоимость (переменная *Сумма*), конечно, массив предметов и их общее количество (переменная *Количество*) и два вспомогательных массива: массив *НаборРезультат*, хранящий набор, имеющий максимальную стоимость, и массив *Набор*, используемый для хранения всех промежуточных наборов. Все эти величины можно передавать в рекурсивную процедуру как формальные параметры, но можно оставить их глобальными, в принципе, это вопрос стиля.

Займемся построением рекурсивной процедуры *Рюкзак*. Для начала ограничим проблему. Выше уже рассказывалось о способе ограничения полного перебора, но мы пока откажемся от каких-либо идей оптимизации. Напишем реализацию полного перебора, так сказать, лишь бы работало.

Это, между прочим, традиционный программистский подход. Если встречается сложная задача, то её можно упростить, решить в ограниченном варианте, затем вернуться к исходной формулировке и решить её в полном объеме, что-то дополнив к ограниченному варианту.

Конечно же, такой подход не всегда дает результат. Если упрощенный вариант реализован неудачно, то доделка может оказаться слишком сложной, а игра на упрощение обратится усложнением. К сожалению, начиная разработку, невозможно сказать, получится упрощение удачным или нет. Поэтому если вы в состоянии спроектировать сразу исходную задачу, то упрощать нет необходимости, если же хороших идей в отношении полной задачи нет, то есть смысл обратиться к стратегии упрощения. В любом случае, если упрощение не даст непосредственного результата, то вы хотя бы получите более точное понимание задачи. А в первой главе более точное понимание условия упоминалось как важнейший инструмент борьбы с неопределенностями.

Вернемся к процедуре Рюкзака. На вход она получает длину уже построенного сочетания. Сочетание хранится в массиве Набор. Единица массива означает, что соответствующий предмет в сочетание входит, нуль, соответственно, означает, что предмет не входит. Очередная активация процедуры что-либо делает только в том случае, если длина сочетания не достигла максимально возможного значения. Это записывается следующим оператором:

```
IF номер<=Количество-1 THEN
    /Перебор вариантов сочетания и их анализ./
END;
```

Здесь номер – это номер позиции в массиве предметов, до которого сочетание уже построено. Переменная Количество, если быть точным, означает номер последнего элемента, а так как нумерация предметов начинается с нуля, то последний номер – это Количество-1.

Если номер меньше своего максимально возможного значения, то, очевидно, достроить сочетание можно несколькими способами, общее описание которых можно уложить в цикл:

```
IF номер<=Количество-1 THEN
    FOR k:=номер TO Количество-1 DO
        /Очередная попытка достройки сочетания./
    END;
END;
```

Очередная попытка заключается в присвоении элементу массива `Набор` одной единицы. После чего новое сочетание считается достроенным и можно выполнить его анализ. Ниже листинг тела цикла:

```
Набор[k]:=1;
вес:=0; сумма:=0;
FOR j:=0 TO Количество-1 DO
    IF Набор[j]=1 THEN
        вес:=вес+Вещь[j].вес;
        сумма:=сумма+Вещь[j].цена;
    END;
END;
IF (вес<=Вес) & (сумма>Сумма) THEN
    НаборРезультат:=Набор;
    Сумма:=сумма;
END;
Набор[k]:=0;
```

Самое первое присвоение, – это и есть операция достройки сочетания. После чего необходимо посчитать его вес и стоимость, что и выполняется последующим оператором цикла. Условный оператор

```
IF Набор[j]=1 THEN
    вес:=вес+Вещь[j].вес;
    сумма:=сумма+Вещь[j].цена;
END;
```

учитывает то обстоятельство, что не все предметы исходного набора участвуют в сочетании. После того как суммарный вес и суммарная стоимость посчитаны, остается выяснить, не превышает ли суммарный вес критического значения, и если нет, то является ли новая стоимость более выгодной, чем уже найденный максимум.

```
IF (вес<=Вес) & (сумма>Сумма) THEN
END;
```

Если оба условия выполняются, то есть смысл запомнить сумму как новый максимум и новый набор предметов. Эти два действия и реализуются операторами в теле условной конструкции:

```
НаборРезультат:=Набор;
Сумма:=сумма;
```

И наконец, последний оператор `Набор[k]:=0`; инициализирует проанализированное сочетание, для того чтобы перейти к следующему. В этом листинге не хватает рекурсивного вызова. Его место очевидно после завершения анализа очередного сочетания, и полный листинг цикла будет следующим:

```

FOR k:=номер TO Количество-1 DO
  Набор[k]:=1;
  вес:=0;сумма:=0;
  FOR j:=0 TO Количество-1 DO
    IF Набор[j]=1 THEN
      вес:=вес+Вещь[j].вес;сумма:=сумма+Вещь[j].цена;
    END;
  END;
  IF (вес<=Вес) & (сумма>Сумма) THEN
    НаборРезультат:=Набор; Сумма:=сумма;
  END;
  Рюкзак(k+1);
  Набор[k]:=0;
END;

```

Все, на этом простой вариант задачи можно считать полностью решенным. Пришло время подумать об оптимизации перебора. Построенная реализация позволяет проблеме оптимизации решить очень просто. Новая активация процедуры Рюкзак имеет смысл только в том случае, если вес очередного набора не превышает критического, и тогда активация процедуры Рюкзак будет выглядеть следующим образом:

```
IF вес<Вес THEN Рюкзак(k+1); END;
```

На этом построение реализации можно считать завершенным. Осталось привести полный листинг решения.

```

MODULE Модуль;
  IMPORT StdLog, In;
  PROCEDURE Главная*;
  TYPE
    ТипВещь=RECORD
      вес,цена:INTEGER;
    END;
  VAR
    Вещь:ARRAY 10 OF ТипВещь;
    Набор,НаборРезультат:ARRAY 10 OF BYTE;
    k,Количество,Сумма,Вес:INTEGER;
  PROCEDURE Рюкзак(номер:INTEGER);
  VAR
    k,j,вес,сумма:INTEGER;
  BEGIN
    IF номер<=Количество-1 THEN
      FOR k:=номер TO Количество-1 DO
        Набор[k]:=1;
        вес:=0;сумма:=0;
        FOR j:=0 TO Количество-1 DO
          IF Набор[j]=1 THEN

```

```

        вес:=вес+Вещь[j].вес;сумма:=сумма
        +Вещь[j].цена;
    END;
END;
IF (вес<=Вес) & (сумма>Сумма) THEN
    НаборРезультат:=Набор; Сумма:=сумма;
END;
IF вес<Вес THEN Рюкзак(k+1); END;
Набор[k]:=0;

END;

END Рюкзак;
BEGIN
    StdLog.Clear;
    In.Open;
    In.Int(Количество);
    FOR k:=0 TO Количество-1 DO
        In.Int(Вещь[k].вес);In.Int(Вещь[k].цена);
        Набор[k]:=0;
    END;
    In.Int(Вес);
    Сумма:=0;
    Рюкзак(0);
    StdLog.String('Сумма = ');StdLog.Int(Сумма);StdLog.Ln;
    StdLog.String('Список : ');StdLog.Ln;
    FOR k:=0 TO Количество-1 DO
        IF НаборРезультат[k]=1 THEN
            StdLog.Int(Вещь[k].вес);StdLog.Int(Вещь[k].
            цена);StdLog.Ln;
        END;
    END;
END Главная;
END Модуль.

```

Задачей о рюкзаке мы проиллюстрировали метод ограничения перебора, именуемый бектрекингом. Еще раз повторим его сущность: есть некий процесс, его цель – найти оптимальное значение какой-то величины. Процесс построен таким образом, что на каждой итерации мы приближаемся к оптимуму, но может так случиться, что на очередной итерации оптимум будет пройден, и последующая работа алгоритма приведет только к ухудшению ситуации. В подобном случае эту счетную веточку (процесс предполагается ветвящимся, иное не интересно) продолжать нет смысла.

Вот так в чистом виде метод применим не всегда, но главная идея имеет очень большую степень общности. Например, идея применима к задаче о двух кучах камней (условие выше). Решается задача перебором сочетаний. При этом одна куча – это сочетание, вторая – это

все оставшиеся камни. Если сочетание строить так, что каждое из них будет получено из уже имеющегося добавлением одного камня, то бектрекинг применим по той же схеме. А именно как только будет получено сочетание, вес которого больше веса оставшихся камней, процесс построения можно завершить, так как это сочетание уже не может быть источником для других сочетаний.

Проблема для самостоятельного исследования

Напишите переборную программу для решения задачи о двух кучах камней и оптимизируйте её бектрекингом.

В задаче о рюкзаке бектрекинг срабатывал по превышении критической массы. В задаче о двух кучах камней схема, в общем-то, такая же. Мы считаем некоторую числовую величину, и когда оказывается, что её увеличение в любом случае ухудшает ситуацию, – это сигнал для возврата по дереву перебора. Необходимое условие для организации бектрекинга здесь – это увеличение характеристической величины на каждой итерации. Такой принцип построения бектрекинга не единственно возможный.

Например, проанализируем с этой позиции задачу о перевозке организмов на космическом корабле. Задача чуть выше уже была рассмотрена, мы уже знаем, что её решение сводится к получению всех возможных перестановок и поиску среди перестановок допустимой. Здесь трудно представить числовую характеристическую величину, так речь идет о структуре данных, именно структура здесь может быть допустимой или нет. Таким образом, необходимо построить такую последовательность перестановок, что:

- каждая перестановка должна быть получена из некоторой другой, следовательно, все перестановки делятся на пары (материнская, дочерняя). Каждая материнская может быть источником для нескольких дочерних;
- если материнская перестановка является недопустимой, то недопустимыми должны быть и все дочерние и дочерние дочерних и т. д.

Проблема для самостоятельного исследования

Попробуйте разработать метод построения перестановок, удовлетворяющий описанным условиям. Вполне возможно, что такого метода и нет, но попробовать можно.

Мы пойдем немного другим путем. Попробуем сформулировать задачу в немного другой терминологии.

Необходимо понимать, что структура данных – это не просто хранилище для чисел, литер, строк и т. д. Структура данных несет в себе какие-то идеи, принципы и может существенно влиять на процесс построения решения. Поэтому переформулировка задачи в немного других терминах может дать радикальный эффект. То, что не было видно раньше, станет очевидным, тяжелые технические проблемы упростятся и т. д. Конечно, это не обязательно так. Переход к другому понятийному аппарату может ничего не дать и даже усложнить поиск, но иногда помогает найти по-настоящему удивительное решение.

Решение задачи заключается в размещении организмов в определенной конфигурации посадочных мест (в нашем случае эта конфигурация линейная). Существует одноименная комбинаторная структура – «размещения». Посмотрим, что эта структура может дать.

Размещения, в отличие от сочетаний, отличаются позицией размещаемых элементов, а в отличие от перестановок не обязательно содержат все элементы исходного множества. Это означает, что размещение можно достраивать. Для множества размещений можно определить материнское и дочерние размещения, так что дочернее размещение будет отличаться от материнского ровно одним добавленным организмом.

Попробуем сформулировать главную идею. Самое первое размещение строится посадкой одного организма на какое-либо место. Пусть организмов N штук, а посадочных мест M . Тогда размещений из одного организма можно построить $N \cdot M$. Но не это важно, это небольшое замечание о количестве, позволяющее понять, как много возможно вариантов комбинаций. Достроить каждое размещение можно множеством способов. Но некоторые из них приведут к противоречию, в том смысле, что дополнительный организм окажется несовместим с уже посаженными, такие размещения уже не смогут стать материнскими для других, и это будут точки отсечения ветвей перебора. Построим реализацию, используя тот же прием, который мы применили в задаче о рюкзаке, то есть первым шагом получим реализацию, строящую все возможные размещения из M элементов на N позиций. Вторым шагом найдем точку, в которой дерево перебора ветвится, и реализуем проверку на ограничения задачи.

Выше уже дважды был обозначен очень интересный подход. Если задача допускает переборное решение, которое возможно ограничить, то есть смысл не бросаться сразу в борьбу за красивое решение. Можно реализовать переборный вариант, он наверняка будет существенно проще, от-

ладить его на небольших примерах, затем определить точки ветвления и в них описать процедуру проверки ограничительных условий.

Таким образом, пока забываем про бектрекинг и реализуем задачу получения размещений. Кстати, сейчас вообще можно забыть о задаче рассадки организмов и в чистом виде решить задачу построения размещений. Это действительно так. Условие, реализующее бектрекинг, будет отсеивать негодные размещения, а это то, что нужно по условию задачи, просто отсеиваться они будут не после полного построения размещения, а несколько раньше, сразу по обнаружении несовместимых организмов.

Задачу построения размещений можно свести к двум задачам. Все возможные размещения получаются, если перебрать все возможные сочетания и для каждого сочетания все возможные перестановки. Но уже было замечено, что для обрезки дерева перебора перебор должен быть построен специальным образом, и перестановки для этой цели не особенно подходят.

Попробуем построить размещения, минуя конструкции сочетаний и перестановок. Тем более что в этом, собственно, нет ничего сложного. Для того чтобы построить размещение из одного элемента, достаточно посадить этот элемент (в нашем случае организм) на все возможные места. Каждая такая посадка и будет начальное размещение. Если в исходном множестве два элемента, то строим множество размещений из одного элемента, затем каждое такое размещение порождает множество размещений из двух элементов посадкой второго организма на свободные места. Аналогично для трех, четырех и т. д.

Можно построить рекурсивную процедуру, каждая активация которой будет перебирать варианты размещений очередного элемента исходного множества. Для такой процедуры исходная информация – это номер очередного размещаемого элемента. А её тело – это, видимо, цикл, перебирающий варианты размещений. Договоримся об обозначениях:

- множество – исходный набор элементов;
- размещение – это структура, являющаяся целью всех наших построений;
- очередной – номер очередного элемента, участвующего в размещении;
- длина – длина размещения, для нашей задачи это количество посадочных мест на корабле;

- количество – количество элементов исходного набора, для нашей задачи количество организмов.

Блок ввода информации обсуждать не будем, его можно посмотреть в конечном листинге. Займемся логикой размещающей процедуры. На вход процедура получает только номер размещаемого элемента, поэтому заголовок опишем так:

```
PROCEDURE ДобавитьЭлемент (Очередной:INTEGER) ;
BEGIN
END ДобавитьЭлемент;
```

Так как процедура рекурсивная, то в первую очередь необходимо описать условие её работы (или, наоборот, условие завершения). Это условие естественным образом вытекает из единственного формального параметра. Очевидно, что процедура завершает свою работу, если построено размещение, в котором участвуют все организмы. Если таковое размещение получено, то активации прекращаются и можно выполнить распечатку полученного размещения.

```
PROCEDURE ДобавитьЭлемент (Очередной:INTEGER) ;
VAR
    k, j, i:INTEGER;
BEGIN
    IF Очередной<Количество THEN
        /Построение размещения продолжается./
    ELSE
        FOR i:=0 TO Длина-1 DO StdLog.Int(Размещение[i]); END;
        StdLog.Ln;
    END;
END ДобавитьЭлемент;
```

Достройка размещения – это попытка поставить очередной элемент на все имеющиеся пустые места, вот так:

```
FOR j:=0 TO Длина-1 DO
    IF Размещение[j]=0 THEN
        Размещение[j]:=Множество[Очередной];
    END;
END;
```

Полученный цикл дает все возможные размещения длиной на единицу больше, чем размещение, полученное в предыдущей активации. Так как, возможно, эти размещения не являются конечными, необходимо выполнить новую активацию, сообщив ей номер следующего элемента.

```
FOR j:=0 TO Длина-1 DO
    IF Размещение[j]=0 THEN
```

```

Размещение[j]:=Множество[Очередной];
ДобавитьЭлемент(Очередной+1);
Размещение[j]:=0;

END;

```

Обратите внимание на оператор: Размещение[j]:=0; это обязательно. Перебор размещений предполагает, что мы сажаем организм на определенное место, затем убираем его и пробуем посадить на другое. Такова логика процедуры построения размещений, логика других частей программы для понимания алгоритма роли не играет, поэтому далее полный листинг:

```

MODULE Модуль;
    IMPORT In, StdLog;
    PROCEDURE ПостроениеРазмещений*;
    VAR
        Множество, Размещение: ARRAY 10 OF INTEGER;
        Количество, Длина, k: INTEGER;
    PROCEDURE ДобавитьЭлемент(Очередной: INTEGER);
    VAR
        k, j, i: INTEGER;
    BEGIN
        IF Очередной < Количество THEN
            FOR j:=0 TO Длина-1 DO
                IF Размещение[j]=0 THEN
                    Размещение[j]:=Множество[Очередной];
                    ДобавитьЭлемент(Очередной+1);
                    Размещение[j]:=0;
                END;
            END;
        ELSE
            FOR i:=0 TO Длина-1 DO StdLog.Int(Размещение[i]); END;
            StdLog.Ln;
        END;
    END ДобавитьЭлемент;
    BEGIN
        In.Open;
        StdLog.Clear;
        In.Int(Количество); In.Int(Длина);
        FOR k:=0 TO Количество-1 DO In.Int(Множество[k]); END;
        FOR k:=0 TO Длина-1 DO Размещение[k]:=0; END;
        ДобавитьЭлемент(0);
    END ПостроениеРазмещений;
END Модуль.

```

Но это листинг не нашей задачи. Программа дает все возможные размещения, и только. А теперь главное, попробуем достроить этот текст до решения исходной задачи о размещении организмов. Необходимо каким-то образом в исходную задачу добавить проверку, яв-

ляется ли полученное размещение допустимым, для чего потребуется прежде всего договориться о том, как ввести информацию о совместимости организмов. Это можно реализовать различным образом, например составить массив двухкомпонентных записей, компоненты которых – номера совместимых организмов. Можно для каждого организма создать массив с номерами совместимых.

От выбора структуры данных зависит если не логика алгоритма, то, во всяком случае, структура программы, поэтому к выбору структуры необходимо подойти максимально ответственно. Тем более что существенная часть программы уже написана, и её не хотелось бы сильно переделывать.

Почему важно спроектировать решение сразу? Тот путь, которым мы пошли, позволил разбить решение на более простые и более понятные этапы, что, наверное, должно упростить работу. Но такое разбиение означает, что мы поделили на этапы и процесс принятия ответственных решений. Может получиться так, что хорошее решение, принимаемое на более позднем этапе, вступит в противоречие с уже построенными конструкциями. Тогда придется либо полностью ломать сделанное, либо отказаться от более оптимального метода в пользу совместимости со сделанным.

Заметим, что возможность для двух организмов находиться рядом можно интерпретировать как ребро графа, если сам организм рассматривать как вершину. Эта аналогия наводит на мысль представить все возможности рассадки организмов как граф. Это дает и необходимую структуру данных – матрицу смежности.

Тогда проверка, возможно некоторое соседство или нет, сводится к проверке ячейки матрицы, находящейся на пересечении соответствующей строки (один организм) и соответствующего столбца (другой организм). Если в ячейке единица, то соседство возможно, иначе нет. Договоримся о новой структуре данных:

```
VAR Матрица: ARRAY 10,10 OF INTEGER;
```

Это и будет матрица смежности, содержащая информацию о совместимости организмов. Если на пересечении k -го столбца и j -ой строки стоит 1, то k -ый и j -ый элементы исходного множества могут находиться рядом, иначе нет. В головной программе это добавит только блок ввода матрицы:

```
FOR k:=0 TO Количество-1 DO
  FOR j:=0 TO Количество-1 DO
    In.Int (Матрица[k, j]);
  END;
END;
```

Более интересны отличия, которые необходимо ввести в собственно расчетную часть, то есть в процедуру `ДобавитьЭлемент`. Повторим еще раз рассматриваемую идею оптимизации перебора. Размещение строится последовательностью итераций, на каждой из которых к размещению добавляется еще один элемент. Согласно условию, если на некоторой итерации будет получено незаконное размещение, то такое размещение дорабатывать нет смысла. Из сказанного следует, что перед рекурсивной активацией

```
ДобавитьЭлемент (Очередной+1) ;
```

необходимо выполнить проверку уже построенного размещения на правильность. И если эта проверка даст положительный вывод, то продолжать построение, иначе нет. Мы пока не определились с механизмом проверки, поэтому просто договоримся, что некая переменная имеет значение `ИСТИНА`, если проверка дала положительное заключение, и `ЛОЖЬ` в противоположном случае.

```
IF Флаг THEN  ДобавитьЭлемент (Очередной+1) ; END;
```

У каждого элемента может быть не более двух соседей: справа и слева. Перед началом проверки предположим, что оба возможных соседства законные:

```
Флаг:=TRUE;  
/Проверим соседа слева./  
/Если сосед слева правильный, то проверим соседа справа./  
IF Флаг THEN  ДобавитьЭлемент (Очередной+1) ; END;
```

Заметим, что соседа слева может не быть, поэтому, прежде чем проверять правильность соседства, есть смысл посмотреть, а существует ли такой сосед. Сосед слева существует, если анализируемый элемент не первый и если слева от него в размещении вообще кто-то есть. С учетом сказанного текст получится такой:

```
/Проверяем соседа слева./  
IF (j>0) & (Размещение[j-1]>0) THEN  
    Флаг:=Проверка(j-1);  
END;
```

Проверка соседа справа возможна только в том случае, если проверка соседа слева дала положительное заключение, при отрицательном заключении продолжать проверку нельзя. Поэтому фрагмент будет выглядеть так:

```
/Проверяем соседа справа./  
IF Флаг & (j<Длина-1) & (Размещение[j+1]>0) THEN
```

```

        Флаг:=Проверка(j+1);
    END;

```

И только если обе проверки сохранили истинное значение флага, то построение размещения продолжается. Ниже полный листинг решения.

```

MODULE Модуль;
    IMPORT In, StdLog;
    PROCEDURE ПостроениеРазмещений*;
    VAR
        Множество,Размещение: ARRAY 10 OF INTEGER;
        Количество,Длина,k,j:INTEGER;
        Матрица: ARRAY 10,10 OF INTEGER;
    PROCEDURE ДобавитьЭлемент(Очередной:INTEGER);
    VAR
        k,j,i:INTEGER;
        Флаг:BOOLEAN;
    PROCEDURE Проверка(d:INTEGER): BOOLEAN;
    VAR
        Номер:INTEGER;
    BEGIN
        Номер:=0;
        WHILE Множество[Номер] # Размещение[d] DO Номер:=Номер+1; END;
        RETURN Матрица[Очередной, Номер]=1;
    END Проверка;
    BEGIN
        IF Очередной<Количество THEN
            FOR j:=0 TO Длина-1 DO
                IF Размещение[j]=0 THEN
                    Размещение[j]:=Множество[Очередной];
                    Флаг:=TRUE;
                    /Проверяем соседа слева./
                    IF (j>0) & (Размещение[j-1]>0) THEN
                        Флаг:=Проверка(j-1);
                    END;
                    /Проверяем соседа справа./
                    IF Флаг & (j<Длина-1) &
                        (Размещение[j+1]>0) THEN
                        Флаг:=Проверка(j+1);
                    END;
                    IF Флаг THEN ДобавитьЭлемент(Оче-
                        редной+1);END;
                    Размещение[j]:=0;
                END;
            END;
        ELSE
            FOR i:=0 TO Длина-1 DO StdLog.Int(Размещение[i]);
            END; StdLog.Ln;
        END;
    END;

```

```
END ДобавитьЭлемент;  
BEGIN  
    In.Open;  
    StdLog.Clear;  
    In.Int (Количество); In.Int (Длина);  
    FOR k:=0 TO Количество-1 DO In.Int (Множество[k]);END;  
    FOR k:=0 TO Количество-1 DO  
        FOR j:=0 TO Количество-1 DO  
            In.Int (Матрица[k, j]);  
        END;  
    END;  
    FOR k:=0 TO Длина-1 DO Размещение[k]:=0; END;  
    ДобавитьЭлемент(0);  
END ПостроениеРазмещений;  
END Модуль.
```

Главная мораль

Если вам предстоит искать решение переборной задачи, то первым делом надо попытаться удостовериться, что она действительно переборная. Возможность уйти от переборных алгоритмов есть, но только тогда, когда на исследуемой структуре данных есть какая-нибудь «железная закономерность».

Если железной закономерности нет, то и тогда не все пропало. Во-первых, если нет необходимости в идеальном алгоритме, то можно ограничиться эвристическим решением. Если все же нужно точное решение, то надо на множестве возможных решений выстроить некоторый порядок. Иначе говоря, придумать алгоритм, который все возможные решения получал бы в определенном порядке. Обычно это будет некоторое дерево, узлы которого – это промежуточные решения, а окончательные решения – вершины. Далее к такому дереву нужен критерий, позволяющий по состоянию промежуточного решения (узлу дерева) определять, есть ли смысл двигаться по дереву далее.

Такой порядок и критерий можно построить довольно часто, может быть даже всегда. Обязательным условием существования подобного критерия и дерева есть какие-то взаимосвязи между элементами данных. Не строгая закономерность, а просто взаимосвязи. Например, в задаче о рюкзаке это отношение тяжелее/легче между камнями. В задаче о космическом корабле это отношение – можно посадить рядом или нельзя. То есть это обычные содержательные отношения, которые, наверное, и делают задачу задачей и без которых трудно представить осмысленное условие.

Наверное, мы не ошибемся, если предположим, что единственными чисто переборными задачами без всякой возможности оптимизации будут задачи на построение комбинаций в чистом виде, без дополнительных условий: построение сочетаний, перестановок и размещений. А если это так, то решение любой комбинаторной задачи есть искусство построения большой задачи из стандартных, или построение эвристики, если решатель готов пожертвовать качеством.



ГЛАВА 3.

Как свести решение к задаче существования

Достаточно часто от программиста требуется найти число или структуру данных, удовлетворяющую определенным условиям. Причем не построить, а именно найти в некотором множестве. Это множество может быть задано перечислением своих элементов, или отношением между элементами, или как-то иначе. В качестве примера задачи такого рода можно привести поиск наибольшего в массиве чисел. Задача имеет очень простое решение:

```
Максимальное = Первому Числу
Для всех чисел выполнить
    Если Очередное больше Максимального То
        Максимальное = Очередному
```

Однако даже эта простая задача при небольших дополнительных требованиях может стать вполне нетривиальной. Например, если потребовать за один проход множества чисел найти сразу два наибольших. Это говорит о том, что поиск данных с нужными свойствами в общем случае может оказаться задачей, требующей специального анализа.

Задача для самостоятельного решения

Для того чтобы убедиться в том, что простая по формулировке задача может оказаться нетривиальной, напишите программу, находящую в массиве два наибольших за один проход массива.

Главная идея

Наша базовая идея заключается в том, что задачу поиска нужной величины можно заменить задачей существования решения. Ответ на вопрос «Где решение?» очень существенно отличается от ответа на

вопрос «А если ли решение?», и иногда вопрос существования можно разрешить, не тратя усилий на поиск величины.

Предположим, что искомая величина находится в некоторой вполне определенной области D . Мы твердо знаем, что величина в этой области существует, но не знаем, где именно. Разделим область D по какому-то признаку на две части: D_1 и D_2 . Предположим, что есть критерий, позволяющий, не ища величины, определить, в какой из двух областей она находится. Если это возможно, то мы уменьшаем область поиска без прямых вычислений искомого значения. Если новую область, возможно, опять разбить на две и еще раз применить критерий существования, то получаем метод, который позволяет за конечное количество итераций уменьшить область определения значения настолько, что её полный перебор станет тривиальной задачей, например выбора из двух значений. Ниже разберем несколько примеров использования сформулированного принципа, начиная с простейшей ситуации.

Задача поиска квадратного корня

Начнем исследование вопроса с простого классического примера. Дано положительное число, найти его квадратный корень. Это очень простая задача, но она хороша для демонстрации основной идеи. Здесь идея поиска решения дается устранением неопределенности самого общего вида:

«Что значит найти квадратный корень?»

Дадим на него такой ответ: найти квадратный корень – значит найти такое число, что его квадрат с заданной точностью равен исходному числу. Требование найти число подразумевает ответ уже на два вопроса: *во-первых*, на каком множестве выполняется поиск и, *во-вторых*, когда он прекращается. Оба вопроса имеют простые ответы.

Ответ на первый вопрос. Квадратный корень из числа A есть смысл искать от нуля до самого числа A . Верхнюю границу можно, впрочем, и уточнить, но это сейчас для нашего исследования не имеет значения.

Проблема для самостоятельного исследования

А все-таки как, не проводя трудоемких вычислений, максимально точно определить верхнюю границу корня? Если сможете ответить на этот вопрос, то попробуйте ответить на такой же вопрос относи-

тельно нижней границы. Если найдете ответ и на второй вопрос, то попробуйте подумать о том, что произойдет с границами, если искомым корень больше нуля, но меньше единицы.

Ответ на второй вопрос. Когда очередное число, возведенное в квадрат, окажется равным исходному с заданной точностью.

Два ответа указывают на переборный характер задачи, а именно есть некоторый числовой отрезок, на котором корень находится гарантированно, и для его обнаружения достаточно просмотреть все числа с некоторым шагом и проверить каждое из них на соответствие сформулированному критерию. Чему равен шаг, также понятно. Если нас интересует корень с точностью, например до одной сотой, то шаг должен быть равен одной сотой или меньше.

Таким образом, элементарный метод получен. Его проблема в том, что исходный отрезок, на котором осуществляется поиск, может оказаться очень велик, и это наводит на мысль, что если скорость перебора чисел жестко зависит от требуемой точности, то, может быть, можно уточнить отрезок поиска. Корень ведь только один, а это означает, что большую часть отрезка или, что наиболее вероятно, половину отрезка мы будем исследовать зря. Следовательно, если знать критерий, позволяющий определить, в какой части отрезка находится корень, то по крайней мере половину отрезка можно отбросить, не вычисляя корня непосредственно.

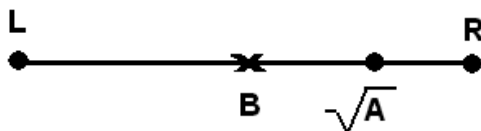


Рис. 3.1. Метод половинного деления

Далее, получив новый отрезок, мы могли бы эту процедуру отсечения половины отрезка повторить еще раз, а потом еще и еще, на каждом шаге получая отрезок, вдвое меньший. Очевидно, что за конечное число таких итераций будет получен отрезок с длиной уже меньше требуемой точности, и тогда за корень можно будет принять середину последнего отрезка.

Это называется метод половинного деления. Разбирать его программную реализацию не будем, её можно легко найти в самых разных учебниках. Для нашего исследования важна только общая идея.

Иногда множество поиска удается разбить на две или более частей, в отношении которых простым критерием можно определить, есть там иско-

мое данное или нет. В этом случае можно построить конечный процесс, в конце которого будет получено очень маленькое множество, найти на котором требуемую величину уже несложно.

Проблема для самостоятельного исследования

Для того чтобы прочувствовать метод, попробуйте самостоятельно решить задачу поиска корня уравнения:

$$f(x) = 0$$

на отрезке $[a, b]$ при условии, что он там действительно есть. И в качестве первого шага определите то общее, что эта задача имеет с задачей поиска квадратного корня. Для начала можно взять квадратное или кубическое уравнение

Поиск отсутствующего числа

Условие задачи. Из целого числового интервала от единицы до миллиарда выбираются случайным образом без повторений миллион чисел и записываются в файл. Необходимо за приемлемое время выяснить, какое наименьшее число отсутствует в файле. Использовать массивы или иные структуры данных, их заменяющие, запрещается.

Изюминка данной задачи заключается в том, что не сразу понятно, в чем, собственно, проблема. Почему эта задача сложна? Почему нельзя использовать какой-нибудь простой, лобовой метод, ведь условие выглядит тривиально? Так иногда бывает. Кажется, что все хорошо и просто, поэтому требуются специальные усилия для того, чтобы хотя бы понять, с чем мы столкнулись. В такой ситуации, чтобы увидеть реальное положение дел, можно придумать любое решение, какое придет в голову, и посмотреть, почему оно не будет работать.

Разрешите предложить следующий вариант: упорядочим числа в файле в порядке возрастания. Алгоритм упорядочения по возрастанию никак нельзя назвать логически сложным, сортировка – хорошо изученный процесс. После упорядочения будет достаточно одного прохода файла, в ходе которого легко обнаружить первое число, чей сосед справа отличается от него более чем на единицу. Тогда найденное число из файла + единица и будет искомым. При таком подходе задача действительно выглядит элементарной. Теперь подумаем, почему это не сработает. Будем упорядочивать файл наибыстрейшим

образом. Какой бы мы при этом не выбрали алгоритм, придется выполнить несколько миллионов файловых операций, а если вспомнить, что файловые операции выполняются медленно, то становится понятно, в чем изюминка задачи – *нельзя читать файл слишком много раз, надо покончить с этим делом за несколько проходов*. Если вопроса, почему эта задача сложная, не осталось, приступим к ее решению.

Формулировка очень похожа на задачу поиска корня заданного числа. В задаче о корне имеется числовой интервал, в котором очень много чисел, и среди них необходимо найти одно, обладающее определенными свойствами (оно является корнем). Есть, правда, очень существенное отличие, в нашем файле числа расположены беспорядочно, в отличие от любого числового интервала. Но, может быть, это окажется не очень существенным. Подумаем вот над чем, в процессе поиска корня на каждом шагу главного цикла интервал поиска разбивается на два, затем определяется новый интервал, в котором далее надо вести поиск. Это суть процесса. Попробуем сделать то же самое в нашей задаче.

Поделим миллиардный интервал на два по 500 миллионов. Где может находиться искомое число? Очевидно, в первом, так как 500-миллионный интервал миллионом чисел не заполнить. Теперь поделим первый 500-миллионный интервал на два и т. д. Рано или поздно заключение о том, что интервал, содержащий искомое число, будет первым, окажется несправедливым. Поэтому давайте для любой пары интервалов разработаем более универсальный критерий.

Заметим, что в простом критерии вывод сделан на основании сравнения того, сколько чисел должен содержать интервал и сколько он их содержит на самом деле. Разовьем эту идею, она кажется перспективной. Предположим, на текущий момент есть интервал [3000, 3999]. Очевидно, для того чтобы быть полностью заполненным, он должен содержать 1000 чисел (напомним, что числа в файле не повторяются, иначе чисел в интервале может оказаться существенно больше 1000 и при этом интервал будет незаполненным). Мы знаем, что все они не меньше 3000 и не больше 3999. Это легко проверяемый факт. Достаточно один раз прочитать файл и подсчитать, сколько в нем чисел, укладывающихся в данный интервал. Если окажется, что чисел меньше, чем длина интервала, то, следовательно, в нем есть число, не записанное в файл.

Процесс разбиения интервала на два следующих необходимо завершать, когда в текущем интервале останутся два числа, тогда следующее разбиение даст два интервала, в одном из которых будет одно

число, а в другом – ни одного. Первое число этого другого интервала и есть искомое. Запишем алгоритм. Его главные объекты – два счетчика. Первый считает числа, входящие в первый интервал, второй – числа, входящие во второй интервал.

```
Пока длина интервала больше единицы, делать
  Разбить интервал на два
  Читать файл и для каждого числа, прочитанного из файла, делать:
    Если число принадлежит первому интервалу
      То первый счетчик увеличивается на единицу
    Иначе второй счетчик увеличивается на единицу
  Текущий интервал – это интервал, значение счетчика
  которого меньше длины интервала.
```

Записанный алгоритм неплохо выражает общую идею, но в нем много неопределенных понятий. Самый главный вопрос – так ли важно делить именно на два интервала, может быть, лучше делить на 3 или 5? Выигрыш, который мы можем получить, изменив количество интервалов, заключается в количестве необходимых прочтений файла, а как вы помните, именно многократное чтение файла и создает проблему для решения задачи. Посмотрим, как быстро мы получим решение с разным количеством интервалов.

Для миллиарда количество шагов будет равно показателю степени двойки, при которой степень превысит миллиард. При делении интервала на 10 мы получим количество шагов, равное показателю степени 10, при котором степень превысит миллиард. Таким образом, действительно увеличение количества интервалов дает выигрыш во времени. Но, рассуждая подобным образом, мы приходим к противоречию, так как наша цепочка рассуждений быстро приведет к необходимости разбить миллиард на миллиард интервалов. Это, конечно, бессмыслица, и надо подумать, откуда она появилась.

Во-первых, мы забыли про память. Чем больше интервалов, тем большее количество счетчиков необходимо помнить, а это затраты на память. Кстати, сейчас можно сформулировать одно общее правило.

Если вам удалось улучшить какую-то характеристику вашей программы, то вы должны ожидать, что какая-то другая характеристика станет хуже.

Это правило не имеет характера закона. В принципе, можно так улучшить программу, что все ее характеристики станут лучше. Она станет быстрее работать, в то же время уменьшатся затраты на память, ее текст станет короче, понятнее и т. д. Это вполне возможно,

но чаще всего такое всеобщее улучшение будет означать, что предыдущий вариант программы был совсем плох. А чем лучше ваша программа, чем изощреннее алгоритм и чем эффективнее использованы средства языка, тем полученное правило действует точнее.

В нашей задаче, увеличивая количество интервалов, мы теряем в памяти, таким образом, количество интервалов ограничено объемом имеющейся оперативной памяти. Но на самом деле мы теряем и в скорости! Для того чтобы перейти на следующий шаг, необходимо найти счетчик, значение которого меньше длины текущего интервала, а если массив будет велик, то его обход также займет время, и совершенно не очевидно, что эти затраты времени можно игнорировать.

Конечно, для идеального решения необходимо взвесить все выигрыши и потери, а также определиться, какое количество интервалов выгодно. Но точное решение этого вопроса вряд ли возможно. Во-первых, для этого необходимо очень точно знать скорость выполнения операций процессором, и, во-вторых, предстоит сравнивать величины, имеющие различную природу (это скорость счета и память), а корректно сравнивать можно только величины одной природы. Поэтому не будем искать идеальное решение (так как непонятно, что это такое) и ограничимся достаточно хорошим. При количестве интервалов, равном 10, нам потребуется всего 12 шагов, то есть 12 прочтений файла, что немного, для миллионного файла такая работа займет приемлемое время.

Проблема для самостоятельного исследования

Все же обдумайте, можно ли определить оптимальный шаг разбиения.

Следующая неясность алгоритма – это разбиение интервала. Как именно это сделать? Вопрос не слишком прост, так как текущий интервал не только уменьшается в 10 раз, но и достаточно произвольно скачет по исходному миллиарду (искомое число может оказаться где угодно).

Ранее были заявлены два важных утверждения относительно интервала: его длина переменна, его положение переменнo. Следовательно, для задания интервала нужны две величины: длина и число, с которого он начинается, левая точка.

Проблем с расчетом длины нет. На первом шаге длина равна миллиарду, на каждом последующем длина уменьшается в десять раз. Левая точка первого шага – это нуль. Обозначим текущую левую точку как `НАЧАЛЬНАЯ`. Попробуем посчитать, куда она переместится после

очередного шага. Пусть новым интервалом будет интервал с номером $L = 1, 2, 3, \dots, 10$. Обозначим длину предыдущего интервала через ДЛИНА. Тогда Новую НАЧАЛЬНУЮ точку можно вычислить по формуле

$$\text{Новая НАЧАЛЬНАЯ} = (L - 1) * (\text{ДЛИНА} / 10)$$

На самом деле эта формула несколько неверна. Она вычисляет новое положение НАЧАЛЬНОЙ точки относительно НАЧАЛЬНОЙ точки текущего интервала. Кроме того, надо учесть, что все числа левее предыдущей НАЧАЛЬНОЙ точки уже исключены из рассмотрения. Это можно учесть следующим образом:

$$\text{Новая НАЧАЛЬНАЯ} = \text{Предыдущая НАЧАЛЬНАЯ} + (L - 1) * (\text{ДЛИНА} / 10)$$

Следующий неясный момент заключается в процедуре определения, к какому из интервалов принадлежит очередное прочитанное число. Рассмотрим первый шаг с исходным интервалом. В каждый из внутренних интервалов может входить 100 миллионов чисел. В первый входят числа от нуля до 99 999 999. Заметим, что если любое число из этого интервала мы разделим нацело на 100 000 000, то результатом будет нуль (делим ведь нацело). Точно так же любое число из интервала от 100 000 000 до 199 999 999, деленное на 100 миллионов, даст 1 и т. д. То есть если мы число разделим на 100 миллионов и прибавим 1, то как раз и получим номер интервала. Запишем это в общем виде, для чего введем обозначения: ДЛИНА — длина интервала, ЧИСЛО — число, прочитанное из файла, и ИНДЕКС — индекс вложенного интервала.

$$\text{ИНДЕКС} = \text{Целая часть от } (\text{ЧИСЛО} / \text{ДЛИНУ}) + 1$$

Эта формула полезна для понимания того, что нам требуется, но она неточна. Попробуйте проверить ее действие хотя бы для второго шага. Ошибка будет в следующем. Предположим, в файле есть число пятьсот миллионов. На первом шаге (деление на 100 миллионов) это число по нашей формуле даст индекс 5, а на втором это же число даст индекс 50, то есть несуществующий, а на третьем это же число даст индекс 500 и т. д. Отсюда следует, что перед тем, как вычислять индекс, надо выяснить, входит ли прочитанное число в интересующий нас интервал, если сказать более точно, то необходимо выяснить, находится ли это число между НАЧАЛЬНОЙ точкой и точкой НАЧАЛЬНАЯ + ДЛИНА интервала. Теперь наша формула превращается в небольшой алгоритмический фрагмент.

$$\text{Если ЧИСЛО} \geq \text{НАЧАЛЬНОГО и ЧИСЛО} < \text{НАЧАЛЬНОЕ} + \text{ДЛИНА}$$

То ИНДЕКС = Целая часть от (ЧИСЛО / ДЛИНУ) + 1

Это еще не все уточнения, но, прежде чем идти дальше, сделаем небольшое замечание о методе наших рассуждений. В этой задаче мы выводим уже вторую формулу. Обратите внимание, как это делается. Во-первых, мы записываем формулу расчета из самых простых и очевидных соображений, чаще всего исходя из какого-то простого примера, например исходя из начального состояния данных. Затем берем более сложный пример, и если формула отказывается работать, то корректируем ее с учетом новой, полученной информации, потом ищем следующую ошибку и т. д. Конечно, может возникнуть вопрос: откуда мы знаем, что ошибки закончатся? Точного, строгого ответа на данный вопрос дать невозможно, но есть одно очень сильное соображение. Если один пример считается правильно, а другой нет, то между этими примерами есть какое-то принципиальное отличие, не сводящееся к разнице в количественном значении. В наших двух примерах, рассмотренных ранее, различие следующее: миллиард – это начальный интервал, второй интервал в 100 миллионов – это интервал, впервые полученный расчетным способом. Такова принципиальная разница. Примеров, между которыми есть принципиальная разница, не может быть сколь угодно много, а следовательно, ошибки должны закончиться.

А теперь вернемся к формуле и продолжим рассуждения об интервале, разбиваемом на меньшие. Ранее уже мелькало понятие вложенного интервала, уточним этот термин. Практически невозможно сразу дать точные определения всем используемым понятиям, но сейчас это сделать необходимо, *мы выходим на конкретные формулы, и неточности в понятиях приведут к неточности в формулах.* И такая неточность в нашей формуле уже появилась. Во фрагменте, приведенном ранее, дважды используется длина, а что при этом имеется в виду? В команде ЕСЛИ имеется в виду длина текущего интервала, в котором осуществляется поиск, а в формуле имеется в виду длина интервала разбиения (давайте далее называть его так), который в 10 раз меньше текущего. Поэтому уточним формулу следующим образом:

Если ЧИСЛО >= НАЧАЛЬНОМУ и ЧИСЛО < НАЧАЛЬНОЕ + ДЛИНА
То ИНДЕКС = Целая часть от (ЧИСЛО / ДЛИНУ РАЗБИЕНИЯ) + 1

И это еще не все. Предположим, что текущий интервал включает в себя число 500 миллионов, а длина текущего интервала, например, 1000. Ничего невозможного в этом нет. То есть число 500 миллионов

окажется вполне допустимым, а индекс, который оно породит, будет 500 миллионов, деленное на 1000, что, конечно же, ошибка. И эта ошибка заключается в том, что индекс надо считать также с учетом числового значения НАЧАЛЬНОЙ точки. В конечном итоге формула приобретает такой вид:

Если ЧИСЛО $\geq$ НАЧАЛЬНОМУ и ЧИСЛО $<$ НАЧАЛЬНОЕ + ДЛИНА

То ИНДЕКС = Целая часть от $((\text{ЧИСЛО} - \text{НАЧАЛЬНАЯ}) / \text{ДЛИНУ РАЗБИЕНИЯ}) + 1$

Конечно, трудно предположить, что все эти рассуждения могли быть проведены до того, как написана программа. На самом деле разумно написать программу с первоначальным вариантом формулы, затем в процессе отладки находить ошибки и, отталкиваясь от них, проводить дальнейшие рассуждения. Так всегда легче. Редко кто может провести полный анализ всех проблем до написания программы. Впрочем, все наиболее существенные моменты решения мы проанализировали, а реализацию оставим желающим.

Задача для самостоятельного решения

Реализуйте описанный выше алгоритм. Для упрощения чисто технических проблем числа из числового отрезка можно записывать не в файл, а в массив. Обязательным остается только требование минимального количества проходов массива.

Пример выше мы разобрали довольно детально, и я опасаясь, как бы мелкие детали технической реализации не скрыли собой главную мораль. А заключается она в следующем.

Сила метода в том, что поиск искомой величины сводится к проблеме существования. В процессе вычисления определяются области, в которых решения гарантированно нет, и область, в которой оно есть. В каком-то смысле мы получаем величину, не вычисляя её непосредственно.

В сказанном заключается главная, соль метода. Все дело именно в возможности найти критерий существования. Иногда, правда, этот критерий выглядит еще более хитро и не сводится напрямую к какому-то простому перечислимому множеству. Рассмотрим еще один пример.

Решение уравнения Диофанта

Решение линейных уравнений вида $ax + by = m$ хорошо известно и выражается простыми формулами, но уже уравнение второго порядка решается достаточно сложно, а для алгебраических уравнений третьего и более высокого порядка общего метода нет. Ниже будет

предложен численный метод решения любых алгебраических неопределенных уравнений, а чтобы у читателя не создалось ошибочного ощущения математической значимости этого текста, сразу предупредим, что эффективность метода очень невелика в большинстве случаев, поэтому и ценен он для нас только как способ продемонстрировать исследуемый в этой главе подход. Хотя для некоторых классов диофантовых уравнений метод может дать хороший результат.

Договоримся, что хотя метод годится для любых уравнений, мы будем все рассуждения проводить на уравнениях вида:

$$ax + bxy + cy = m$$

и даже более, пусть искомые решения будут обязательно положительными числами. Это ни к чему не обязывает, но рассуждения несколько упростит. Итак, необходимо из множества значений выражения $ax + bxy + cy$ найти одно, равное m .

Общий подход требует некоторой области поиска решения, но здесь таковая совпадает со множеством целых чисел, и совершенно не ясно, как выделять какие-то конечные или бесконечные области, для которых можно было решить вопрос существования решения. Интуитивно, однако, понятно, что если такие области выделить удастся, то их можно будет описать также неопределенными уравнениями, связанными с исходным, так как и исходное уравнение описывает какую-то числовую область. Таким образом, мы ищем способ преобразовать исходное уравнение в несколько уравнений, так что одно из уравнений будет содержать решение, а остальные нет. Во-вторых, надо быть уверенным, что процесс преобразования можно будет продолжать, и удастся определить критерий его завершения.

Итак, что мы знаем о уравнении. Только то, что его решения – положительные целые числа. Значения этих чисел мы не знаем, поэтому опереться на отношение больше/меньше не получится. Но какое-то свойство, разводящее числа по разные стороны баррикад, необходимо. Таким свойством может быть свойство четности. Число может быть либо четным, либо нечетным. Третьего не дано. Посмотрим, что это нам дает на конкретном примере:

$$5x + 4xy + 3y = 1042.$$

Уравнение имеет положительное решение: $x = 11$; $y = 21$, но пусть этот факт пока неизвестен. А известно, что есть решение x, y , и эти два числа могут быть как четными, так и нечетными. Это утверждение означает, что к исходному уравнению возможно применить четыре преобразования, дающие четыре новых выражения:

- $x = 2x; y = 2y$ (оба четны);
- $x = 2x; y = 2y + 1$;
- $x = 2x + 1; y = 2y$;
- $x = 2x + 1; y = 2y + 1$ (оба нечетны).

Выполним все записанные преобразования:

Вариант 1. Исследуемое преобразование: $x = 2x; y = 2y$. После подстановки получаем следующее выражение: $10x + 16xy + 6y = 1042$, левая и правая части этого выражения четны, поделим уравнение на два и получим: $5x + 8xy + 3y = 521$. Вполне возможное уравнение, о разрешимости которого нельзя сказать ничего определенного.

Вариант 2. Исследуемое преобразование: $x = 2x; y = 2y + 1$. После подстановки получаем следующее выражение: $18x + 16xy + 6y = 1039$. Левая часть выражения четна, правая нечетна, следовательно, относительно данного уравнения можно сделать однозначный вывод о его неразрешимости.

Вариант 3. Исследуемое преобразование: $x = 2x + 1; y = 2y$. После подстановки получаем следующее выражение: $10x + 16xy + 14y = 1037$. Левая часть выражения четна, правая нечетна, следовательно, относительно данного уравнения можно сделать однозначный вывод о его неразрешимости.

Вариант 4. Исследуемое преобразование: $x = 2x + 1; y = 2y + 1$. После подстановки получаем следующее выражение: $18x + 16xy + 14y = 1030$, левая и правая части этого выражения четны, поделим уравнение на два и получим: $9x + 8xy + 7y = 515$. Вполне возможное уравнение, о разрешимости которого нельзя сказать ничего определенного.

Таким образом, четыре возможных преобразования дали два уравнения, правая часть которых уменьшилась вдвое, что очевидно уменьшает область поиска решения вдвое, а коэффициенты левой части также изменились в сторону увеличения, из чего можно сделать вывод о том, что область поиска реально изменилась более чем вдвое. И главный вывод – полученное преобразование, хотя и увеличивает количество уравнений вдвое, тем не менее выгодно, за счет еще большего уменьшения области поиска решения.

Первое требование общего метода сведения вычислений к задаче существования выполнено. Мы научились разбивать уравнение на четыре и отбрасывать два из них, уменьшая область поиска решения более чем в два раза. Конечно, было бы интересно оценить, насколько реально уменьшается область поиска, но это не входит в наши цели.

Полученное уравнение можно опять разбить на два и т. д. Ясно, что процесс не может быть бесконечным, так как правая часть уравнения на каждой итерации уменьшается вдвое. Но все же необходимо точное понимание, когда завершать вычисления. Разберемся с этим вопросом.

Внешний вид преобразования говорит о том, что корни уравнения уменьшаются вдвое. Это означает, что через конечное число шагов будет иметь место одна из следующих ситуаций:

- $x = 0; y = 1;$
- $x = 1; y = 0;$
- $x = 1; y = 1.$

Для уравнения в общей записи $ax + bxy + cy = m$ это означает, что через конечное число шагов окажется справедливым одно из следующих равенств:

- $c = m;$
- $a = m;$
- $a + b + c = m.$

Это и есть критерий остановки процесса. Таким образом, мы научились преобразовывать исходное уравнение в более решаемые и определили критерий завершения вычислительного процесса. Наверное, чтобы было более понятно, как этим пользоваться, решим от начала до конца еще одно уравнение:

$$7x + 4xy + 5y = 89.$$

Выше не было сказано, но вы можете сами в том убедиться, что при нечетной правой части возможны только два преобразования:

- $x = 2x + 1; y = 2y;$
- $x = 2x; y = 2y + 1.$

А при четной правой части – два других:

- $x = 2x; y = 2y;$
- $x = 2x + 1; y = 2y + 1.$

В нашем уравнении правая часть нечетна, и после выполнения допустимых преобразований получаем два новых уравнения:

- $11x + 8xy + 5y = 42;$
- $7x + 8xy + 9y = 41.$

Ни одно из этих уравнений не удовлетворяет критерию прекращения вычислительного процесса, поэтому необходима еще одна итерация, с учетом того, что в первом уравнении правая часть четна, а

у второго нечетна. Применив допустимые преобразования, получим уже четыре уравнения:

- $11x + 16xy + 5y = 2$;
- $19x + 16xy + 13y = 9$;
- $15x + 16xy + 9y = 16$;
- $7x + 16xy + 17y = 17$.

В полученной четверке, последнее уравнение удовлетворяет критерию $c = t$, следовательно, для четвертого уравнения имеем корни: $x = 0$; $y = 1$. Но это не исходное уравнение. Чтобы получить корни исходного, запишем еще раз выполняемые преобразования и промежуточные результаты, уже пропуская ненужные ветвления:

Шаг первый. Уравнение $7x + 4xy + 5y = 89$ после преобразования $x = 2x + 1$; $y = 2y + 1$ превратилось в уравнение:
 $7x + 8xy + 9y = 41$.

Шаг второй. Уравнение $7x + 8xy + 9y = 41$ после преобразования $x = 2x + 1$; $y = 2y + 1$ превратилось в уравнение:
 $7x + 16xy + 17y = 17$ корни которого $x = 0$; $y = 1$.

Теперь, чтобы получить корни исходного уравнения, достаточно пройти по преобразованиям в обратную сторону:

Шаг первый. $x = 2 \times 0 + 1 = 1$; $y = 2 \times 1 = 2$.

Шаг второй. $x = 2 \times 1 + 1 = 3$; $y = 2 \times 2 = 4$.

Окончательно уравнение $7x + 4xy + 5y = 89$ имеет следующие корни: $x = 3$; $y = 4$.

Проблема для самостоятельного исследования

Мы построили нетривиальный пример использования принципа сведения к задаче существования. А в завершение сформулируем интересную проблему, решение которой даст очень эффективный метод счета неопределенных уравнений. Заметим, что на каждом шаге итерации очередное уравнение превращается в четыре, два из которых мы отсекаем очень простым критерием неразрешимости, но о двух оставшихся нельзя сказать ничего определенного. Если бы удалось найти критерий неразрешимости для уравнений Диофанта (что дало бы возможность отбрасывать одно уравнение из двух оставшихся), тогда на каждой итерации было бы возможно одно уравнение преобразовать не в два уравнения, а в одно, и тогда описанный метод решения диофантовых уравнений превратился бы в универсальный алгоритм для любого уравнения. К сожалению, этому всеобщему решению мешает результат, полученный Юрием Матиясевичем, до-

казавшим, что не существует алгоритма, способного решить любое неопределенное уравнение или указать на отсутствие решения за конечное число шагов. Но отсутствие всеобщего решения не означает невозможности такого события для отдельных классов уравнений. Поиск таких классов и критериев неразрешимости и есть наша проблема.



ГЛАВА 4.

Тождественные преобразования условий

Не всегда бывает целесообразно решать задачу в той формулировке, в какой она дана изначально. Во-первых, очень часто условие задачи бывает перегружено информацией. Перегрузка может иметь стилистическую природу, и иногда так бывает, что условие содержит какие-то лишние утверждения. Хороший пример такой ситуации – классическая задача Дейкстры. В следующей главе мы дадим её полный анализ, а сейчас только условие. Дана функция, определяемая следующими выражениями:

$$\begin{cases} f(1) = 1 \\ f(2N) = f(N) \\ f(2N+1) = f(N) + f(N+1) \end{cases}$$

Прежде всего необходимо убрать мусор из текста условия

Это определение утверждает, что получить числовое значение за одно действие можно только от единичного аргумента. Если аргумент четный, то функция преобразуется в функцию от половинного аргумента. Если же аргумент нечетный, то функция переходит в сумму двух функций с аргументами, отличающимися друг от друга на единицу и в сумме дающими исходный.

Мне доводилось видеть следующую её формулировку.

$$\begin{cases} f(0) = 1 \\ f(1) = 1 \\ f(2N) = f(N) \\ f(2N+1) = f(N) + f(N+1) \end{cases}$$

Первое равенство не является некорректным, и его включение в условие не изменяет природы функции. Но достаточно просчитать несколько примеров, чтобы убедиться – нуль получить в вычислительном процессе невозможно. Следовательно, добавленное условие излишне. Еще один пример. Дано неопределенное уравнение вида:

$$x^2 + ay = b.$$

Необходимо найти все пары взаимнопростых решений.

Условие вполне понятно, но есть смысл проанализировать требование взаимной простоты (отсутствие общих делителей x и y). Пусть у пары решений есть общий делитель. Обозначим его через c . Тогда уравнение принимает вид:

$$c^2 x^2 + acy = b.$$

Ясно, что это новое уравнение не имеет корней, если b не делится на c . И уравнение принимает новый вид:

$$c^2 x^2 + acy = cd \quad \text{или} \quad cx^2 + ay = d.$$

То есть сводится к уравнению с меньшими коэффициентами. Следовательно, допущение невзаимнопростых пар корней сводится к поиску делителей правой части уравнения и сведению к уравнению того же вида. А это означает, что принципиально важно уметь искать именно взаимнопростые корни. И следовательно, озвученное требование условия можно назвать само собой разумеющимся.

Такие вполне корректные, но не очень нужные требования в условиях задачи встречаются чаще, чем это можно было бы ожидать. Их небольшой анализ важен, так как дает определенность в понимании того, что нужно на самом деле. А борьба с неопределенностью на этапе анализа условия избавит решателя от лишней траты энергии на других более трудоемких этапах.

Избавляясь от такого рода утверждений, мы получаем, очевидно, тождественную формулировку, но более ясную. Еще один способ ухода от сути дела очень широко практикуется на различных олимпиа-

дах по программированию. Ниже условие задачи одной из российских олимпиад. Текст приведен без сокращений:

«Совсем недавно семья Гарнетт переехала в новый город. Земельный участок, приобретенный ими, находится рядом с домом, в котором живут Шерлок Холмс и доктор Ватсон. Гарнетты рады такому знакомству, ведь теперь в случае загадочных и необъяснимых происшествий у них будет возможность посоветоваться со знаменитым сыщиком.

Однако в проблеме, стоящей перед Гарнеттами сейчас, нет ничего загадочного. Участок, на котором они планируют построить себе дом, имеет форму прямоугольника размером $n \times m$ метров. Гарнетты, как и все истинные англичане, любят порядок, и поэтому они хотят, чтобы их дом также имел прямоугольную форму, его стены были параллельны границам участка, а расстояние от любой стены дома до параллельной ей границы участка выразалось бы целым числом метров. При этом они хотят построить дом максимальной площади.

К сожалению, есть проблема, мешающая им построить дом, совпадающий с границами участка. Заключается она в том, что на участке расположены две скважины с водой, и Гарнетты хотят, чтобы одна из них оказалась внутри дома, а другая – за его пределами. Чтобы узнать максимальную площадь, которую может иметь дом, отвечающий описанным выше требованиям, они обратились к Шерлоку Холмсу. Помогите им ответить на этот вопрос».

Длиннейшее условие, большая часть информации в котором, очевидно, лишняя. Если выделить только значимые понятия и утверждения, то текст условия может принять следующий вид:

Даны прямоугольник размерами $n \times m$ и две произвольно расположенные внутри него точки. Необходимо найти прямоугольник максимальной площади, такой что:

- *он полностью лежит внутри данного прямоугольника;*
- *его стороны параллельны сторонам данного прямоугольника и расстояния до соответствующих сторон выражаются целыми числами;*
- *одна из произвольных точек лежит внутри построенного прямоугольника, и одна за его пределами.*

Вот такое короткое условие. Надобно отметить, что подобная информационная избыточность характерна не только для олимпиадных

задач. Примеры избыточных условий стилистического характера можно приводить в самых разных областях. И если на олимпиадах это скорее дань традиции, то в иных, более прикладных сферах человеческой деятельности появление лишнего текста есть проявление фундаментального свойства человеческого языка – его избыточности. Отсюда еще один важный этап преобразования текста в тождественный – это запись текста в более формальной и менее избыточной форме.

Что делать после генеральной уборки

Избавившись от всей лишней литературы, тривиальных и не очень содержательных условий, мы подготовим себе почву для поиска решения. Говорят, что правильно сформулировать задачу, – это почти решить её. Это, конечно, сильное преувеличение. Именно сейчас понадобятся более сильные средства. Выше мы обсуждали некоторую технику перехода к тождественным условиям, не требующую радикальных изменений. Но иногда бывает возможно изменить условие очень основательно, перейдя к новой терминологии, даже новой понятийной базе, и при этом получить условие, тождественное исходному в том смысле, что решение новой задачи легко дает и решение задачи исходной. Ниже – простой пример.

Задача о рекурсивной процедуре

Сформулируем условие не вполне строго, а лишь достаточно для понимания сути дела. Итак, дан некоторый язык программирования, в котором определено понятие процедуры и допустимы рекурсивные вызовы. Необходимо определить, есть ли хотя бы один рекурсивный вызов в конкретной программе.

Условие предельно ясно, никаких литературных излишеств. Попробуем переформулировать задачу, уйдя от понятий языка программирования и программы. Заметим для начала, что любой язык программирования, кроме вызовов процедур, содержит некоторое количество различных операторов: присваиваний, циклов и т. д., и все это множество операторов, большое или маленькое для нашей задачи, не имеет никакой смысловой ценности. Следовательно, в плане описания языка есть смысл рассматривать только структуру процедуры,

состоящей из вызовов процедур. Программа в этом смысле представляет собой некоторый набор процедур, одна из которых объявляется стартовой. Ниже пример:

```

Стартовая процедура
  Начало
    Вызов Первой процедуры
    Вызов Третьей процедуры
    Вызов Второй процедуры
  Конец стартовой процедуры
Первая процедура
  Начало
    Вызов Второй процедуры
  Конец первой процедуры
Вторая процедура
  Начало
    Вызов первой процедуры
  Конец второй процедуры
Третья процедура
  Начало
    Вызов третьей процедуры
  Конец третьей процедуры
  
```

Пример состоит из трех рекурсивных вызовов. Легко заметить, что данную языковую конструкцию легко представить в виде графа:

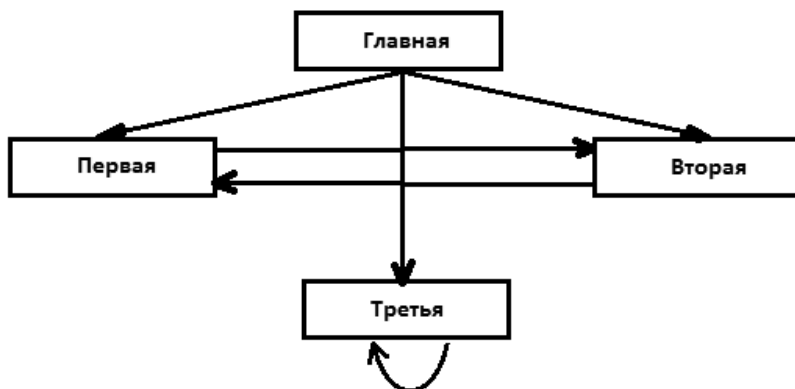


Рис. 4.1. Представление структуры программы графом

и проблема полностью сводится к стандартной задаче поиска цикла на графе. Единственное, необходимо решить чисто технический вопрос, как текст программы, представляющий собой набор строк, перевести в форму описания графа, например матрицей смежности. Разберем еще один нетривиальный пример.

Проблема для самостоятельного исследования

Выше был сформулирован вопрос технического характера, как текст программы, являющийся входным данным для нашего алгоритма, перевести в представление графом. Дайте на него ответ.

Задача о бесконечном слове

Условие задачи. Некоторый язык состоит из символов и имен подстановок. Множество символов исчерпывается строчными латинскими буквами. Имя подстановки – это заглавная латинская буква. Каждая подстановка также имеет описание следующего вида:

Имя подстановки (список символов);

Символ = строчная или заглавная буква латинского алфавита.

Пример описания подстановки: LaauG – подстановка состоит из трех символов aa и одной подстановки G, возможно пустой, если она не имеет описания. Её имя – L.

Слово языка состоит как из строчных, так и, заглавных латинских букв, причем заглавные буквы интерпретируются как имена подстановок. Если в слове присутствует хотя бы одна заглавная буква, то такое слово называется сжатым. Развернутое слово – это слово, состоящее только из строчных букв. Каждое сжатое слово можно превратить в развернутое, заменив заглавные буквы соответствующей подстановкой. Если для какой-то заглавной буквы во множестве подстановок нет описания подстановки (то есть она не является именем подстановки), то такую заглавную букву можно из сжатого слова просто убрать, как незначащую. Если из сжатого слова возможно убрать все имена подстановок за конечное число итераций, то такое слово называется конечным. Но вполне возможны языки, в которых можно построить бесконечные слова. Простой пример: если язык содержит только одну подстановку – LL, то сжатое слово Lf будет бесконечным, хотя на каждом шаге преобразования слово не изменяет своего вида. А для языка, содержащего подстановку LLd, это же слово Lf будет бесконечным и неограниченным по длине, так как каждый шаг преобразования будет увеличивать слово на один символ.

Пример. Если язык содержит две подстановки Dffg и FssD, то сжатое слово aDsF в развернутом виде примет следующий вид: affgssffg.

Итак, пусть дано описание грамматики языка (множество подстановок) и дано слово, записанное по правилам этой грамматики. Требуется определить, конечное это слово или бесконечное.

Проведем анализ задачи. На первый взгляд, нет ничего общего с задачей о рекурсивной процедуре. Однако мы сейчас покажем, что это одна и та же задача, в том смысле, что существует формулировка, тождественная как задаче о процедуре, так и задаче о бесконечном слове. Для начала построим еще один пример грамматики языка и пример слова, как конечного, так и бесконечного, и посмотрим, что реально происходит в процессе замены имен подстановок.

Для определения условия задачи необходимо описать множество подстановок языка. Итак, пусть язык содержит следующие подстановки: AffD, Rewq, CEhh, DdfsA, Ewqw. И пусть дано слово CfdsaR. Выпишем последовательность подстановок:

C: AfdsaR $\rightarrow$ EhhfdsaR

E: EhhfdsaR $\rightarrow$ wqwhhfdsaR

R: wqwhhfdsaR $\rightarrow$ wqwhhfdsaewq

Имен подстановок не осталось, преобразования завершены, получена конечная последовательность символов. Рассмотрим еще один пример, очень короткое слово, состоящее из одного имени подстановки: D (язык тот же). Что получится в процессе преобразований:

D: D $\rightarrow$ dfsA

A: dfsA $\rightarrow$ dfsffD

D: dfsffD $\rightarrow$ dfsffdfsA

и т. д., процесс, очевидно, бесконечный. Что мы здесь видим? На каждой итерации в слове появляются строчные символы языка, никоим образом не влияющие на процесс и имена подстановок, определяющие последующие итерации. Если имя подстановки рассматривать как узел графа, а описание подстановки (выбросив из него строчные символы) как перечень ребер, выходящих из данной вершины, то процесс построения слова, можно рассматривать как построение графа, такого, что графу с циклом соответствует бесконечное слово, а графу без циклов – конечное. Таким образом, мы свели исходную задачу к тождественной – поиску циклов на графе, а следовательно показали её тождественность задаче о рекурсивных процедурах.

Проблема для самостоятельного исследования

Попробуйте самостоятельно ответить на следующий вопрос: можно ли только по описанию грамматики, построенной по правилам нашей

задачи, определить, существуют ли в языке бесконечные слова, не изобретая примеров?

Надобно сказать, что для любой математической конструкции можно указать множество задач с нетривиальной формулировкой условия, сводимых к вопросу существования того или иного признака на этой конструкции, так же как мы только что свели две задачи к вопросу существования цикла на графе. Например, довольно много задач сводится к стандартным алгоритмам комбинаторики на построение сочетаний, размещений, перестановок.

Неопределенные уравнения

Идея замены исходной задачи тождественной, но решаемой более просто, наверное, такая же древняя, как и математика. Об этом свидетельствует и пример, которым мы сейчас займемся. Начиная с Диофанта, математика интересовалась уравнениями, корни которых представляют собой целые или рациональные числа. В предыдущей главе уже использовались такие уравнения для демонстрации метода, сводящего вычисление величин к задаче существования. Сейчас мы их используем для демонстрации новой идеи. Итак, пусть дано уравнение с двумя неизвестными. Запишем его в наиболее общем виде:

$$f(x, y) = 0,$$

где f – функция второй степени для упрощения рассуждений. Необходимо определить все его рациональные корни. Сразу заметим, что это одновременно общий вид для алгебраической записи кривых на плоскости, что дает надежду свести поиск корней к анализу кривой. Действительно рациональный корень уравнения, – это не что иное, как рациональная точка на кривой. Следовательно, нам необходимо научиться находить все рациональные точки (с рациональными координатами x и y).

В условии требуется найти все точки. Это означает, что результатом должно стать выражение, содержащее в себе все рациональные решения.

Существует общий прием решения такого рода проблем, заключающийся в двух действиях: во-первых, найдем одно любое решение любым способом. Зачастую это бывает несложно. Во-вторых, научимся выражать еще одно решение через уже известное. Конечно, не факт, что таким способом удастся исчерпать все множество решений, но вполне возможно, что именно так и будет.

Найти одну рациональную точку на кривой несложно. Так как выражение $f(x, y) = 0$ содержит два неизвестных, одному из них можно придать произвольное значение, и вычислить второе. Допустим, что, зная вид уравнения, мы можем подобрать рациональные значения: $x = a$; $y = b$, тогда прямая, проведенная через точку (a, b) , будет иметь с кривой еще одну точку пересечения, и тоже рациональную, при условии что прямая линия имеет рациональный угловой коэффициент. В параметрическом виде такую прямую линию можно записать в следующем виде:

$$\begin{cases} x = a + t \\ y = b + kt \end{cases}$$

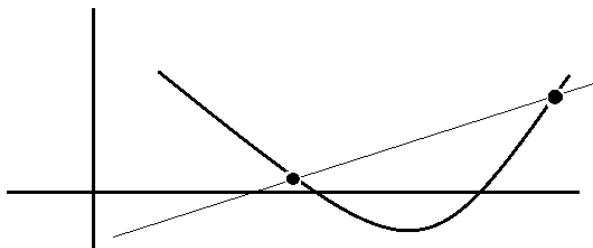


Рис. 4.2. Две рациональные точки

Эти два выражения можно рассматривать как подстановку в выражение $f(x, y) = 0$, выполнив которую, получим следующую запись:

$$f_2(a + t, b + kt) = f_2(a, b) + tA(a, b) + ktB(a, b) + t^2C(a, b, k) = 0.$$

Но известно, что $f_2(x, y) = 0$, поэтому получаем:

$$t = -\frac{A(a, b) + kB(a, b)}{C(a, b, k)},$$

откуда видно, что для каждого рационального k действительно получаем рациональное t и, следовательно, рациональные x, y . Конечно, такой вывод можно получить и чисто алгебраическим путем, но переход в область геометрии и переформулировка задачи как геометрической делает все рассуждения проще. Рассмотрим еще один пример, когда анализ задачи как геометрической значительно упрощает если не решение, то по крайней мере понимание задачи.

Расчет оптимального плана производства

Метод, который рассмотрим ниже, применим для очень широкого круга задач, но мы для удобства ограничимся задачей построения плана производства группы товаров. Итак, представим себе, что в нашем распоряжении есть небольшой завод. Конечно, он располагает какой-то технической базой, есть помещения, сотрудники, управленческий аппарат и много других вещей и обстоятельств, влияющих на коммерческий успех, но давайте от всего этого отвлечемся, введя понятие ресурса. Сырье, необходимое для производства, – это ресурс, электроэнергия – это ресурс, рабочая сила – это тоже ресурс и т. д. Начнем анализ с неформального примера. Пусть таким примером станет мебельная фабрика, выпускающая два вида продукции – шкафы и стулья. Для упрощения ситуации пусть шкафы изготавливаются из древесно-стружечных плит, а стулья – из дерева, для обоих видов продукции необходимы клей, электроэнергия и затраты труда рабочих – мебельщиков. Вопрос: что фабрике лучше производить, если известно, что рабочих только десять человек, дерева и ДСП не бесконечно много и затраты на электроэнергию ограничены, кроме того, известно, что шкаф дороже стула?

Прямолинейная логика скажет, что шкафы собирать выгоднее. Однако это не обязательно так. Отправив всех рабочих на сборку шкафов, руководитель такой фабрики оставит без использования материал для изготовления стульев. Видимо, без нагрузки останется какое-то специальное оборудование, рассчитанное на стулья. Кроме того, возможно, запасы ДСП окажутся не так велики, чтобы загрузить работой всех рабочих, а следовательно, и такой ресурс, как рабочая сила, также окажется недоиспользованным. Интуитивно понятно, что доход, полученный от дорогостоящих шкафов, может и не покрыть издержки, связанные с нерациональной тратой ресурсов. А это означает, что задача извлечения максимальной прибыли даже в таком простом примере не тривиальна, и она может оказаться совсем сложной в примере с десятками видов продукции и сотнями видов ресурсов. Другими словами, необходимо более тщательно обдумать задачу и начать с построения формальной математической модели, то есть в компактной форме описать процесс расходования ресурсов и способ расчета прибыли.

Математическая модель задачи

Попробуем формально описать производственный процесс как расход ресурсов и выпуск объема продукции. Каждый ресурс имеется в определенном объеме. Обозначим величиной D_k объем ресурса с номером k . Допустим, наше предприятие способно выпускать несколько видов продукции. Обозначим через A_j – объем выпуска продукта с номером j . Пусть единица продукта с номером j нуждается в d_{kj} – объеме ресурса k . Тогда суммарная потребность продукта с номером j в ресурсе с номером k дается выражением $A_j d_{kj}$. Предположим, что каждый продукт нуждается в каждом ресурсе в рамках нашей модели, а если реальной потребности нет, то соответствующий коэффициент d просто равен нулю. Если для каждого ресурса посчитать, сколько его тратится для каждого вида продукции, и найти суммарные затраты, то понятно, что сумма не может быть больше общего запаса ресурса. Это утверждение, например для ресурса с номером единица, можно записать следующей формулой:

$$A_1 d_{11} + A_2 d_{12} + \dots + A_n d_{1n} \leq D_1.$$

Если записать в таком виде потребность по каждому ресурсу, то получим общую систему неравенств, ограничивающих возможности предприятия по выпуску продукции. А выглядеть система неравенств будет так:

$$\begin{cases} A_1 d_{11} + A_2 d_{12} + \dots + A_n d_{1n} \leq D_1 \\ A_1 d_{21} + A_2 d_{22} + \dots + A_n d_{2n} \leq D_2 \\ \dots\dots\dots \\ A_1 d_{m1} + A_2 d_{m2} + \dots + A_n d_{mn} \leq D_m \end{cases}$$

Здесь n – количество видов производимой продукции, m – количество используемых ресурсов. Набор величин A_j можно назвать планом выпуска, и тогда записанная выше система неравенств описывает все возможные производственные планы. Таким образом, мы получили модель, описывающую расход ресурсов. Следующая задача – описание получаемой выручки.

Способ расчета выручки

Выручка от продажи всех шкафов равна стоимости одного шкафа, умноженной на количество произведенных шкафов. То есть выручка

от одного вида изделия равна произведению цены изделия на количество изделий. Если цену j -го изделия обозначим через p_j , то общая выручка будет равна $A_j p_j$, и тогда выручка от всего объема запланированного производства будет выражаться формулой

$$A_1 p_1 + A_2 p_2 + \dots A_n p_n = P.$$

Это выражение называется ценовой функцией. Понятно, что, выбирая различные производственные планы в рамках того, что позволяет система ограничений, мы получаем различные значения ценовой функции. Можно констатировать, что теперь в нашем распоряжении есть математический аппарат, позволяющий точно сформулировать условие задачи.

Точная формулировка: при заданной системе ограничений на затраты ресурсов определить план, обеспечивающий максимальное значение ценовой функции.

Вот такие достаточно объемные рассуждения потребовались только для того, чтобы получить математически грамотную постановку задачи. И заметьте, что формулировка чисто алгебраическая, и теперь самое время приступить к выполнению обещания продемонстрировать возможность геометрического подхода. Нам необходимо показать возможность тождественной постановки задачи, но в геометрических терминах.

Итак, где здесь геометрия?

Мы написали набор ограничений для произвольного количества видов продукции и для произвольного количества ресурсов. Давайте для удобства анализа ограничим количество товаров двумя и запишем короткую форму системы ограничений:

$$\begin{cases} A_1 d_{11} + A_2 d_{12} \leq D_1 \\ A_1 d_{21} + A_2 d_{22} \leq D_2 \\ \dots\dots\dots \\ \dots\dots\dots \\ A_1 d_{m1} + A_2 d_{m2} \leq D_m \end{cases}$$

Возьмем любое неравенство, например первое: $A_1 d_{11} + A_2 d_{12} \leq D_1$. Легко заметить, что оно задает область координатной плоскости, ле-

жащую ниже прямой, описываемой уравнением: $A_1 d_{11} + A_2 d_{12} = D_1$. Значит, все множество решений неравенства – это полуплоскость. Тогда с учетом того, что отрицательные решения для нашей задачи не имеют смысла, все множество решений системы ограничений лежит на пересечении полуплоскостей, ограничиваемых соответствующими прямыми с положительным квадрантом координатной плоскости.

Пример. Пусть дана следующая система ограничений:

$$\begin{cases} Y \leq 2 \\ 5Y + 2X \leq 11 \end{cases}$$

Здесь речь идет о производстве двух продуктов X и Y при двух ресурсах, запасы которых равны 5 и 2 единицы. При этом продукт X в первом ресурсе не нуждается. Множество возможных производственных планов на рисунке ниже лежит в заштрихованной области.

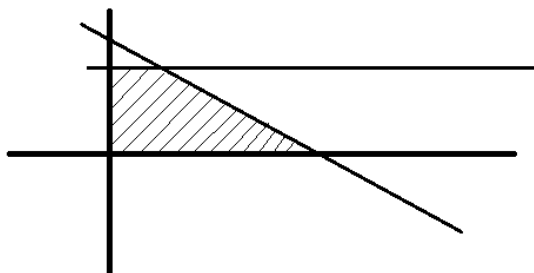


Рис. 4.3. Геометрическая иллюстрация

Как уже было сказано, каждая точка заштрихованной области соответствует некоторому значению ценовой функции. Мы не определили вид функции, так как это не важно, существенно значимо только её существование. Принципиально нужную точку можно определить прямым перебором, что может оказаться очень затратным. Заметим, однако, что даже визуально не все точки области одинаковы. Есть внутренняя область, есть точки на ребрах фигуры, и есть вершины. И оказывается, что максимальное значение, как и следовало ожидать от критической величины, находится в критической же точке, а именно в вершине.

Проблема для самостоятельного исследования

Утверждение, сформулированное выше, не так сложно для анализа, поэтому попробуйте сами доказать, что максимальное значение оценочной функции необходимо искать среди вершин фигуры.

Проблема для самостоятельного исследования

В нашем примере построенная фигура имеет вид выпуклого многоугольника. Попробуйте выяснить, существуют ли разумные ограничения (соответствующие какой-то реальной производственной задаче), дающие геометрически не выпуклый многоугольник.

Вне зависимости от того, смогли вы или нет решить сформулированные выше проблемы, мы можем утверждать, что смогли дать геометрическую формулировку задачи для случая двух видов продукции. Что изменится, если потребуется решить задачу для трех продуктов или более? Принципиально ничего. Ограничение, описывающее производство трех видов продукции, будет выглядеть так:

$$aX + bY + cZ \leq D.$$

Это выражение описывает трехмерное полупространство, ограниченное плоскостью. Тоже геометрия, хотя и немного более трудно представимая. Выражение от n -переменных будет, таким образом, описывать многомерное полупространство, в котором наша цель – многомерная фигура, принципиально ничем не отличимая от многоугольника из нашего примера, точно так же остается задача нахождения вершин и анализа в них ценовой функции, а значит, мы действительно получили наглядную геометрическую формулировку. И, следовательно, можно переходить к следующему примеру.

Задача. Раскладывание колечек по штырькам

Первое условие. В каждую клетку шахматной доски воткнут штырек. В нашем распоряжении есть некоторое количество колечек, которые можно нанизывать на штырьки, причем на один штырек можно нанизать несколько колечек. Требуется подсчитать, сколькими способами можно распределить все эти колечки по штырькам. Два распределения считаются разными, если на двух соответствующих штырьках находится разное количество колечек.

Прежде всего обратим внимание на одно важное обстоятельство. Два разбиения считаются различными, если в них хотя бы один штырек содержит различное количество колечек и ничего не сказано о взаимном расположении штырьков. Из этого следует, что способ расположения штырьков роли не играет и информация о квадратной доске – это не более чем способ ввести нас в заблуждение. Вытянем дос-

ку в линию, линию представим в виде массива, а кольца пусть будут единицами. Тогда нанизывание колечка на штырек сведется к добавлению еще одной единицы к элементу массива. С учетом сказанного условие можно переформулировать так:

Второе условие. Дан одномерный массив длины N и число L . Найти все возможные различные разложения числа L по элементам массива. Массив считается разложением числа L , если сумма его элементов равна L .

А можно сформулировать это же условие вообще без применения программистской терминологии.

Третье условие. Дано число L . Найти все возможные разбиения этого числа на суммы положительных чисел, при условии что количество слагаемых не превышает N и перемена места слагаемого дает другое разбиение.

Это хороший пример формализации, то есть записи условия на специальном языке, слова которого имеют четко определенный смысл. Сформулировать задачу несколькими способами всегда полезно. Это дает дополнительную информацию к размышлению, а иногда удачная переформулировка даже может подсказать и очень изящное решение. В нашем случае, как минимум, удалось избавиться от лишнего понятия доски и привести все к понятиям, легко переводимым на язык программирования.

Третье условие удобно для понимания, но оно, пожалуй, более математическое, чем программистское, поэтому мы далее воспользуемся вторым. По типу определения требуется построить объект с установленными свойствами (массив, сумма элементов которого равна заданному числу). Такие задачи решаются двумя способами. Во-первых, можно придумать метод получения всех возможных объектов и из них выбрать те, которые удовлетворяют поставленному условию. Во-вторых, можно придумать способ получения объектов, уже удовлетворяющих поставленному условию. Обычно это делается так: строится первый объект, уже удовлетворяющий заданному условию, затем последующий строится из имеющегося. Это сложнее, и, кроме того, при таком подходе надо еще доказать, что в построенной последовательности все объекты будут удовлетворять условию.

Мы используем в решении задачи оба подхода, получим два решения и посмотрим на их плюсы и минусы.

Первое решение. Задача по формулировке похожа на следующую: найти среди всех N -значных чисел такие, что сумма их цифр равна заданному числу. Действительно, массив, по элементам которого рас-

кладывается число L , можно представить себе как число по основанию L , а его элементы – как цифры. Тогда идея решения выглядит так: требуется получить все N -значные числа по основанию L , а на экран вывести только те, сумма цифр которых равна L .

Для того чтобы получить, например, все десятичные числа, не превышающие заданного числа, необходимо взять нуль и прибавлять к нему единицу, пока не получится заданное число. Такая процедура гарантирует полный перебор всех десятичных чисел от единицы до заданного числа. Очевидно, что процесс не зависит от основания системы счисления. Опишем его для числа с произвольным основанием.

Возьмем число по основанию L и запишем его цифры в массив. Прибавим к первому разряду единицу. Возможны две ситуации. Первый разряд меньше величины основания, тогда все нормально и никаких дополнительных действий выполнять не надо. Но, возможно, величина первого разряда превысит величину основания, тогда необходимо этот разряд обнулить и единицу перенести в следующий, однако, возможно, в следующем разряде также возникнет переполнение, тогда перенос нужно выполнять до тех пор, пока не будет найден разряд, такой что при добавлении к нему единицы переполнения не возникнет. Заметим, что один раз возникнет неустраняемая ситуация переполнения со старшим разрядом, но это как раз и будет означать, что все числа уже получены. Запишем алгоритм прибавления единицы:

```
Прибавим единицу к первому разряду.  
От первого разряда до предпоследнего делать  
    Если значение текущего разряда больше основания,  
        то Текущий разряд обнулить.  
        К следующему разряду прибавить единицу.  
Если Значение старшего больше основания,  
То Значение=основанию.
```

Замечание. Алгоритм несколько отличается от алгоритма сложения двух чисел. При сложении столбиком двух разрядов, если возникает переполнение, то, выполняя перенос, мы не обнуляем текущий разряд числа результата. В текущем разряде может остаться число, меньшее основания. Различие это происходит от того, что в нашем алгоритме прибавляется не произвольное число, а единица.

Построенный алгоритм верен, но это наиболее очевидное решение, и оно, как все очевидные решения, обладает недостатком. Если в какой-то момент окажется, что текущий разряд имеет значение, меньшее или равное основанию, то последующие разряды также не могут

иметь значения большего основания, и проверка от первого до последнего разряда теряет смысл. Это небольшой недостаток. С учетом того, что длина массива невелика, выигрыш в раннем завершении проверки несуществен. Но если вам нравится бороться за эффективность, то можно предложить другой алгоритм. Кстати, желание придумать более быстрый алгоритм всегда похвально. Итак, немного более эффективный алгоритм:

```
К первому разряду прибавить единицу.  
Начиная со второго и до тех пор, пока значение разряда больше  
основания на единицу, Делать  
    Если разряд не последний, То  
        Обнулить разряд  
        Следующий увеличить на единицу  
Иначе значение разряда равно значению основания.
```

Небольшая техническая проблема с увеличением текущего числа на единицу решена. Теперь можно приступить к разработке общего алгоритма. Ядро его составит цикл, внутри которого прибавлением единицы формируется новое N -значное число по основанию L . Затем алгоритм просчитает сумму цифр этого числа, и если она равна L , то массив-число распечатывается. Далее записан алгоритм:

```
Инициализировать массив нулями (исходное число)  
Пока не получено последнее число, Делать  
    Выполнить алгоритм прибавления единицы  
    Найти сумму элементов массива  
    Если сумма равна  $L$ , то распечатать массив на экран.
```

Решение, полученное ранее, имеет одно серьезное достоинство и один серьезный недостаток. Достоинство его – в простоте логики, а недостаток – в том, что всех возможных чисел много больше, чем нужных. Если доска велика и колечек много, то даже на хорошем компьютере программа может «подвисать». И этот недостаток настолько серьезен, что выигранное преимущество, пожалуй, теряется. Поэтому сейчас подумаем о втором подходе. Его сущность заключается в том, чтобы из одной подходящей комбинации строить следующую подходящую, игнорируя массу ненужного. Это, конечно, существенно сложнее логически, но выигрыш в скорости будет огромный.

При таком подходе задача выглядит достаточно сложной, поэтому подумаем, как ее можно упростить и что, собственно, ее усложняет. Ясно, что это задача – на построение различных комбинаций из значимых чисел (то есть больших нуля), нули нас не интересуют, так как они не вносят никакого вклада в общую сумму, но они могут в массиве

располагаться различными способами, формируя различные числа. Впрочем, для удобства исследования более удобно говорить не о расположении нулей, а о расположении значимых чисел. Следовательно, комбинации будут строиться в два этапа: во-первых, необходимо построить комбинацию позиций, в которые можно ставить значимые числа, а затем для заданной комбинации позиций необходимо построить все возможные разложения числа L . То есть задача явно разбивается на две. Поговорим о первой.

Как построить комбинацию позиций? Если допустимую позицию помечать единицей, а недопустимую – нулем, то комбинация позиций, – это двоичное число длины N . Задача перебора всех двоичных чисел очень проста.

Задача для самостоятельного решения

Собственно, задачу перебора всех двоичных чисел мы уже обсудили чуть выше, когда говорили о переборе чисел по основанию L . Но двойка – совершенно особое основание, так как оно минимальное из возможных. Это обстоятельство должно заметно упростить программу. Попробуйте написать такую программу с учетом особенности основания два.

Как строить разложение? Исходные данные – это массив, содержащий числа, и массив–двоичное число, определяющий, в какие позиции можно записывать числа. Как именно строить очередное разложение? Можно попытаться придумать формулу, которая, исходя из уже полученного разложения, вычисляла бы числа для каждой позиции нового, а можно записать в допустимые позиции единицы, а затем добавлять по какому-либо правилу единицы, пока не будет получена нужная сумма. Мы пойдем вторым путем, а пока сформируем важное правило.

Если от вас требуется построить набор чисел с заданной количественной характеристикой (например, определенной общей суммой, как в нашей задаче), то можно построить набор, в котором данная характеристика меньше, чем требуемая, и затем небольшими изменениями подгонять ее к требуемому значению.

Наиболее очевидный способ построения минимальной суммы – это простое заполнение единицами массива числа. К примеру, вот так, как в таблице:

В	0	1	1	0	1	1
А	0	1	1	0	1	1

Здесь массив B – это массив двоичного числа, а массив A – массив, сумму элементов которого требуется довести до L . Значение этой исходной суммы может быть больше, чем L , может быть равно L , а может быть меньше L . Рассмотрим все три случая.

1. Если значение суммы равно L , то мы уже получили одно из допустимых разложений, и прибавление еще одной единицы ничего не даст.
2. Если значение суммы меньше L , то решение еще не получено, но добавлением некоторого количества единиц мы получим решение, и, возможно, вариантов будет много.
3. Если значение суммы больше L , то решений нет, прибавление даже одной единицы только ухудшит ситуацию, эта комбинация допустимых позиций тупиковая, ее можно не рассматривать.

Примерную структуру алгоритма можно представить в виде цикла, работающего до тех пор, пока не получено последнее двоичное число, то есть число, состоящее из одних единиц. Для каждого двоичного числа необходимо построить минимальное разбиение, состоящее из одних единиц, и если окажется, что это минимальное разбиение уже дает нужную сумму или нужная сумма может быть из нее получена, то выполняется работа по дополнению минимальной суммы до значения L . А теперь самый сложный вопрос. Как дополнять минимальную сумму?

Предположим, очередная комбинация построена и заполнена единицами. Предположим, что количество единиц равно H . Тогда дополнительно не хватает $L-H$ единиц. Мы можем добавить в первый разряд 1, 2, 3, ..., $L-H$ единиц. Предположим, уже добавили в первый разряд t_1 единиц, тогда не хватает еще $L-H-t_1$ единиц, которые можно восполнить во втором, третьем, ... разрядах.

Наша задача – перебрать все возможные разложения. Следовательно, необходимо для первого разряда рассмотреть все возможные дополнительные суммы от 1 до $L-H$, а для второго – все возможные дополнительные суммы от 1 до $L-H-t_1$, для третьего – все возможные дополнительные суммы от 1 до $L-H-t_1-t_2$.

То есть если длина стороны квадрата равна, например, 2 и, соответственно, длина одномерного массива, представляющего этот квадрат, равна 4, то полный перебор всех разложений для очередной допустимой комбинации позиций будет состоять из четырех вложенных циклов. Внешний из них имеет границы изменения параметра от 1 до 4, следующий – от 1 до 3 и т. д. Это для длины стороны, равной 4.

Длина стороны может быть различной, следовательно, количество вложенных циклов так же должно быть различным. Построить переменное количество циклов можно с помощью механизма рекурсии. Например, так:

```
Процедура Много_Циклов(параметр)
Если параметр меньше длины массива, то
    Для всех значений от параметра до длины массива Делать
        Вызывать процедуру Много_Циклов(параметр+1)
```

Это примерная конструкция, не вполне отвечающая нашим целям. Поэтому следующим шагом проанализируем ее, посмотрим, что в ней не соответствует цели и как ее усовершенствовать.

Во-первых, заметим, что данная конструкция просто показывает, как организовать произвольное количество вложенных циклов, здесь нет ни одной содержательной команды, ради которых, собственно, эти циклы и создаются. Наша же главная содержательная команда – это прибавление единицы к разряду. Добавим ее и перепишем алгоритм следующим образом:

```
Процедура Много_Циклов(параметр)
Если параметр меньше длины массива, то
    Для всех значений от параметра до длины массива Делать
        К текущему разряду прибавить единицу.
        Вызывать процедуру Много_Циклов(параметр+1)
```

Этот алгоритм выглядит лучше, но если вы не поленитесь и напишете по нему программу, то результат будет отрицательным. Попробуем найти расхождение между алгоритмом и изложенной выше идеей.

В идее сказано, что к каждому разряду может быть прибавлено много единиц. Но построенный алгоритм к каждому разряду прибавит не более одной. Действительно, в текущем вызове процедуры ее цикл пробегает массив один раз, следовательно, в текущем цикле к конкретному разряду будет прибавлено не более одной единицы, а следующий вызов начинается свою работу уже со следующего разряда. Можно решить эту проблему, передавая в следующий вызов тот же параметр, который получил и данный вызов, но тогда рекурсивные вызовы никогда не закончатся, чтобы в этом убедиться, посмотрите на условие завершения рекурсии. Чтобы все-таки завершить рекурсию, мы доопределим условие выхода, указав границу увеличения для разряда. Очевидно, что рекурсивную процедуру нет смысла вызывать, если сумма чисел массива достигла L . Обозначив для краткости длину массива через N , запишем новый вариант алгоритма:

```
Процедура Много_Циклов(параметр)
Если параметр меньше N и Сумма меньше L, то
Для всех значений от параметра до длины массива Делать
    К текущему разряду прибавить единицу
Вызывать процедуру Много_Циклов(параметр)
```

Алгоритм стал еще более содержательным, но продолжим анализ. Еще одна ошибка бросается в глаза сразу. Если какая-то величина используется, то ее значение должно быть определено. В нашем тексте используется переменная *Сумма*, а ее значение никак и нигде не изменяется. Ее изменение учесть легко, как только к разряду (любому) прибавляется 1, сумма так же должна измениться на единицу. Новый вариант алгоритма:

```
Процедура Много_Циклов(параметр)
Если параметр меньше N и Сумма меньше L, то
    Для всех значений от параметра до длины массива делать
        К текущему разряду прибавить единицу
        К Сумме прибавить единицу
Вызывать процедуру Много_Циклов(параметр)
```

А теперь произойдет удивительная вещь. Следующую довольно серьезную ошибку мы исправим, исходя из свойств рекурсии, независимо от смысла задачи. Выход из рекурсивного вызова предполагает, что данные, с которыми работала процедура до того, как она вызвала сама себя, сохранились. Это не всегда так, бывает в программировании всякое, но вообще-то смысл рекурсии в том, что ее механизм берет на себя заботу о сохранности промежуточных данных. В нашем алгоритме до вызова увеличивается разряд и увеличивается значение суммы. Вернем их значения обратно, то есть после вызова уменьшим значения разряда и суммы.

И последнее, сумму можно сделать глобальной переменной, а можно и локальной. Локальные величины всегда лучше, но возиться с передачей массива не очень хочется, поэтому массив у нас будет глобальным, а переменную *Сумма* сделать локальной труда не составляет. Впрочем, это уже вопрос вкуса. С учетом сказанного наш алгоритм окончательно выглядит так:

```
Процедура Много_Циклов(Сумма, параметр)
Если параметр меньше N и Сумма меньше L, то
    Для всех значений от параметра до длины массива Делать
        К текущему разряду прибавить единицу
        К Сумме прибавить единицу
    Если Сумма = L, то массив Числа вывести на экран
Вызывать процедуру Много_Циклов(параметр)
```



Отнять единицу от разряда

Отнять единицу от Суммы

Итак, главная смысловая проблема решена, и у нас есть уже решенная задача о получении всех двоичных чисел (если вы выполнили это самостоятельно). Осталось прописать общую структуру алгоритма. Подробно разбирать его не будем, это всего лишь небольшая техническая работа.

Заполняем нулями массив двоичного числа

Пока не получено последнее двоичное число, Делать

Получить очередное двоичное число и сразу посчитать минимальную сумму

Если минимальная сумма меньше или равна L , То

Массив Числа заполнить единицами в соответствии с массивом двоичного числа

Если полученная минимальная сумма равна L , то массив

Числа вывести на экран, так как это искомый результат

Вызывать рекурсивную процедуру дополнения массива Числа

Проблема для реализации

Выше достаточно подробно описаны два алгоритма решения задачи на псевдокоде. Реализуйте эти алгоритмы в виде программы.

В заключение

Обратите внимание на уровень сложности разработанного алгоритма. Это работает золотое правило механики. Сущность его в том, что выигрыш в одном компенсируется проигрышем в другом. Наш выигрыш в эффективности программы потребовал существенно более сложной логики.



ГЛАВА 5.

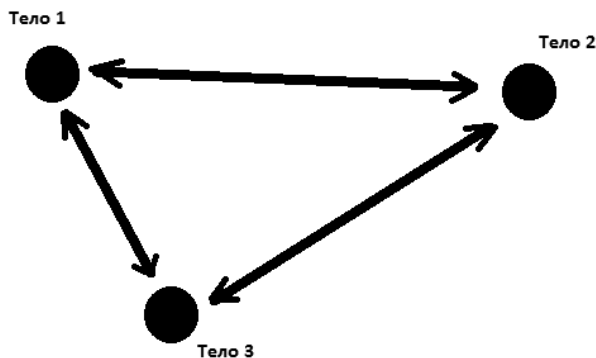
Моделирование физических процессов

В этой главе мы рассмотрим очень сложный класс задач – задачи моделирования физических процессов. Но, прежде чем приступить к работе, необходимо понять очень важную вещь. Программисты, моделируя физический процесс, явление, в действительности имеют дело не с явлением, а с его математической моделью, и хотя все рассуждения о том, как должна работать создаваемая модель, конечно, основаны на понимании физической сути, все равно это будет модель для модели, то есть программистская реализация математического представления физического явления. Чтобы понять принципиальность высказанного положения, разберем несколько задач и начнем с классической задачи движения нескольких тел в поле тяготения. Математика такую ситуацию описывает системой дифференциальных уравнений, мы же попробуем обойтись более простым аппаратом.

Модель движения системы тел в гравитационном поле

Условие задачи. Несколько тел известной массы с известными векторами начальных скоростей движутся в пустом пространстве (если другие тела и существуют, то их влиянием можно пренебречь), необходимо построить траектории их движения (см. рис. 5.1).

Задачи моделирования физических процессов обладают особенной сложностью. Связано это отчасти с ошибочным пониманием того, что требуется получить. Например, очень часто человек, моделирующий что-то, задается вопросом, а что мы должны увидеть, когда модель заработает? Вопрос совершенно неверный. Если бы мы знали, что долж-

**Рис. 5.1.** Система трех тел

но получиться, то создавать модель не было бы необходимости. Вторая, часто встречающаяся ошибка повторяет первую – программист, моделируя движение, добивается того, чтобы была правдоподобная картинка движения, но при этом исходит из каких-то соображений, не имеющих отношения к природе рассматриваемого движения. Если дать такому программисту задачу на моделирование движения тела в поле тяготения Земли, он может рассуждать так: тело движется с течением времени сначала вверх, затем, достигнув максимальной высоты, падает вниз. Это все так, но не совсем. Во-первых, так называемая максимальная высота неизвестна в процессе движения (её нельзя взять с потолка), а во-вторых, оно не просто движется вверх и вниз, оно движется по определенному закону, и в этом соль. Поэтому сейчас, прежде чем идти на борьбу с трудной задачей, потренируемся на более простой. И в качестве таковой возьмем модель движения тела в поле силы тяжести под углом к горизонту (см. рис. 5.2).

Единственное исходное данное для процесса – это вектор начальной скорости. Мы определим его через скалярную величину скорости и величину угла наклона. Далее воспользуемся красивой математической идеей разбиения движения в плоскости на два независимых движения: движение в вертикальном направлении под воздействием силы тяжести и равномерное прямолинейное движение по горизонтали. Для того чтобы выполнить такое разбиение, сделаем две вещи: во-первых, вектор скорости разложим на две составляющие: горизонтальную и вертикальную, во-вторых, вертикальную и горизонтальную координаты будем вычислять независимо друг от друга по соответствующим им законам.

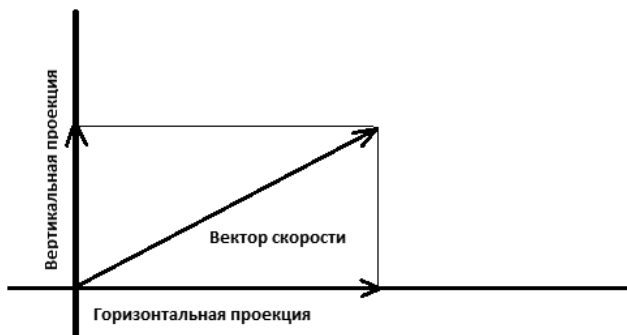


Рис. 5.2. Проекции вектора скорости

Величина, увязывающая два движения в одно целое, – это время. Перед началом движения время равно нулю, затем в процессе оно пошагово изменяется на малую величину. С точки зрения физики время в любом физическом процессе течет непрерывно, но мы не умеем моделировать непрерывные процессы, компьютер – по своей природе устройство дискретное. Но если временной интервал, принятый за единицу измерения времени, взять достаточно малым, то дискретность будет не заметна.

Вычислить проекции вектора скорости v_x и v_y
 $t = 0$ (через t обозначим время, через Δt малое изменение времени)
 $t = t + \Delta t$
 $x = v_x t$; $y = v_y t - gt^2/2$ (Расчет необходим для инициализации координат перед входом в цикл)
 Пока координата $y > 0$, делать
 $t = t + \Delta t$
 $x = v_x t$; $y = v_y t - gt^2/2$

Рассмотренный пример нужен только для того, чтобы показать главную идею компьютерной модели физического процесса – основой модели является математически корректная запись физических законов, а не ожидаемый визуальный результат.

К сожалению, именно это часто представляет существенную сложность, что мы и увидим на задаче о движении тел в поле силы тяготения. Необходимо также сказать, что потребности математической модели быстро наталкиваются на ограниченность средств, которыми они реализуются. С этим мы столкнулись даже в рассмотренном простом примере. Физика требует, чтобы время было непрерывным, математика против этого требования физики не возражает, а мы, про-

граммисты, его удовлетворить не можем. Но конфликт разрешился достаточно легко. От абсолютной точности легко отказаться, а если пожертвовать точностью, то отпадает потребность и в непрерывности.

Второй конфликт, с которым мы столкнемся, – конфликт между действием и взаимодействием. Это уже и сложнее понять, и сложнее победить. Закон всемирного тяготения, который лежит в основе модели, – это закон взаимодействия, он описывает, как тела взаимодействуют. Первое тело воздействует в некоторый момент времени на второе тело, а второе тело в этот же момент, до того как воздействие первого изменило его состояние, воздействует на первое. Иначе говоря, взаимодействие – это мгновенные одновременные воздействия каждого тела на все остальные. Но компьютер не может выполнять никаких два действия одновременно, а только последовательно операцию за операцией.

Это, в свою очередь, означает, что какое-то тело нам придется выделить и его воздействия рассматривать в первую очередь. За один шаг работы модели такое выделение, наверное, не приведет к заметному искажению, но нельзя гарантировать, что искажения (а они будут) не накопятся очень быстро. Причем даже неизвестно, как быстро они станут заметными. Чтобы оценить скорость накопления погрешности, придется провести серьезную математическую работу.

Может быть, впрочем, погрешности будут и невелики, или учесть их окажется не слишком сложно, но проблема выглядит принципиальной. Одно из тел получает больше прав, становится более значимым, чем другие, а это уже говорит о том, что мы моделируем не закон всемирного тяготения, а что-то несколько иное. Наверное, уже понятно, чем действие отличается от взаимодействия, но все же приведем еще одно небольшое рассуждение. Пусть даны два тела ТЕЛО1 и ТЕЛО2. Предположим, мы желаем рассчитать результат их взаимодействия в некоторый момент времени, который для удобства назовем нулевым. Сделаем это так:

Учтем воздействие ТЕЛО1 на ТЕЛО2, для чего

Рассчитаем ускорение, приобретаемое ТЕЛОМ2 в результате действия силы со стороны ТЕЛО1.

Рассчитаем перемещение ТЕЛО2 в соответствии в формулой, данной физикой для ускоренного движения.

Учтем воздействие ТЕЛО2 на ТЕЛО1 аналогично тому, как это сделано ранее.

На первый взгляд все очевидно, но здесь кроются две ошибки.

1. Первый пункт расчетов действительно проводится для данного нулевого момента времени, а второй – уже для другого. Конечно, можно возразить, что появляющуюся погрешность можно сколь угодно сильно сократить, уменьшая интервал времени между расчетами первого действия и второго, но дело в том, что чем меньше интервал, тем большее количество таких интервалов уложится в период наблюдения за движением тел. Уменьшая значение погрешности, увеличиваем количество слагаемых. Совершенно не очевидно, что выигрыш от уменьшения абсолютного значения слагаемых погрешностей перевесит проигрыш от увеличения количества слагаемых.
2. Вторая ошибка заключается в серьезнейшей логической неувязке. С одной стороны, мы отказываемся от непрерывного расчета взаимодействия и переходим к учету последовательности разовых моментальных воздействий, а с другой стороны, предполагаем использование формулы расчета пути при равноускоренном движении, которая может использоваться только в случае *непрерывного силового воздействия!*

Вся описанная путаница получилась от того, что, отказываясь моделировать непрерывное движение, мы продолжали пользоваться непрерывной математикой. А нам нужно быть последовательными, все законы и все расчеты должны стать дискретными. Утончим модель следующим образом: тела не взаимодействуют непрерывно; имеет место последовательность мгновенных воздействий тел друг на друга, в результате которых возрастают их скорости, а затем в течение некоторого малого времени они движутся равномерно и прямолинейно, с полученной скоростью до следующего воздействия, в котором скорости опять изменяются.

Новая модель уже лучше, но придется скорректировать и ее. Проблема в скорости. В равноускоренном движении скорость вычисляется так: $v = v_0 + (t - t_0) \cdot a$. Из этой формулы следует, что скорость в нашей модели вообще не будет изменяться, так как мгновенное воздействие происходит за нулевое время, то есть $t = t_0$.

Придется во имя принципа дискретности и под флагом борьбы с нарастанием погрешности пойти еще на одну корректировку модели, а именно разделим процесс изменения скорости и процесс изменения координат. Будем полагать теперь, что в течение некоторого малого интервала времени скорость изменяется в соответствии с законами физики, но на равномерное движение тела внутри данного интервала влияния не оказывает, а в последней точке интервала скорость тела

скачком меняется на накопленную. Таким образом, внутри каждого расчетного интервала каждое тело, участвующее в движении, движется по закону равномерного и прямолинейного движения, но для каждого интервала формула этого закона своя, ее числовые параметры определяются результатом изменения скорости на предыдущем расчетном интервале.

Думаю, согласитесь, что мы провели достаточно сложные рассуждения. Для профессиональных математиков внешне все было бы проще. Была бы записана система дифференциальных уравнений, и выбран численный метод ее решения. Получение хорошего результата было бы гарантировано всей мощью математического аппарата, но тогда сущность перехода от непрерывности к дискретности оказалась бы скрыта, а это, как вы видите, совсем не тривиальная вещь.

На данный момент, пожалуй, все проблемы решены и можно записать алгоритм работы модели:

Пока не завершено наблюдение, делать

Для каждого тела

Рассчитать ускорение, получаемое им от воздействия других тел.

Рассчитать скорость, которое оно получит за небольшой временной интервал от ускоренного движения.

Для каждого тела

Рассчитать изменения координат, полагая, что тело движется равномерно и прямолинейно с текущей скоростью.

Прорисовать участок траектории или прорисовать точку в новом положении тела.

Проблема для самостоятельного исследования

В приведенном алгоритме ни слова не сказано о важном обстоятельстве – двигались ли тела до начала наблюдения за ними. В действительности разумно предположить, что каждое из тел обладало какой-то начальной скоростью (точнее, вектором скорости). Подумайте, что это меняет и как учесть скорость тел, приобретенную ими до момента наблюдения.

В заключение

Решена достаточно сложная задача, показаны серьезные проблемы и важные подходы к их решению, но это еще не все. Все вопросы физического моделирования, конечно, на одном примере не раскрыть, но совершенно необходимо упомянуть о грамотном визуальном отображении информации. Если, например, строится траектория движе-

ния и она при этом не умещается на экране, говорить о соответствии с физическими законами нет большого смысла. Если строится модель процесса, но визуально все происходит так быстро, что глаз не успевает ничего увидеть, то в этом опять же нет смысла. Конечно, не все физические модели подразумевают визуальное представление, как раз наоборот, для физиков часто важнее числа, чем прямо наблюдаемые эффекты, но если визуальное представление все же нужно, то необходимо обдумать его масштабы. А именно масштабы времени и расстояния.

Вернемся к задаче визуального построения траектории. Физика для расчета любых величин пользуется системами единиц. Расстояние измеряется в метрах, время – в секундах. Но у экрана монитора своя единица измерения – пиксел. Сколько один пиксел вмещает в себя метров – это условность. Поэтому вы вполне можете для вашего процесса ввести такой масштаб, какой вам удобен. С единицей времени произвола будет еще больше. Фактически единицей времени является шаг цикла, в котором реализуется моделируемый процесс. Но такая развитая демократия в вопросах масштаба расстояний и времени влечет за собой и серьезные проблемы.

Если вы для вертикальной оси координат и для горизонтальной установите разные масштабы, то искажения реального физического процесса могут стать слишком большими. Далее, представьте, что в вашей программе моделируются два процесса, для каждого созданы свои циклы, в теле которых выполняются некоторые вычисления, и предположим, что вычисления, моделирующие первый процесс, существенно сложнее, чем вычисления, моделирующие второй процесс (требуют больше времени от аппаратуры). Из этого автоматически следует, что скорость течения времени первого процесса самым серьезным образом отличается от скорости течения времени второго процесса, и если при этом моделируемые процессы взаимодействуют друг с другом, например обмениваются энергией, то модель будет сильно искажена.

Сделаем акцент еще на одном важном выводе. Для построения программисткой модели физического процесса крайне важно построить удачную математическую модель, и здесь выбор не всегда однозначный. К решению проблем моделирования, как правило, можно подходить с разных сторон, и если вас интересует качественная картина изучаемого явления, то совершенно не обязательно использовать сложную математику. Для иллюстрации этого утверждения рассмотрим еще одну интересную задачу.

Задача о сложении прямого и отраженного колебаний

Колебательное движение является одним из самых распространенных видов движения в природе и, возможно, самым сложным. Чтобы в этом убедиться, достаточно посмотреть на запись законов, его описывающих. Квалифицированные физики с этим утверждением, наверное, не согласятся, но мы подходим к любой проблеме как программисты, и для нас моделирование колебательного движения – задача, намного более сложная, чем ускоренного или равномерного. Сейчас мы рассмотрим частный случай сложения двух колебаний: некоторого исходного, будем называть его прямым, и колебания, получающегося отражением прямого от непроницаемого препятствия, будем называть его отраженным.

Уточним условия

Формулировка предыдущей задачи выглядела достаточно ясно в силу простой формы движения и закона, им управляющего. Здесь ситуация несколько иная, поэтому стоит уточнить, что именно необходимо и что реально. Во-первых, мы знаем, что колебательное движение бывает поперечным и продольным, во-вторых, его характер зависит от среды, так как это движение именно в среде, в-третьих, большое значение имеет размерность пространства, в котором распространяется колебание, в-четвертых, форма препятствия также небезразлична. С учетом даже сказанных моментов задача становится почти необозримой или, во всяком случае, очень сложной. Но если поставить перед собой цель обнаружить качественную особенность суммарного движения, то, наверное, от некоторых вещей можно отвлечься. И такая особенность есть. Физика утверждает, что суммарное колебание будет представлять собой стоячую волну. Исходя из этого, условия задачи можно упростить более чем существенно.

Во-первых, нам достаточно двумерного пространства – в котором колебание распространяется вдоль прямой (для наблюдения стоячей волны двумерного пространства вполне достаточно). В этом случае непроницаемое препятствие – просто точка на линии. Совершенно нет никакой необходимости рассматривать оба типа колебаний: продольное и поперечное. Выберем поперечное как более наглядное. В отношении среды вполне можно отказаться от свойства непрерывности и договориться, что линия, вдоль которого распространяется

колебание, состоит из точек, расставленных на прямой через равные промежутки.

Немного справочной информации

Мы не физики, как уже выше было сказано, поэтому есть смысл пояснить главные понятия: колебание в точке и колебательное движение, стоячая волна.

Точка. Математически точка участвует в колебательном движении, если её положение с течением времени описывается законом синуса или косинуса в следующем виде:

$$Y = A \sin(\omega t + \varphi_0).$$

Здесь: A – амплитуда, то есть величина, показывающая максимальное отклонение точки от первоначального положения, ω – частота колебания, величина показывающая как много колебаний укладывается в единицу времени. t – физическое время, φ_0 – так называемая начальная фаза – точка с которой начинается колебание (колебание не обязательно наблюдается от нулевой точки). Если вместо синуса в формуле появится косинус, это не внесет в физический смысл задачи ничего существенного, поэтому ограничимся функцией синуса.

Колебательное движение вдоль линии в двумерном пространстве (волновое движение) – это такое движение всех точек линии, в результате которого вдоль линии происходит перенос энергии, а поперечное колебательное движение, – это такое при котором точки линии совершают колебательное движение перпендикулярно направлению движения. Стоячая поперечная волна, – это волновое движение, в процессе которого вдоль волны не происходит перенос энергии.

Вся подготовительная работа закончена, можно заняться собственно вопросами моделирования. Что будет из себя представлять такая модель, графически более или менее понятно. Что же касается структуры данных, то точки на линии распространения волны – это, видимо, самый обычный одномерный числовой массив, можно даже целого типа, так как точность вычислений для качественной картинке роли не играет. Числовые значения элементов массива должны содержать в себе текущее значение координаты Y – точки, представленной определенным элементом массива.

Какая-то точка должна быть назначена источником, и, наверное, естественно в качестве источника принять первый элемент массива (с индексом, равным нулю или единице в зависимости от принятой нумерации массива). Также будет естественным назначить в качестве

препятствия последнюю точку массива. Задать колебательное движение – это определить: во-первых, параметры колебательного движения источника колебания (амплитуду, частоту, начальную фазу без всякого ущерба можно принять равной нулю). За точку отсчета времени также можно принять ноль.

И важнейший исходный параметр – скорость распространения колебания. Вопрос скорости необходимо рассмотреть отдельно. Уже было сказано, что колебательное движение (не стоячая волна) – это процесс переноса энергии, следовательно, скорость колебательного движения – это скорость переноса энергии. Но в нашей модели нет понятия энергии. Её представитель – амплитуда. Физика утверждает, что чем выше амплитуда, тем большую энергию несет волна. Это же подтверждает и наш обыденный опыт. Все, кто наблюдал хоть раз волны на воде, согласятся, что волны тем энергичнее, чем они выше. Отсюда следует, что скорость распространения волны можно представить скоростью переноса высоты точки колебания. Если исходная точка в некоторый момент времени t_1 достигла высоты h и соседняя с ней точка достигнет той же высоты в момент времени t_2 , то скорость распространения волны можно принять за:

$$\Delta t = t_2 - t_1,$$

где физически Δt – это время переноса энергии между двумя соседними точками волны. Пусть вас не смущает, что скорость выражена через время. Скорость есть расстояние, преодоленное за единицу времени, но для массива не определено понятие расстояния, оно появится только в графической интерпретации, а до этого еще достаточно далеко. Но такое представление поможет нам получить формулу волны для следующей от источника точки. Если колебание источника записать в виде:

$$Y = A \sin(wt),$$

то тогда в соседней точке колебание будет записано в виде:

$$Y_1 = A \sin(wt + w\Delta t).$$

Говоря языком физики, колебание в соседней точке отличается от колебания в источнике на величину фазы, равную выражению $w\Delta t$. Теперь мы имеем возможность записать формулу для колебания в любой точке волны. А именно между точкой – источником и точкой с номером n имеется ровно $(n - 1)$ отрезков, на преодоление которых необходимо $(n - 1) \Delta t$ времени. И для n -ой точки получаем формулу

$$Y_n = A \sin(\omega t + (n-1)\omega \Delta t).$$

Следующий важный момент заключается в том, что полученная формула верна только для уже установившегося колебания в точке, но с момента начала колебания в источнике до начала колебания в отдаленной точке должно пройти некоторое время. Колебательные движения не начинаются одновременно по всей длине волны. Поэтому в программе, прежде чем пересчитывать значения Y_k , необходимо проверять, дошла ли волна до точки с номером k . С учетом всего сказанного можно записать общий алгоритм распространения волны по массиву точек:

```

t = 0
Для k от 0 до N выполнять (здесь k – номер точки)
    Если t ≥ kΔt То (волна дошла до точки)
        Вычислить  $Y_k$ 
        Прорисовать точку в графической модели в новой позиции
        t = t + d

```

Здесь d – некая малая величина, существенно меньшая, чем Δt . Это программистская модель без сложения с отраженной волной. Впрочем, доработку модели оставим читателю, снабдив его некоторыми важными замечаниями.

Во-первых, конечный элемент массива, соответствующий точке отражения в момент времени, когда до него дойдет волна, можно рассматривать как второй источник с известными параметрами колебания, но распространяющимися в противоположную сторону.

Во-вторых, каждую точку среды, участвующую в волновом движении, можно считать участвующей в двух волновых движениях, следовательно, для каждой точки можно рассчитывать две величины Y с суммарной вычисляемой по формуле:

$$Y_{\text{суммарная}} = Y_{\text{прямой волны}} + Y_{\text{отраженной волны}}$$

В-третьих, необходимо учесть физический закон, утверждающий, что отраженная волна отстает от прямой на половину длины волны. Заметим сразу, что в нашей модели нет понятия длины волны, мы смогли без него обойтись. Но есть понятие разности фаз, через которое длина волны и выражается. Это означает, что к фазе отраженной волны необходимо прибавить (в её источнике) некоторую константу, являющуюся половиной длины волны. Зная скорость распространения волны и расстояние между точками среды, вычислить длину волны, равную расстоянию между точками, находящимися в одной фазе,

возможно, и эта задача также остается вам, но, в принципе, можно попробовать прибавлять к фазе отраженной волны любые константы, достаточно интересно посмотреть, на что это влияет.

Проблема для самостоятельного исследования

Написав программу, проанализируйте, как ведет себя суммарная волна при различных разностях в длинах прямой и отраженной волн.

Задача для самостоятельного решения

Все рассуждения выше проведены для поперечной волны. Однако можно ожидать, что задача с продольной волной должна решаться принципиально так же. Напишите программу по условиям задачи, но для продольной волны.

В заключение

Главная проблема построения такого рода моделей, как вы, наверное, заметили, – это переход от непрерывного движения к дискретному, для чего от законов движения на отрезке пространства и времени надо перейти к законам поведения в пространственной и временной точках. Конечно, такое представление всегда ведет к потере точности, но, уменьшая пространственные и временные интервалы, всегда или почти всегда можно увеличить точность модели.

Выше было уже сказано, но заметим еще раз, что наша модель, по сути, не модель физического явления, а скорее модель математического процесса, если это не учитывать, то можно столкнуться с непонятными вещами. Например, попробуйте в нашей первой модели создать ситуацию, в которой тела сталкиваются друг с другом. Физически в момент столкновения начинаются совсем иные процессы, не связанные с движением. Математически в момент столкновения силы тяготения, обратно пропорциональные расстоянию, возрастают до бесконечности и, соответственно, до бесконечности возрастает и ускорение, но в программисткой модели бесконечных величин не бывает. Попробуйте смоделировать такую ситуацию и посмотрите, что получится.

Задача о колебательном движении пружины

Условие задачи. Построить модель колебательного движения тела, подвешенного на пружине.

В требуемом движении участвуют три силы: сила тяготения, сила упругости и сила трения. Совместная работа трех сил создает довольно сложное колебательное движение, поэтому с учетом того, что требуется только качественная картина, можно подумать о том, как упростить условие задачи. Прежде всего заметим, что сила трения наиболее сложно поддается учету, так как необходимо учесть трение о воздух, какие-то процессы внутри пружины, и, кроме того, сила трения своим результатом будет иметь не движение, а его затухание, поэтому, наверное, от неё можно отказаться в первую очередь. Затем не будет большим отступлением от условия задачи, если мы откажемся учитывать воздействие силы тяжести на пружину, учтя её действие только на тело, подвешенное к пружине. Как-то упрощать действие силы упругости, наверное, нет необходимости. Физика утверждает, что сила упругости прямо пропорциональна величине деформации пружины и выражается формулой:

$$F = -kx.$$

Что-либо более простое придумать уже нельзя, поэтому здесь наш анализ прекращается. Заметим только, что величина k – коэффициент упругости определяется физическими свойствами пружины и, стало быть, также выходит за границы исследования, минус после знака равенства говорит о том, что сила направлена против направления деформации и стремится к восстановлению формы пружины. Математически для построения модели осталось добавить формулу для силы тяжести:

$$F = mg.$$

Но мы исходим из необходимости писать программу, а это дело требует детального понимания процесса, поэтому обсудим некоторые физические и технические детали. Начнем с физического вопроса: за счет чего образуется колебательное движение? Ведь ясно, что сила тяжести, действующая на тело, будет растягивать пружину, увеличивая силу упругости, и настанет момент, когда упругая сила, увеличиваясь, сравняется с силой тяжести. Почему бы телу в этот момент (в этой точке суммарная сила равна нулю) не остановиться?

Дело в том, что в момент достижения равенства сил суммарное ускорение будет действительно равно нулю, но не скорость, которую тело набирает, падая и растягивая пружину. А так как в точке равновесия ускорение нулевое, то погасить набранную скорость нечем, и тело проскакивает точку равновесия. За ней начинается обратный

процесс, сила упругости перебарывает силу тяжести, появляется ускорение, направленное против движения, постепенно оно гасит скорость и начинает разгонять тело в обратном направлении, до тех пор, пока сила тяготения опять не возьмет верх.

Что делать далее, принципиально понятно, то же самое, что мы делали и в двух предыдущих задачах, – необходим переход от непрерывного движения к дискретному. Для этого выполним разбиение вертикальной оси координат, вдоль которой движется тело, на равные отрезки, помня, что чем отрезки меньше, тем точнее будет модель движения. Будем полагать, что на отрезке тело движется с постоянным ускорением (суммарная сила не меняется), а пересчет суммарного ускорения происходит на границе очередного отрезка. На этом анализ завершим, дальнейшие рассуждения принципиально ничем не отличаются от работы, проделанной в задаче о движении в гравитационном поле.

Проблема для самостоятельного исследования

Если вы были внимательны, то должны были заметить очень существенную разницу между моделью движения в первой и третьей задачах, хотя именно процесс движения, после того как учтены все силы, наверное, может описываться в рамках какой-то общей схемы. Но есть отличие. В задаче о гравитационном поле в точке пересчитываются скорости, а на отрезке тела движутся равномерно и прямолинейно. В задаче о пружине в точках пересчитываются ускорения, а на отрезках тело движется прямолинейно и равноускоренно. Подумайте, насколько это принципиальная разница и не совершили ли мы ошибки в одной из задач.



ГЛАВА 6.

Несколько интересных задач

Довольно многие вычислительные процессы сводятся к рекуррентным вычислениям, каковые можно определить как построение ряда чисел, в котором каждое определяется через одно или несколько предыдущих. Простейшая рекуррентная последовательность – это арифметическая прогрессия – ряд, в котором каждый элемент определяется через предыдущий выражением:

$$A_k = A_{k-1} + d,$$

где величина d называется разностью арифметической прогрессии. Если известны разность прогрессии и её первый элемент, то можно получить любое количество членов ряда, выполняя простую операцию суммирования. Однако в таком подходе есть существенный минус. Для того чтобы вычислить сотый элемент, необходимо вычислить 99 предыдущих. Возникает естественное желание научиться вычислять любой элемент прогрессии, не считая промежуточных. Для арифметической и геометрической прогрессий, как известно, такая задача решается достаточно легко. Однако существуют более сложные виды рекуррентно определяемых последовательностей, для которых такой формулы, очевидно, нет. Естественный вопрос, который следует задать сейчас, – это такой: а существует ли достаточно широкий класс рекуррентных определений, в которых все же можно избавиться от большого объема вычислений? Оказывается, ответ на этот вопрос положителен. Есть очень большой класс так называемых возвратных последовательностей, для которых можно построить формулу, так же как это сделано для арифметической и геометрической прогрессий.

Понятно, что если есть класс задач, имеющих хорошее решение, то в любом случае множество задач, такового решения не имеющих, значительно шире. А что делать с ними? В случае с рекуррентными

определениями такими «нерешаемыми» задачами будут задачи с ветвящимися процессами.

Задача Дейкстры

Рассмотрим в качестве примера задачу Дейкстры, уже упоминавшуюся ранее, и разберем проблемы её счета детально. Функция из условия задачи определяется следующими формулами:

$$\begin{cases} f(1) = 1 \\ f(2N) = f(N) \\ f(2N + 1) = f(N) + f(N + 1) \end{cases}$$

Чтобы научиться её считать, разберем еще раз эти выражения. Первое утверждает, что реально значение функции можно подсчитать только от единичного аргумента. Второе говорит о том, что если аргумент четный, то функция сводится к функции от вдвое меньшего аргумента. Третье утверждает, что функция от нечетного аргумента сводится к сумме двух функций, таких что их аргументы в сумме дают аргумент исходной функции, и отличаются друг от друга на единицу.

Рекурсивное решение задачи очевидно, но наша цель – уйти от рекурсии. Отметим для начала, что проблем здесь существенно больше, нежели в задачах на простые возвратные последовательности. Для той же арифметической прогрессии необходимо помнить только один предыдущий элемент, для ряда чисел Фибоначчи – два. Здесь же требуется знание всего ряда. Например, посчитаем $f(5)$.

$$f(5) = f(3) + f(2) = f(2) + f(1) + f(1) = f(1) + f(1) + f(1) = 3f(1) = 3.$$

Для того чтобы вычислить функцию от 5, необходимо построить все дерево промежуточных результатов, дойти в каждой ветви до вершины и посчитать количество полученных единиц. С ростом аргумента очень быстро растет потребность в памяти для хранения промежуточных результатов, и мы получаем две проблемы: наличие рекуррентного процесса и необходимость хранить в памяти все промежуточные результаты. Избавиться от рекуррентных формул, видимо, не получится, единственное – есть надежда, что можно придумать метод счета, не нуждающийся в запоминании всех промежуточных результатов, хотя и эта задача выглядит достаточно непростой в силу разветвления вычислительного процесса при нечетных аргументах.

Итак, главная загвоздка – в необходимости хранить массу промежуточных результатов. Нужна какая-то красивая математическая закономерность, позволяющая на протяжении всего процесса пользоваться ограниченным набором данных. Найти такую закономерность можно математическим анализом структуры функции и процесса её вычисления, при условии что мы хорошие математики и обладаем развитым аналитическим аппаратом. Но давайте будем исходить из худшего предположения – математики мы слабые, но закономерность найти все же хочется. Тогда остается только возможность эксперимента. Попробуем набросать некоторое количество хороших примеров и посмотрим на них, по возможности, внимательно.

Но построить хороший пример не так просто. С одной стороны, он должен продемонстрировать как можно больше особенностей рассматриваемого процесса, а следовательно, должен быть достаточно сложен. С другой стороны, если пример сложен, то возникает вероятность ошибки счета (экспериментатором), и чем пример сложнее, тем эта вероятность выше. Если же пример прост, то вероятность ошибки невелика, но такой пример может быть неинформативен. Сформулируем правило построения хорошего примера.

Для того чтобы получить хороший пример, посмотрим, какие особенности исследуемого вычислительного процесса необходимо продемонстрировать. Сложность и трудоемкость примера должны быть таковы, чтобы исследуемые особенности были представлены полно, но не избыточно.

Покажем на исследуемой задаче, как применить полученное правило. Функция определяется тремя формулами, каждая из которых вносит какие-то нюансы в вычислительный процесс. Следовательно, нужен пример, в котором каждое правило применялось бы больше одного раза. Конечно, чем больше, тем лучше, но с ростом аргумента растет объем необходимых вычислений и, следовательно, растет вероятность человеческой ошибки.

Мы будем искать закономерность на одном примере. Интуитивно ясно, что на большом множестве примеров закономерность должна проявиться ярче. Но тем не менее стратегия одного примера вполне возможна из психологических соображений, так как чем примеров больше, тем выше вероятность не увидеть леса за деревьями.

Математические закономерности не случайны, на то они и закономерности. Поэтому если закономерность есть, она будет видна уже на одном примере, почему мы и надеемся, что его будет вполне достаточно.

Перейдем к построению. Выше уже приведен пример счета, но сейчас посчитаем функцию несколько иначе. Заметим, что на каждой итерации функция может быть представлена либо одной функцией, либо суммой двух. Это наводит на мысль представить процесс расчетов как дерево уменьшающихся аргументов. Проведем расчеты с аргументом 113.

[illegible]

Для того чтобы получить хорошую информацию из результатов численного эксперимента, данные необходимо так расположить на бумаге, чтобы взаимосвязи между ними были как можно более наглядны.

Нарушение данного правила может свести на нет вычислительную работу. Например, вычислим значение функции от того же аргумента, пользуясь только определением:

$$\begin{aligned}
 F(113) &= f(56) + f(57) = f(28) + f(28) + f(29) = 2f(28) + f(29) = \\
 &= 2f(14) + f(14) + f(15) = 3f(14) + f(15) = 3f(7) + f(7) + f(8) = \\
 &= 4f(7) + f(8) = 4f(3) + 4f(4) + f(4) = 4f(3) + 5f(4) = \\
 &= 4f(1) + 4f(2) + 5f(2) = 4f(1) + 9f(2) = 4f(1) + 9f(1) = 13f(1) = 13.
 \end{aligned}$$

Согласитесь, что эта запись очень сильно проигрывает в сравнении с табличной. А теперь вернемся к таблице и применим полученное правило еще раз. Распишем количества четных и нечетных на каждом шаге, так как уже решено, что закономерность следует искать именно здесь.

Таблица 6.2. Количества четных и нечетных

Итерация	Кол-во четных	Кол-во нечетных
1	1	1
2	2	1
3	3	1
4	1	4
5	5	4
6	9	4
7	0	13

Последний шаг как-то выпадает, здесь появляется ноль, но это последний шаг, он может не укладываться в схему. Последняя и первая итерации очень часто представляют собой что-то особенное, не вписывающиеся в общую схему. А вот промежуточные результаты очень любопытны. Обратите внимание, что на каждой итерации, начиная со второй, одно из количеств равно сумме двух верхних, а второе количество равно одному из верхних. Одна из величин равна сумме двух величин предыдущего шага, но иногда она равна сумме четных, а иногда сумме нечетных.

Чтобы разобраться в том, как именно образуются новые количества четных и нечетных, посмотрим, что происходит на очередном шаге. Пусть после некоторой итерации есть два числа, назовем их **ЧЕТНОЕ** и **НЕЧЕТНОЕ**. **НЕЧЕТНОЕ** число всегда (на следующей итерации) порождает два аргумента: один четный и второй нечетный, — и так всегда независимо от значения **НЕЧЕТНОГО**. Следовательно, **НЕЧЕТНОЕ** качественно на ситуацию не влияет. **ЧЕТНОЕ** ведет себя несколько иначе. Уменьшаясь в два раза, оно может стать как четным, так и нечетным. Следовательно, именно оно изменяет ситуацию существенным образом. Рассмотрим эти два варианта развития событий.

Вариант 1. ЧЕТНОЕ преобразуется в ЧЕТНОЕ. В этом случае количество четных новой итерации есть сумма количества четных и количества нечетных, нечетных остается столько, сколько и было. Сказанное можно записать следующими формулами:

1. Кол-во НЕЧЕТНЫХ = Кол-ву НЕЧЕТНЫХ.
2. Кол-во ЧЕТНЫХ = Кол-во ЧЕТНЫХ + Кол-во НЕЧЕТНЫХ.

(Слева от равенства количества очередной итерации, справа – предыдущей).

Вариант 2. ЧЕТНОЕ преобразуется в НЕЧЕТНОЕ. В этом случае количество нечетных новой итерации равно сумме количеств четных и нечетных, четных же остается столько, сколько и было, что можно записать так:

1. Кол-во НЕЧЕТНЫХ = Кол-во ЧЕТНЫХ + Кол-во НЕЧЕТНЫХ.
2. Кол-во ЧЕТНЫХ = Кол-ву НЕЧЕТНЫХ.

Таким образом, мы получили необходимые закономерности. Они не избавляют от необходимости просчитывать все промежуточные результаты, но потребности в их хранении больше нет. Все расчеты на каждой итерации используют только четыре числа: ЧЕТНОЕ, НЕЧЕТНОЕ и их количества. Эти четыре числа имеют некоторые исходные значения и пересчитываются на каждой итерации, все завершается тогда, когда остаются только нечетные, а это возможно только тогда, когда остались одни единицы. Если такая ситуация достигнута, то количество нечетных и есть требуемый результат.

Прежде чем приступить к построению реализации, обсудим два особых случая: последнюю итерацию и начало процесса. Выше уже сказано, что возможны такого рода особые случаи. В отношении нашей задачи их особенность заключается в том, что в начале процесса есть только одно число ИСХОДНОЕ, оно либо ЧЕТНОЕ, либо НЕЧЕТНОЕ. Следовательно, и количеств не два, а одно. Причем если ИСХОДНОЕ число в своем разложении содержит степень двойки, то нужна целая серия итераций, чтобы появилось НЕЧЕТНОЕ. Особенность последней итерации в том, что ЧЕТНЫХ больше нет.

Исключительность последней итерации нам в общем-то на руку, так как отсутствие ЧЕТНЫХ есть сигнал к завершению процесса. А вот об особенностях исходного состояния необходимо позаботиться специальным образом. А именно убрать все степени двойки, деля исходное число на два до тех пор, пока это возможно.

Теперь все существенные моменты обсуждены и можно приступать к построению реализации. Первым действием необходимо ввести исходное число. Основной вычислительный процесс будем запускать от нечетного числа, так как именно оно на первой же итерации даст и четное, и нечетное. Поэтому специального обозначения для нечетного вводить не будем, используем для ввода переменную НЕЧЕТНОЕ.

```
In.Open;  
In.Int (Нечетное) ;
```

Однако число, взятое из потока, может оказаться четным и содержать в своем разложении степень двойки. Учтем это обстоятельство следующим циклом:

```
WHILE Нечетное MOD 2=0 DO Нечетное:=Нечетное DIV 2 END;
```

Инициализируем начало процесса, а сам процесс оформим циклом по условию:

```
СуммаЧетных:=1;  
СуммаНечетных:=1;  
REPEAT  
    /Все необходимые расчеты./  
UNTIL Нечетное=1;
```

Первым действием очередной итерации может быть вычисление очередного четного и очередного нечетного. Мы уже решили, что главным числом для нас будет нечетное. Поэтому расчет очередной пары выполним так:

```
Четное:=Нечетное DIV 2;  
Нечетное:=Нечетное - Четное;
```

Но, в общем-то, не очевидно, что при делении нечетного на 2 в результате может получиться именно четное. Собственно, может быть всякое, поэтому скорректируем действие:

```
Четное:=Нечетное DIV 2;  
Нечетное:=Нечетное - Четное;  
IF Четное MOD 2 = 1 THEN  
    Число:=Четное;  
    Четное:=Нечетное;  
    Нечетное:=Число;  
END;
```

Далее необходимо просчитать новые значения сумм. Из проведенного анализа уже знаем, что выбор расчетных формул зависит от того, какое число, четное или нечетное, получится при делении на двойку четного. Поэтому требуется проверка следующего условия:

```
IF ((Четное DIV 2) MOD 2)=0 THEN  
ELSE  
END;
```

Если четное при делении на два дает четное, то есть условие истинно, то выполняется следующий расчет:

```
СуммаЧетных:=СуммаЧетных+СуммаНечетных;
```

Сумма нечетных не пересчитывается, так как она и не меняется. Если же результат нечетен, то есть условие ложно, то ситуация несколько усложняется, так как пересчитываются обе суммы.

```
СуммаЧетных:=СуммаНечетных;  
СуммаНечетных:=СуммаНечетных+СуммаЧетных;
```

И наконец, полный листинг.

```
MODULE Модуль;  
    IMPORT StdLog, In;  
    PROCEDURE Дейкстра*;  
    VAR  
        Число, Четное, Нечетное, СуммаЧетных, СуммаНечетных: INTEGER;  
    BEGIN  
        StdLog.Clear;  
        In.Open;  
        In.Int (Нечетное);  
        WHILE Нечетное MOD 2=0 DO Нечетное:=Нечетное DIV 2 END;  
        СуммаЧетных:=1;  
        СуммаНечетных:=1;  
        REPEAT  
            Четное:=Нечетное DIV 2;  
            Нечетное:=Нечетное - Четное;  
            IF Четное MOD 2 = 1 THEN  
                Число:=Четное;  
                Четное:=Нечетное;  
                Нечетное:=Число;  
            END;  
            IF ((Четное DIV 2) MOD 2)=0 THEN  
                СуммаЧетных:=СуммаЧетных+СуммаНечетных;  
            ELSE  
                СуммаЧетных:=СуммаНечетных;  
                СуммаНечетных:=СуммаНечетных+СуммаЧетных;  
            END;  
            UNTIL Нечетное=1;  
            StdLog.Int (СуммаНечетных);  
        END Дейкстра;  
    END Модуль.
```

Проблема для самостоятельного исследования

Функция Дейкстры обладает одним очень ценным качеством: все ветви дерева, представляющего разложение аргумента, имеют равную длину. Попробуйте придумать пример разветвляющейся рекуррентной функции, которая бы не обладала этим качеством.

Проблема для самостоятельного исследования

Подумайте, так ли нужно учитывать исходную ситуацию с числом, содержащим в разложении на множители степень двойки. Иначе говоря, нельзя ли все же обойтись без оператора:

```
WHILE Нечетное MOD 2=0 DO Нечетное:=Нечетное DIV 2 END;
```

Перейдем к следующей задаче. В предыдущих главах мы уже рассматривали задачи, имеющие переборную природу. Их еще называют комбинаторными. И нам уже известна проблема комбинаторного взрыва. И уже знаем, что есть способы, которыми эту проблему хоть и нельзя полностью решить, но все можно существенно ослабить. В той же главе был прозрачный намек, что вполне возможна ситуация, когда задача, имея комбинаторную природу с первого взгляда, на самом деле таковой не является. Все, что для этого надо, — это хорошая закономерность в исследуемой структуре данных. Начнем анализ с простого примера.

Задача о поиске пути с наибольшим весом

Условие задачи. Дана таблица положительных целых чисел, сформированная случайным образом. Необходимо найти путь из левого верхнего угла в правый нижний с наибольшим весом. Весом пути будем называть сумму чисел всех элементов таблицы, через которые проходит путь. Путь можно строить только двумя типами смещений: вправо или вниз на один шаг.

На первый взгляд, задача переборная. Неопытного исследователя может смутить тот факт, что таблица строится случайным образом. Это так, но более интересно, что случайные числа упакованы в прямоугольную таблицу. А строгая форма может перевесить случайность содержания. Кроме того, свобода передвижения по таблице ограничена, что тоже немаловажно. Но все же если сейчас взять произвольную прямоугольную таблицу или даже квадратную, то вряд ли в ней удастся что-либо увидеть. Но иногда красивая закономерность, не прояв-

ляющаяся в исходных данных, видна на какой-то дополнительной структуре, которую можно построить из исходной.

Наша задача – уйти от полного перебора. Если это получится, то процесс построения пути должен состоять из $N-1$ шагов (если таблица имеет размерность $N \times N$), на каждом из которых обходчик, находясь в некоторой ячейке, принимает единственное решение о следующем шаге из двух возможностей: вправо или вниз. Для того чтобы такая возможность появилась, необходимо к каждому элементу таблицы привязать числовую величину, характеризующую самый тяжелый путь до этого элемента. Если построить таблицу таких характеристик, то исходная задача решится достаточно просто.

Мы ответили на вопрос, при каком условии исходная задача решается легко. Теперь остается надеяться, что реализация придуманного искусственного условия вполне возможна. Но в любом случае перебор таких «искусственных условий» полезен для понимания природы задачи.

Итак, нам известно, что в ячейке новой искусственной таблицы стоит вес наиболее тяжелого пути. Видимо, это будет сумма всех весов ячеек вдоль пути. Но от левой верхней ячейки до любой другой можно выстроить целое дерево путей. А это опять перебор, и мы зашли в тупик. Заметим, что тупиковая ситуация произошла от гипотезы, что ячейка новой таблицы должна характеризовать путь до ячейки.

Сейчас читайте очень внимательно, мы в этой врезке сформулируем очень сильную и непростую идею. Если речь идет о порядке обхода некоторого графа, то далеко не все равно, в каком порядке этот обход выполнять. Изменение порядка может дать неожиданный эффект. Допустим, что на рассматриваемом множестве задан некоторый порядок. Порядок позволяет организовать перебор множества разными способами. Если некоторый способ перебора не позволяет достигнуть желаемого результата, то есть смысл подумать о другом порядке элементов множества.

Мы предположили, что ячейка характеризует путь до неё, и попали в тупиковую ситуацию. Попробуем использовать идею, сформулированную во врезке выше. Тупиковый вариант получился при попытке пройти таблицу от начала, попробуем пройти её от конца, также собирая информацию о весе пути. В этом случае вспомогательная величина, навешиваемая на элемент таблицы, будет характеризовать путь не до элемента, а путь после элемента.

Рассмотрим пример. Из правой нижней точки возможны два пути в точку 'с' и в точку 'b'. Пересчитаем веса этих ячеек следующим образом:

Таблица 6.3.

	d	b
	c	a

1. Новый вес точки 'b' = Её старый вес + Вес ячейки 'a'.
2. Новый вес точки 'c' = Её старый вес + Вес ячейки 'a'.

После такого пересчета веса ячеек 'b' и 'c' характеризуют вес пути, который может быть пройден от этих ячеек до ячейки 'a'. Если теперь проходчик дойдет до ячейки 'd', он будет знать, куда ему идти. Вопрос: а зачем идти в точку 'd'? Что это может дать? Ответ: это создает возможность движения в точки 'b' и 'c'. Следовательно, характеристикой точки 'd' должна быть следующая величина:

Новый вес точки 'd' = Её старый вес + вес ячейки 'b' + вес ячейки 'c'.

Таким образом, сейчас каждая ячейка содержит сумму наиболее легкого продолжения и наиболее тяжелого. Третьего не дано. А стало быть, мы добились своего. Новая таблица строится без перебора и действительно дает возможность ответить на вопрос, куда двигаться из текущей ячейки, не перебирая всех возможных вариантов, а просто сравнивая два значения: то, что справа, и то, что снизу от текущей ячейки.

Таблица 6.4. Исходная и характеристическая таблицы

Таблица исходная

1	2	8	3	5
3	2	2	1	45
5	1	1	0	4
2	3	4	3	8
1	1	3	4	8

Таблица характеристическая

1480	816	419	190	70
663	395	221	117	65
265	172	102	51	20
88	69	50	31	16
17	16	15	12	8

Построение программы

Будем для упрощения работать только с квадратными таблицами.

VAR

Таблица: ARRAY 10, 10 OF INTEGER;

Перед началом работы основного алгоритма заберем из входного потока реальный размер таблицы и её значения:

```

In.Open;
In.Int(n);
FOR x:=0 TO n-1 DO
    FOR y:=0 TO n-1 DO
        In.Int(Таблица[x,y]);
    END;
END;

```

Преобразуем исходную таблицу заменой значения в ячейках на сумму старого значения ячейки и значений соседей слева и снизу:

```

FOR x:=n-1 TO 0 BY -1 DO
    FOR y:=n-1 TO 0 BY -1 DO
        IF x<n-1 THEN Таблица[x,y]:= Таблица[x,y]+ Таблица[x+1,y];
        END;
        IF y<n-1 THEN Таблица[x,y]:=Таблица[x,y]+ Таблица[x,y+1];
        END;
    END;
END;

```

Условные операторы необходимы, чтобы исключить пересчет правого нижнего элемента, так как для него пересчет невозможен. После описанного преобразования остается построить путь. Начинается он, естественно, с левой верхней ячейки и заканчивается в правой нижней. Сказанное запишется циклом:

```

x:=0;y:=0;
StdLog.Int(x)StdLog.Int(y);StdLog.String(*Распечатка координат
начала пути*)
WHILE (x#n-1) OR (y#n-1) DO
    StdLog.Int(x);StdLog.Int(y);StdLog.Ln; (*Распечатка
    координат текущего положения*)
END;

```

Для построения тела цикла заметим, что если проходчик таблицы достиг стенки, правой или нижней, то далее он однозначно будет двигаться вдоль стены. То есть

```

IF x=n-1 THEN
    y:=y+1;
ELSIF y=n-1 THEN
    x:=x+1;
ELSE
    /Более хитрая ситуация./
END;

```

Более хитрая ситуация – это положение где-то внутри таблицы. Тогда выбор осуществляется из двух возможных: вправо или вниз, в зависимости от того, где в таблице находится большее значение. Сказанное можно описать следующим оператором выбора:


```

IF Таблица[x+1,y]> Таблица[x,y+1] THEN
    x:=x+1;
ELSE
    y:=y+1;
END;

```

И наконец, полный листинг.

```

MODULE Модуль;
    IMPORT In, StdLog;
    PROCEDURE Процедура*;
    VAR
        Таблица: ARRAY 10, 10 OF INTEGER;
        x,y,n:INTEGER;
    BEGIN
        StdLog.Clear;
        In.Open;
        In.Int(n);
        FOR x:=0 TO n-1 DO
            FOR y:=0 TO n-1 DO
                In.Int(Таблица[x,y]);
            END;
        END;
        FOR x:=n-1 TO 0 BY -1 DO
            FOR y:=n-1 TO 0 BY -1 DO
                IF x<n-1 THEN Таблица[x,y]:= Таблица[x,y]+
                    Таблица[x+1,y]; END;
                IF y<n-1 THEN Таблица[x,y]:= Таблица[x,y]+
                    Таблица[x,y+1]; END;
            END;
        END;
        x:=0;y:=0;
        StdLog.Int(x);StdLog.Int(y);StdLog.Ln;
        WHILE (x#n-1) OR (y#n-1) DO
            IF x=n-1 THEN
                y:=y+1;
            ELSIF y=n-1 THEN
                x:=x+1;
            ELSE
                IF Таблица[x+1,y]> Таблица[x,y+1] THEN
                    x:=x+1;
                ELSE
                    y:=y+1;
                END;
            END;
            StdLog.Int(x);StdLog.Int(y);StdLog.Ln;
        END;
    END Процедура;
END Модуль.

```

Проблема для самостоятельного исследования

Построение пути по графу часто наталкивается на проблему цикла. Если граф содержит цикл, то путь, являясь конечным по количеству включенных в него вершин, тем не менее становится непроходимым. Докажите, что путь, построенный программой, не может содержать цикла.

В заключение

Решенную задачу, наверное, нельзя назвать сложной. Но она очень сильный пример того, что случайно сформированные данные тем не менее не обречают решателя на комбинаторный алгоритм.

Задача о минимальном количестве заправок

Условие задачи. Автомобиль движется из пункта A в пункт B , начав движение с полным баком. Расход бензина на один километр пути известен. Между пунктами A и B находится некоторое количество заправок с неограниченным запасом бензина на каждой. Определить, на каких колонках необходимо заправляться, чтобы количество заправок было минимальным.

Очень важно в любой задаче точно понять, что требуется. В задаче о заправках есть угроза изменить условие и начать искать возможность заправиться минимальным количеством бензина. Угроза чисто психологического характера. Решатель может подумать, что речь идет о возможной экономии, следовательно, надо постараться сделать так, чтобы суммарно на заправках брать как можно меньше бензина, или, что то же самое, постараться, чтобы к концу пути бак оказался по возможности пустым. Угрозы такого рода весьма реальны, мой личный преподавательский опыт говорит о том, что даже искушенный решатель зачастую исходит не из условия, а из какого-то своего первого представления. Конкретно в этой задаче мне приходилось несколько раз встречаться с подменой условия. Такого рода подмена выглядит довольно странно, так как при фиксированном расходе бензина для каждого типа автомобиля ни о какой экономии в принципе речи быть не может, но тем не менее так бывает.

Итак, все же от нас требуется построить свое поведение на дороге таким образом, чтобы именно количество заправок было минимальным. Задача в подобной постановке вполне допускает переборное ре-

шение. Достаточно построить все возможные сочетания из заправок, для каждого сочетания выяснить, позволит ли такая расстановка заправок автомобилю доехать до пункта *В*, и затем из всех допустимых сочетаний выбрать содержащее наименьшее количество заправочных станций. Такое решение выглядит вполне естественным.

Подумаем, почему в этом случае естественно не означает хорошо. Попробуем искать не решение, а некоторое общее соображение. Это общее соображение лежит на поверхности. Для каждой заправки верно утверждение, что путь от неё автомобиль начинает с полным баком. Следовательно, для каждой заправки можно построить строго фиксированный путь, который машина пройдет гарантированно. В сказанном нет ничего вариативного. Следовательно, это можно принять за отправную точку в поиске некомбинаторного решения.

Проведем эксперимент. Возьмем некоторую произвольную расстановку заправок, но, конечно, такую, в которой расстояния между станциями самые различные, для общности эксперимента. Построим от каждой заправки отрезок, равный расстоянию, которое автомобиль может пройти с полным баком. Во взаимном положении отрезков возможны два варианта.

Первый вариант. Есть хотя бы одна пара смежных отрезков, не пересекающихся ни одной точкой. В этом случае решения нет.



Рис. 6.1. Задача, не имеющая решения

Второй вариант. Есть пары пересекающихся отрезков. Это более сложная ситуация. Первое, что приходит в голову, – это решение и есть, хотя оно включает в себя все заправки.

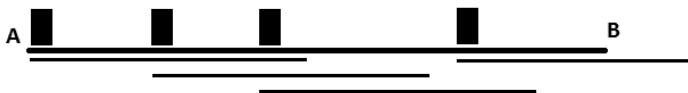


Рис. 6.2. Задача, имеющая тривиальное решение

Третий вариант появляется из более внимательного рассмотрения второго. Трудно предположить, что есть только два варианта: ни одного решения и одно тривиальное. Разумно ожидать, что какие-то из заправок можно выбросить из рассмотрения. Но если отрезки только

пересекаются, то удаление одного из них делает весь путь невозможным. Следовательно, необходимо придумать ситуацию, в которой удаление отрезка не разорвет путь. Идея опять-таки лежит на поверхности. Это возможно в том случае, если некоторый отрезок полностью перекрывается своими соседями.



Рис. 6.3. Ситуация, позволяющая выбросить одну станцию

Еще раз идея решения. Построим от каждой заправки отрезок, равный длине пути, проходимого с полным баком. Рассмотрим полученную систему отрезков и выбросим заправки, порождающие отрезки, полностью покрываемые своими соседями.

Как получить решение, ясно. Но это еще, к сожалению, не все. Мы нашли способ выявления лишних заправок, но, может быть, этот способ не дает однозначного результата?!

Если в задаче требуется найти некоторое минимальное или максимальное значение, то надо быть уверенным, что построенный метод дает одно решение. В противном случае в задаче опять появляется необходимость перебора.

Вообще говоря, в любой задаче на поиск минимума или максимума необходимо доказательство, что найденное решение будет искомым. В нашем случае возможно следующее рассуждение. Допустим, процесс построения решения привел к удалению N заправок. Если доказать, что метод дает только одно решение, то оно и есть минимальное, если метод неоднозначен, то просто делаем небольшой перебор и находим минимум. Но здесь кроется логическая ошибка. На самом деле мы такими умозаключениями показываем, что улучшить решение невозможно нашим методом, но это не означает, что нет другого более эффективного рассуждения. То есть пока не построено строгое доказательство, полученное решение можно считать только эвристическим.

Но, в общем-то, описанная процедура действительно позволяет отбрасывать заправки, следовательно, в любом случае полученное решение лучше тривиального, заключающегося во включении всех заправок. Поэтому, отметив необходимость доказательства, все же не будем тратить на него времени.

Проблема для самостоятельного исследования

Проведите анализ и ответьте на вопрос: действительно ли этот алгоритм дает наилучшее решение?

Сущность алгоритма описана, но это лишь промежуточный этап между интуитивным пониманием решения и абсолютно строгой записью. Поэтому продолжим анализ.

Разрыв между представлением о решении и конечной программой очень велик. Перескочить через эту пропасть одним прыжком почти невозможно, поэтому, прежде чем переходить к написанию программы, надо полученное решение максимально уточнить в обычных, привычных программисту терминах.

Ключевой пункт дальнейшего анализа – следующее утверждение: «если отрезок полностью перекрывается своими соседями, то он лишний». Здесь необходимо уточнить термин «отрезок» и более строго определить процедуру определения перекрытия.

Замечание об отрезке. Не нарушая общности задачи, можно положить, что весь путь равен некоторому целому количеству единиц, например километров, и заправки расположены на стыке километров. Пусть также путь начинается с нулевой отметки. Тогда отрезок можно задать двумя числами – номером первого и номером последнего километра. На логику построения алгоритма эти договоренности не окажут никакого влияния.

О критерии перекрытия. Если некоторый отрезок перекрывается своими соседями, то перекрывается любой его километр. Введем для каждого километра индекс перекрытия – число, равное количеству отрезков, перекрывающих данный километр. Тогда все очень просто. Если для рассматриваемого отрезка все его километровые индексы больше единицы, то отрезок в соответствии с нашим алгоритмом можно объявить лишним.

Два предыдущих абзаца существенно приблизили нас к точному алгоритму. Сейчас уже можно заняться построением программы. Но если разработчик – человек неопытный, то можно еще набросать схему алгоритма и дать описание используемых структур данных. Сделаем это.

Структуры данных:

1. Очевидно, нужен массив заправок. Элементом такого массива можно определить номер километра, на котором стоит заправка.
2. Разумно определить массив отрезков первым и последним километром. Но первый километр отрезка, – это номер кило-

метра, на котором стоит заправка, а последний километр определяется суммой первого и длины отрезка. Поэтому местоположение отрезка возможно вычислять.

3. Индекс километра. Это тоже массив, индекс которого есть номер километра, а значение – количество отрезков, покрывающих данный километр. Принципиально индекс пересечения также допустимо сделать вычисляемой величиной, но постоянный его пересчет заметно утяжелит программу. Поэтому все же остановимся на специальном массиве.

Схема алгоритма:

1. Ввод массива заправочных станций.
2. Расчет индексов для каждого километра.
3. Выявление лишних заправок.

Второй и третий пункты не тривиальны, есть смысл их доработать. Расчет индексов можно выполнить двумя способами. Во-первых, пройдя все километры, определить для каждого из них количество покрывающих отрезков. Во-вторых, можно пойти по отрезкам. Вторая стратегия кажется предпочтительной. Для решения проблемы лишних станций будем танцевать от заправок, анализируя для каждой все её километры. Если у текущей станции все километры имеют индекс больше единицы, то заправка отбрасывается.

Построение программы

Входные данные для программы – это три числовых параметра: полный путь, который должен пройти автомобиль, пробег автомобиля с полным баком и массив заправочных станций. Положение заправок определим номерами километров. Еще одна важная структура данных – это массив километров, значения которого – это индекс покрытия – количество заправок, дающих возможность проехать этот километр. После ввода всех данных необходимо позаботиться об инициализации массива километров:

```
FOR k:=0 TO ПолныйПуть-1 DO Километр[k]:=0; END;
```

И о подсчете индекса покрытия для каждого километра:

```
FOR k:=0 TO Количество DO  
  FOR j:=Станция[k] TO Станция[k]+Пробег-1 DO  
    Километр[j]:=Километр[j]+1;  
  END;  
END;
```

После чего начинается главный процесс поиска лишних заправочных станций, каковой оформим в виде одного цикла, пробегающего по всем станциям и на каждой итерации принимающего решение, оставить очередную станцию в списке или нет.

```
k:=0;
WHILE k<=Количество DO
    /Принятие решения об очередной заправке./
    /Корректировка данных в случае удаления заправки./
    k:=k+1;
END;
```

Для принятия решения необходимо и достаточно посмотреть каждый километр, достигаемый из данной заправки, и если индекс всех километров больше единицы, то заправка, очевидно, лишняя. Перед началом проверки предположим, что заправка лишняя, а если найдется хотя бы один километр с индексом покрытия, не большим единицы, то заправка должна быть объявлена нужной:

```
flag:=TRUE;
FOR j:=Станция[k] TO Станция[k]+Пробег-1 DO
    IF Километр[j]<=1 THEN flag:=FALSE; END;
END;
IF flag THEN /Обнаружена лишняя заправка./
    /Уменьшаем индексы километров./
    /Убираем лишнюю заправку./
END;
```

От каждого индекса километра, покрываемого данной заправкой, необходимо отнять единицу:

```
FOR j:=Станция[k] TO Станция[k]+Пробег-1 DO
    Километр[j]:=Километр[j]-1;
END;
```

После чего заправка вычеркивается из списка:

```
FOR j:=k TO Количество-1 DO Станция[j]:=Станция[j+1]; END;
Количество:=Количество-1;
```

Ниже полный листинг.

```
MODULE Модуль;
    IMPORT In, StdLog;
    PROCEDURE Расчет*;
    VAR
        ПолныйПуть, k, j, Количество, Пробег, t: INTEGER;
        Станция: ARRAY 20 OF INTEGER;
```

```
Километр: ARRAY 100 OF INTEGER;
flag:BOOLEAN;

BEGIN
  StdLog.Clear;
  In.Open;
  In.Int (ПолныйПуть);
  In.Int (Пробег);
  In.Int (Количество);
  Станция[0]:=0;
  FOR k:=1 TO Количество DO
    In.Int (Станция[k]);

  END;
  FOR k:=0 TO ПолныйПуть-1 DO Километр[k]:=0; END;
  FOR k:=0 TO Количество DO
    FOR j:=Станция[k] TO Станция[k]+Пробег-1 DO
      Километр[j]:=Километр[j]+1;

    END;

  END;
  k:=0;
  WHILE k<=Количество DO
    flag:=TRUE;
    FOR j:=Станция[k] TO Станция[k]+Пробег-1 DO
      IF Километр[j]<=1 THEN flag:=FALSE; END;

    END;
    IF flag THEN
      FOR j:=Станция[k] TO Станция[k]+Пробег-1 DO
        Километр[j]:=Километр[j]-1;

      END;
      FOR j:=k TO Количество-1 DO Станция[j]:=Станция
[j+1]; END;
        Количество:=Количество-1;
        k:=k-1;

    END;
    k:=k+1;
  END;
  FOR k:=0 TO Количество DO StdLog.Int (Станция[k]);END;
END Расчет;
END Модуль.
```

Примечание. В главном цикле после вычеркивания заправочной станции стоит оператор $k:=k-1$. Это необходимо для того, чтобы ни одна заправка не выпала из анализа. Действительно, если заправка с номером k вычеркнута из списка, то заправка с номером $k+1$ меняет свой номер на k . Но номер очередной анализируемой заправки увеличивается на единицу, и, следовательно, станция, которая изменила номер с $k+1$ на k выпадет из дальнейшего анализа.

Проблема для самостоятельного исследования

Вернемся к вопросу об однозначности решения. Легко придумать пример, в котором лишними могут оказаться две заправки, *A* и *B*, но не одновременно.



Рис. 6.4. Выбор из двух заправок

По рисунку хорошо видно, что ликвидация заправки *A* делает необходимой заправку *B*, и наоборот. Но такая ситуация не создает сомнения в нашем алгоритме, так как по условию алгоритм должен определить только минимальное количество, но никак не конфигурацию заправочных станций. Подумайте, возможен ли пример, когда ликвидация в списке одной заправки делает необходимыми сразу несколько заправок. И если да, то можно ли модифицировать построенный алгоритм для учета такой ситуации.

Задача. Постфиксная и префиксная записи арифметического выражения

Условие задачи. Попробуем найти решение двух взаимобратных задач. В первой дано арифметическое выражение, состоящее из натуральных чисел, знаков арифметических операций и скобок, необходимо преобразовать его в равнозначное в форме обратной польской записи (постфиксной). Во второй задаче, наоборот, преобразовать выражение, записанное в виде обратной польской записи, в обычную форму (префиксную).

Прямая задача

Что такое обратная польская запись? Это форма арифметического выражения, позволяющая записать любое выражение без скобок, при этом арифметические операции записываются в том же порядке, в каком и выполняются. В обратной польской записи аргументы записываются перед знаком своей операции через какой-либо разделительный знак. Например, $2+2$ можно записать так $2\ 2\ +$, в качест-

ве разделительного знака будем использовать пробел. Пример более сложного выражения: $4 * (2 + 3) = 4 \ 2 \ 3 \ + \ *$.

Вычислим выражение во второй форме. Выполнять умножение нельзя, так как перед знаком умножения нет чисел. Первый знак, перед которым есть два числа, — это знак сложения, следовательно, необходимо выполнить операцию сложения. Выполним ее и получим следующее выражение: $4 \ 5 \ *$. В этом выражении единственный знак операции — это знак умножения, и перед ним два аргумента. Выполняем операцию и получаем 20. Действительно, обошлись без скобок. Идея, наверное, уже понятна, но для лучшего усвоения попробуем перевести более сложное выражение: $2 * ((1 + 3 + 4) - 8 * (5 + 3 - 3))$.

Метод перевода заключается в том, что на каждом шаге обработки арифметического выражения работаем только с двумя числами и одной операцией. Операции обрабатываем последовательно, иначе говоря, обратная польская запись — это не более чем форма обычного человеческого способа выполнения вычислений. Начнем с последней скобки. Первая операция $5 + 3$. После перевода $5 \ 3 \ +$. Далее вычтем тройку, это даст $3 \ 5 \ 3 \ + \ -$. Следующий шаг — умножение, переведем его, получим $8 \ 3 \ 5 \ 3 \ + \ - \ *$. Далее, чтобы не загромождать текст выкладками, оформим всю процедуру перевода в виде таблицы:

Таблица 6.5. Пример расчета

$5 + 3$	$5 \ 3 \ +$
$5 + 3 - 3$	$3 \ 5 \ 3 \ + \ -$
$8 * (5 + 3 - 3)$	$8 \ 3 \ 5 \ 3 \ + \ - \ *$
$1 + 3$	$1 \ 3 \ +$
$1 + 3 + 4$	$4 \ 1 \ 3 \ + \ +$
$(1 + 3 + 4) - 8 * (5 + 3 - 3)$	$4 \ 1 \ 3 \ + \ + \ 8 \ 3 \ 5 \ 3 \ + \ - \ * \ -$
$2 * ((1 + 3 + 4) - 8 * (5 + 3 - 3))$	$2 \ 4 \ 1 \ 3 \ + \ + \ 8 \ 3 \ 5 \ 3 \ + \ - \ * \ - \ *$

Можете вычислить оба окончательных выражения и убедиться, что их результат одинаков. А метод счета для новой формы несколько проще. Возможно, такому методу проще обучиться и человеку. В младшей школе на уроках математики специально учат работать со скобками, и не у всех учеников это сразу получается, а если бы основным способом записи в арифметике была бесскобочная запись, то обучение арифметике, возможно, шло бы легче. Но как-то так исторически сложилось, что основной метод записи скобочный, а бесскобочная запись — это интересный фокус.

Если метод построения обратной польской записи понятен, можно начать поиск программистского решения. Для начала заметим, что в случае элементарного арифметического выражения, состоящего из одной операции, обратная польская запись получается простым переносом знака операции за аргументы. Сразу возникает естественная идея использовать прием переноса знака для любого выражения. Действительно, конкретный знак «ничего не знает» обо всем выражении, он видит только аргумент слева и аргумент справа, и чтобы стать знаком обратной польской записи, он должен перепрыгнуть через правый аргумент. Аргумент справа вполне может оказаться сложным выражением, но это проблема скорее техническая, чем принципиальная. Сложность может быть в двух вещах:

- аргумент справа может оказаться выражением в форме постфиксной записи, состоящим из нескольких чисел и знаков;
- аргумент справа может оказаться еще не обработанным выражением.

Это проблемы действительно не принципиального, а технического характера. В первом случае, если справа стоит обратная польская запись, то проблема переноса знака сводится к проблеме различения уже обработанной части выражения и необработанной. Представьте себе ситуацию, вы берете знак, переносите его через число, стоящее справа от него, и обнаруживаете еще одно число. Вывод может быть только один. Это следующее число есть часть выражения, уже записанного в постфиксной форме, так как в префиксной форме не могут два числа стоять рядом без знака между ними.

Другая ситуация. Вы берете знак, переносите его через число и обнаруживаете знак. Вот такая ситуация уже не однозначна. Этот знак может быть как частью обработанного, так и частью необработанного выражения. Причем сам знак не может нести в себе никаких дополнительных признаков, позволяющих определить, что он есть такое. А если признаков нет, то их необходимо создать. Например, перенесенный знак (являющийся частью новой записи) можно выделять точками. Тогда проблема решается легко. Если перенесенный знак наталкивается на знак, то надо поискать точки выделения. Если они есть, то мы имеем дело с частью новой записи, если их нет, то мы имеем дело с необработанной записью.

Вторая проблема – это баланс скобок. Если на пути переносимого знака встречаются скобки, то необходимо пройти закрывающих скобок ровно столько, сколько и открывающих. Это можно и не доказывать, так как практически очевидно.

Алгоритм можно оформить как процесс последовательного переноса знаков за правый аргумент с учетом ситуаций, описанных ранее. Каждый переброшенный знак будем выделять точками, для того чтобы его можно было отличить от еще не обработанного. По завершении процесса достаточно удалить все точки и оставшиеся скобки.

Обратная задача

А сейчас займемся обратной задачей. Дано выражение, записанное в форме обратной польской записи. Перевести его в традиционную форму. Поиск решения не займет много времени с учетом того, что прямая задача уже решена. Мы можем пойти обратным путем, то есть переносить знак справа налево до того места, где ему должно стоять. Но необходимо точно определить, где это место.

Во-первых, заметим, что в постфиксной записи все знаки стоят правее своих аргументов. Это означает, что алгоритм можно построить как цикл прохода строки, на каждом шаге которого выясняется, не знак ли очередной символ, если да, то символ вырезается и переносится влево. Индекс, пробегающий строку символов, всегда уходит вправо, а знаки – влево. Это означает, что здесь нет, как в предыдущей задаче, необходимости выяснять, с каким знаком имеем дело – обработанным или необработанным, любой знак справа от текущей позиции индекса – это необработанный знак.

Далее, предположим, что при выполнении прямого преобразования, перенося знак, мы оставляли на его месте пробел, и пробелы в строке есть только там, где ранее стояли знаки. Тогда справедливо следующее утверждение: законное место знака – это первый пробел слева от него при условии, что последовательность переноса знаков правильная, такая как описана ранее (в прямой задаче), то есть если знаки пронумерованы слева направо, то в первую очередь переносятся те, которые имеют меньшие номера. Данное утверждение позволяет очень легко найти законное место знака, это, как и сказано в утверждении, первый пробел слева.

Проблема для самостоятельного исследования

Обратная задача все же имеет очень существенное отличие. В прямом преобразовании алгоритм практически игнорировал проблему скобок. А именно было вполне достаточно при переносе знака считать баланс скобок, а после завершения всех преобразований убрать все скобки. Здесь же скобки необходимо возвращать на место, причем в процессе преобразования. А как определить это самое место

и как определить место закрывающей скобки для только что поставленной открывающей (или наоборот)?

Проблема для самостоятельного исследования

В алгоритме есть одно очень слабое место. Предполагается, что выражение для алгоритма обратной задачи получено после работы алгоритма прямой задачи, только так можно гарантировать наличие пробелов, указывающих, где должен стоять знак. А что делать, если пробелов нет? Например, если выражение для обратной задачи было изначально записано без пробелов, то есть было получено не нашим алгоритмом, а как-то иначе?

В заключение

Разобранные задачи, и прямая, и обратная, – хороший пример на выполнение сложного символьного преобразования. В таких ситуациях бывает полезно построить модель того, как бы эту задачу решал человек. Конечно, далеко не всегда такой подход эффективен. Например, моделировать человеческое поведение в численных расчетах вряд ли бывает разумно, но что касается операций преобразования символьных множеств, то зачастую это единственно возможный путь поиска решения.

Нам были даны две задачи, прямая и обратная. Обратимость задач привела к хорошей обратимости алгоритмов. Вообще-то, это большая удача. Далеко не всегда так бывает. Если прямой алгоритм эффективен, это не означает, что так же эффективен будет и обратный. То же самое можно сказать и об остальных свойствах алгоритма.

Но тем не менее на обратимости алгоритмов можно выстроить метод решения задач. Представьте себе, что решение некоей задачи очень долго не удается найти. Вполне может оказаться, что обратная задача существенно проще. Поэтому:

- сформулируйте обратную задачу;
- не доводя ее до уровня программы, сформулируйте основные идеи возможного решения;
- подумайте, как эти идеи соотносятся с прямой задачей.

Вероятность того, что идея обратной задачи поможет вам лучше понять прямую, очень велика.

Задача для самостоятельного решения

Мы довольно детально разобрали алгоритмы и прямой, и обратной задач. Попробуйте самостоятельно написать программы.

Задача. Самый длинный путь рубки

Условие задачи. На стандартном поле 8×8 расставлены черные шашки и одна белая. Необходимо найти самый длинный путь рубки для белой шашки.

Ясно, что совокупность всех возможных путей рубки представляет собой ориентированный граф, а графы очень хорошо представляются связными списками (это к вопросу о технике представления данных). А так как совокупность путей рубки хорошо представляется ориентированным графом, следовательно, иная более программистская постановка задачи звучит так: дан ориентированный граф, найти его самую длинную ветвь (или все самые длинные ветви, что не намного сложнее).

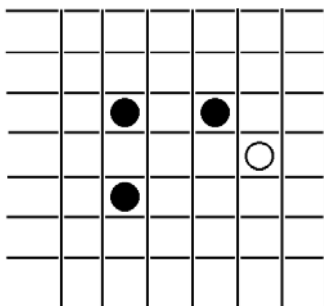


Рис. 6.5. Пример позиции

Конечно, новая формулировка не совсем идентична первоначальной, она исходит из данности графа, а в постановке задачи дана позиция на шашечной доске. Поэтому мысль разделить задачу на две – построение графа и поиск на графе – будет вполне естественной. Получив позицию, мы преобразуем ее в граф, а затем в соответствии с новой формулировкой подумаем о том, как обойти этот граф в поисках самой длинной ветви.

Примечание. Обратите внимание, что наши рассуждения носят подготовительный характер и не имеют прямого отношения к идее решения. О том, как, собственно, решать задачу, еще не было сказано ни одного слова. А вывод о возможности разбиения задачи на две нетерпеливому человеку может показаться даже лишним. Он может сказать: «Я хочу сразу придумать, как именно искать самый длинный путь». И, скорее всего, если этот человек не без спо-

собностей, то так оно и будет. А дальше будет одно большое НО. Придумать идею можно, но ее надо еще и реализовать! И нужно сказать, что зачастую возникающие технические проблемы настолько велики, что эффект от идеи сходит на нет. Что же мы фактически сделали, переформулировав задачу? А мы как раз и озаботились хотя бы обозначением этих будущих технических проблем, приблизили постановку к тем структурам данных, которыми мы реально располагаем в языке программирования.

Внимательный читатель мог бы сказать, что ранее мы начинали именно с идеи, выраженной в общепонятных терминах, и зачастую даже не математических. Но здесь нет противоречия. Надо просто видеть разницу в задачах. Если задача включает в себе большую смысловую проблему, то, конечно, главные усилия должны быть сосредоточены на разработке логики. Рассматриваемая задача – то, что называется технически сложная, поэтому и подход к ней иной. Конечно, что сложно технически, а что логически – это решается каждым разработчиком относительно себя. Что для одного сложно, то для другого легко! Что для одного сложная логика, то для другого технические детали!

Если вы согласны с тем, что было сказано в примечании, то вы должны быть согласны и с тем, что наш следующий шаг – это описание структуры данных, представляющих собой граф. Структуру данных определяет условие. Мы решили использовать динамическую память, следовательно, использование связанных списков – дело решенное, и осталось только определить структуру записи связанного списка.

Очевидно, что элемент связанного списка нужен для описания текущей позиции рубящей шашки, следовательно, необходимо просто посмотреть, что известно про позицию. А известно следующее:

- координаты текущей позиции;
- адреса элементов связанного списка, описывающих позиции, в которые рубящая шашка может попасть из текущей.

Достижимых позиций не более трех (позицию, из которой шашка пришла, можно исключить), а координат две. Следовательно, в структуре элемента связанного списка должно быть три поля, являющихся указателями на элемент связанного списка, и два числовых поля, имеющих смысл координат.

После обсуждения структуры данных можно обдумать, как эта структура данных должна быть построена:

- количество указателей в нашей структуре данных также фиксировано, но они не обязаны все указывать на реально существующие адреса;

- ветви строятся до тех пор, пока это возможно, различные ветви могут иметь различную длину.

Эти замечания существенны, но они не касаются общей структуры программы, которую можно построить как поиск на двоичном дереве. Это рекурсивная процедура, получающая на вход координаты текущей позиции и выполняющая следующие действия:

Для каждого из 4 возможных направлений рубки

Если в рассматриваемом направлении рубка возможна, то
Осуществляем рубку.

Ищем свободный указатель и от него создаем указатель на новый элемент списка, который и будет содержать информацию о новой позиции рубящей шашки.

Вызываем процедуру с координатами нового положения шашки.

Далее зададим себе вопрос, что в нашем алгоритме неопределенного, ответ на который даст информацию для дальнейшего анализа. А неопределенности есть, и достаточно существенные, например:

- не вполне понятно, как определить, возможно данное направление или нет. И что такое направление вообще;
- в чем заключается процедура рубки;
- предполагается, что возможных направлений рубки четыре. Это не так. Их только три, так как с одного из направлений шашка пришла, и его как направление рубки рассматривать нельзя. Но если мы желаем одно из направлений исключить, то необходимо придумать, как его отделить от других трех;
- и самое главное – из сказанного ранее совершенно не ясно, как мы собираемся обходить построенный граф в поисках самого длинного продолжения, хотя, может быть, пока это и не важно.

Попробуйте подумать хотя бы над первым вопросом. Даже интуитивно ясно, что в структуре данных нет достаточной информации для ответа. Это от того, что была совершена серьезная ошибка. Описав необходимую структуру графа и начав рассуждать об алгоритме (программе), мы ничего не сказали о том, как данный граф будет построен. Повторимся еще раз. Необходимо понять не только, то что граф будет из себя представлять, но и то, как он строится из первичной информации, то есть из той информации, которая дана непосредственно в условии.

Первичная информация – это позиция на доске. Ее можно представить, например, двумерным массивом. Договоримся, что пустая клетка будет представлена нулем. Шашка, осуществляющая рубку, пусть изображается числом два, а ее противники пусть будут единицами.

Теперь можно ответить на поставленные вопросы.

1. Направление – это вектор, такой что если его прибавить к координатам (индексам) элемента двумерного массива доски, то получим элемент массива, представляющий клетку, в которую можно осуществить ход. Таких направлений (векторов) четыре: $(-2, -2)$, $(-2, 2)$, $(2, -2)$, $(2, 2)$.
2. Для того чтобы выяснить допустимость направления, необходимо установить истинность двух утверждений: первое – ячейка массива, полученная прибавлением к координатам (индексам) текущего элемента вектора направления, содержит ноль (клетка доски пуста); второе – ячейка массива, полученная прибавлением половины вектора направления, содержит единицу (в клетке доски стоит шашка противника).
3. Процедура рубки заключается в двух действиях:
 - изменении координат (индексов) текущего элемента на вектор направления;
 - обнулении элемента, полученного прибавлением половины вектора направления.

Отбросить одно из направлений можно двумя способами. Например, запоминать, откуда пришла шашка, для чего в элементе связанного списка ввести одно дополнительное поле, описывающее источник перехода. Но можно этого и не делать, если немного подумать о том, как проводится проверка на допустимость направления.

Только что было сказано, что одно из условий допустимости – это наличие единицы в ячейке через «половину вектора направления», а в описании процедуры рубки сказано, что эта единица обнуляется. Это означает, что направление, с которого шашка пришла, при расчете следующего хода будет недопустимым (там после выполнения хода стоит ноль). Следовательно, можно проблему допустимости проигнорировать и проверять все четыре направления. Одна из проверок окажется лишней, но в качестве компенсации отпадет необходимость в дополнительных операциях по проверке допустимости, которые почти наверняка окажутся более сложными, а следовательно, мы получим и еще выигрыш в упрощении логики программы.

Проблема восстановления данных. Переборные алгоритмы, а мы имеем дело именно с перебором, требуют «отката», то есть восстановления данных в некоторой точке перебора с целью построения нового варианта. В нашей ситуации «откат» заключается в восстановлении одной-единственной шашки, срубленной только что. Из смысла процедуры рубки ясно, что срубленная шашка находится ровно посре-

дине между исходной и конечной позициями рубящей шашки (при рубке одной шашки противника). То есть координаты поля, на котором стояла шашка (индексы элемента массива, содержащего единицу), можно найти как среднее арифметическое координат исходного и конечного полей. Это, в свою очередь, потребует запоминания информации об исходном поле.

Все, что обсуждалось до сих пор, касается только построения графа (связного списка), а теперь подходим к главному, к проблеме поиска наидлиннейшего пути рубки, или, в наших терминах, наидлиннейшей ветви графа, или, что то же самое, наидлиннейшей ветви связного списка.

Решение этой задачи сводится к необходимости помечать элементы связного списка, через которые проходит путь рубки. Можно, конечно, создать дополнительную структуру, в которую наидлиннейший путь будет записываться, но дополнительные структуры данных – это дополнительное усложнение логики, и, может быть, существенное. Поэтому лучше создадим еще одно поле в элементе связного списка для хранения информации о пути. Это тоже дополнительная структура, но она уже локальная.

Эта дополнительная информация может быть длиной уже построенного пути, проходящего через данное поле. В этом случае алгоритм поиска наидлиннейшего пути построить несложно:

Числовую характеристику начального узла объявить максимальной.

Пока не достигнут тупик, делать

 Проверить все поля ссылок

 Если ссылка не тупиковая, то

 Если элемент, на который указывает ссылка,
 содержит характеристику, равную максимальной,
 то перейти по этой ссылке.

Нетрудно заметить, что алгоритм обеспечивает поиск не одного маршрута, а всех существующих. Сложность в понимании, наверное, вызывает первый пункт, в котором числовая характеристика начального узла объявляется максимальной. Может быть, не вполне понятно, на каком основании. Заметим, что наидлиннейший маршрут обязан проходить через начальный элемент, а так как числовая характеристика длины только одна, то она и будет содержать длиннейший маршрут по окончании работы.

А теперь все-таки о механизме. Значение длины маршрута известно только на последнем поле (последнем элементе связного списка). Следовательно, определение характеристики длины должно заключаться в возврате этого конечного числа при возврате по маршруту.

Пусть шашка находится в некоторой точке A и предстоит принять решение о числовой характеристике для пункта A . Предположим, что из A выходит максимальное количество ненулевых указателей (то есть три), это означает, что характеристику для A необходимо выбирать из трех возможных значений, которые возвращены в A рекурсивными вызовами. Какое из них выбрать? Конечно же, максимальное.

И последнее, когда принимать решение о числовой характеристике для текущего элемента (поля доски)? Конечно же, тогда, когда будут отработаны все допустимые направления рубки. Из этого следует, что формировать числовую характеристику мы можем в процессе построения графа (связного списка). Все существенные технические проблемы обсуждены, можно записать алгоритм. Головной алгоритм, видимо, будет состоять из двух процедур:

- алгоритм построения связного списка – графа;
- алгоритм поиска длиннейшего маршрута.

Алгоритм построения связного списка:

Ввод данных в виде двумерного массива

Создание начального элемента списка с записью в него исходного положения рубящей шашки

Вызов рекурсивной процедуры построения связного списка

Запись в начальный элемент полученной длины длиннейшего маршрута

Рекурсивная процедура построения связного списка:

Входные данные: адрес на текущий элемент списка, длина уже пройденного маршрута

Запомнить адрес на текущий элемент

Обработка направления $(-2, -2)$

Максимальная длина = возвращенной длине

Восстановить адрес на текущий элемент

Обработка направления $(2, -2)$

Длина = возвращенной длине

Если Длина > Максимальной То Максимальная = Длине

Восстановить адрес на текущий элемент

Обработка направления $(2, 2)$

Длина = возвращенной длине

Если Длина > Максимальной То Максимальная = Длине

Восстановить адрес на текущий элемент

Обработка направления $(-2, 2)$

Длина = возвращенной длине

Если Длина > Максимальной То Максимальная = Длине

Восстановить адрес на текущий элемент

Записать Максимальную длину как характеристику длины маршрута для текущего элемента связного списка.

Обработка направления:

Проверить допустимость хода в заданном направлении

Если Ход допустим, То

Ищем свободный (то есть указывающий в никуда) указатель на адрес элемента списка

Создаем новый элемент списка

Записываем в новый элемент координаты новой позиции рубящей шашки

Вызываем процедуру построения связного списка с новыми фактическими параметрами

Восстанавливаем позицию

Возвращаем длину, равную пройденной длине

Иначе

Возвращаем длину, меньшую пройденной длины на 1 (так как ход оказался недопустимым)

Задача для самостоятельного решения

Все алгоритмические проблемы задачи уже исследованы, напишите самостоятельно программу.

В заключение

Мы решили достаточно сложную задачу на построение и обход графа. Есть смысл еще раз выделить ключевые моменты логических построений:

- переборная задача на графе – это перебор всех возможных путей;
- перебор всех возможных путей осуществляется двумя взаимобратными операциями: ход на один шаг вглубь дерева и возврат;
- для того чтобы обеспечить возврат, необходимо обеспечить запоминание данных перед тем, как совершать очередной шаг в глубину;
- если нас интересует характеристика, значение которой будет известно только в конечной точке, а использование этой характеристики предполагается в начале графа, то есть смысл ее значение при возврате переносить из конца в начало.

Задача. Одинокий путник с плохой памятью

Условие задачи. На квадратном поле установлены препятствия произвольной формы и два пункта *A* и *B*. Перед путником

поставлена задача – найти путь из A в B . Известно, что путник не располагает картой местности, у него очень плохое зрение и очень плохая память, настолько плохая, что он практически не может запомнить приметы пройденного маршрута. Из средств ориентации путник располагает естественным чувством ориентации в пространстве своего тела, то есть он может сказать, где у него левая рука, а где правая, также есть прибор, напоминающий компас. Этот прибор всегда, в любой точке поля показывает направление на пункт B .

Задача может показаться простой. Действительно, рассудим так. У путника есть только две возможности для движения: по пустому пространству и вдоль препятствия. Стратегия движения в первом случае очевидна. Он идет по компасу (будем это устройство называть компасом), пока не встретит препятствия, после чего просто идет вдоль препятствия до тех пор, пока не обойдет его, а обойдя, опять движется по компасу.

Рассуждение вроде бы верное, за исключением одного – не ясно, откуда он узнает, что препятствие пройдено. Этот факт легко установить только в том случае, когда препятствие представляет собой простую фигуру, например квадрат. А в примере, приведенном на рисунке ниже, это очень даже непросто. Представьте себе, что путник попал в такой лабиринт, и подумайте, откуда, не имея карты, он может взять информацию, что лабиринт пройден.

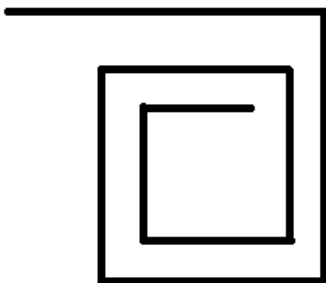


Рис. 6.6. Лабиринт

Можно попробовать выйти из положения, используя следующую стратегию: движемся вдоль стенки препятствия, держась за стенку правой (левой) рукой, до тех пор пока не появится возможность движения в сторону пункта B . Как только появляется возможность такого движения, отрываемся от стенки и движемся по компасу.

Идея неплохая. Заметьте, здесь используется способность путника распознавать, где право, а где лево. Это хорошая примета. В решении корректно поставленных задач должны использоваться все условия, это знают те, кто имеет опыт решения математических или физических задач. Но, несмотря на хорошую примету, идея ошибочна, что видно на следующем рисунке:

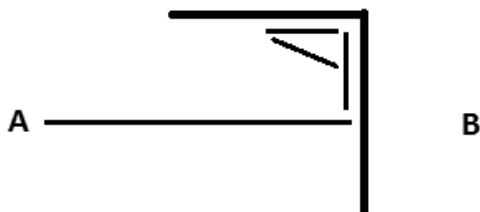


Рис. 6.7. Хождение по кругу

Путник начинает движение от A к B . Встречает препятствие. Пользуясь правилом правой руки, он начинает движение вдоль вертикальной стенки и движется, пока не встретит горизонтальную стенку. Пока возможности двигаться по компасу нет, вертикальная стенка этого не позволяет. Поэтому он начинает движение вдоль горизонтальной стенки и, пройдя совсем немного, обнаруживает, что движение в направлении B возможно, и начинает движение по компасу. Но, пройдя совсем немного, путник опять упирается в вертикальную стенку, более того, он упирается в одну из тех точек, в которой он уже был, и начинает двигаться по кругу.

Два примера, рассмотренные ранее, были нужны для того, чтобы показать сложность задачи, необходимость тщательного анализа.

Еще один важный вопрос связан с проблемой существования решения. Это любимая проблема фундаментальной математики. Программисты – в основном прикладники, мы всегда полагаем, что поставленная задача имеет решение. Но это совсем не очевидно, огромное количество задач вообще алгоритмически неразрешимо, а условие исследуемой задачи выглядит весьма сложно, и было бы полезно подумать, а для какого типа препятствий можно найти решение гарантированно.

Решение задачи существования может оказаться весьма сложным, тем более что мы пока даже не решили, как провести классификацию препятствий и что вообще такое «тип препятствия». Но давайте сейчас поступим как самые обычные прикладники, полагая, что алгоритмическое решение есть, найдем его, а затем, глядя на полученный

алгоритм, попробуем вывести условия, для которых он работает. Это вполне разумный подход. Действительно, не совсем ясно, как можно искать условия работоспособности алгоритма, его не имея.

Кое-что пробудущий алгоритм уже известно. Очевидно, что единственная разумная стратегия поведения путника на пустом пространстве, — это движение по компасу. Других причин для выбора направления у него на пустом пространстве просто нет. Единственный способ движения при обнаружении препятствия — это, конечно, движение вдоль стенок, как уже предлагалось. Использование правила левой или правой руки также вполне целесообразно. Действительно, пользуясь одним из этих правил, можно гарантированно обойти препятствие и вернуться в ту точку, с которой движение вдоль препятствия началось, а следовательно, и точка, в которой надо продолжить движение по компасу, также будет достигнута. Вопрос остается только в том, как определить точку отрыва от стенок.

Прежде всего зададим себе вопрос: на основании какой информации мы можем сделать заключение о том, что нужная точка достигнута. У этого вопроса есть две формы. Можно спросить, что необходимо знать для определения точки отрыва, а можно спросить, как использовать то, что уже известно. Ответ на любой из этих вопросов даст решение, но вопрос в первой форме выглядит как-то неопределенно. Может получиться так, что мы придумаем необходимые условия, а их в наличии не окажется. Все-таки более реально исходить из того, что есть.

Что же мы знаем о движении вдоль препятствия? Путь вдоль стены информации в себе не несет никакой. Путник не может определить положение этой стенки в пространстве. Он, наверное, может определить длину пройденного участка в шагах, но это вряд ли ему поможет. Еще одно важное событие, с которым он сталкивается, — это поворот. Типов поворота может быть два. Они показаны на рисунке:

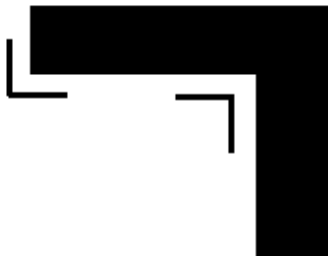


Рис. 6.8. Два типа поворотов

Различие между этими двумя поворотами, в типе угла. Для одного из них угол вогнутый, для другого выпуклый. Мы так их и назовем: выпуклый и вогнутый повороты. Можно назвать их еще левым поворотом и правым, но первое название нагляднее. Теперь путник может собрать содержательную информацию, а именно, двигаясь вдоль препятствия, он может считать выпуклые и вогнутые повороты. Ясно, что последовательность этих поворотов как-то характеризует форму препятствия, а знание формы как раз и требуется для определения точки отрыва.

А сейчас обратите внимание на следующий интересный факт. Ранее были приведены примеры проблемных препятствий. Их особенность в том, что они имеют вогнутые углы, для обхода которых необходимы вогнутые повороты. Рассмотрим полностью выпуклое препятствие – такое, как на картинке далее.

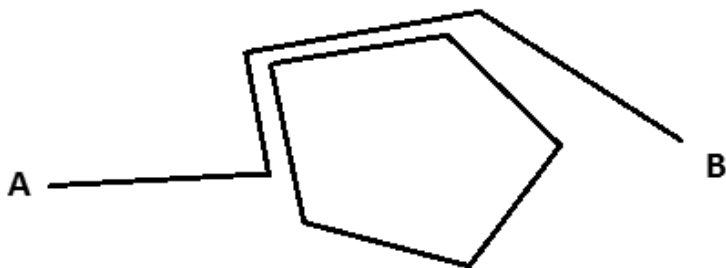


Рис. 6.9. Обход выпуклого препятствия

К этому препятствию вполне применим простейший алгоритм отрыва, предложенный в самом начале анализа. Действительно, возможно идти вдоль стенок, пока не появится свободное пространство в направлении, показываемом компасом, после чего можно отойти от стенки и пойти к В. Проблемы начинаются с появлением вогнутых углов. Попробуем провести их анализ, применив следующий исследовательский прием: предположим, есть хорошая структура данных, хорошая в том смысле, что мы знаем, как с ней решать задачу обхода. В нашей задаче такой хорошей структурой будет полностью выпуклая фигура. И есть плохие структуры, с которыми мы пока не знаем, как работать, в нашей задаче плохая структура – это фигура с вогнутыми углами. Новый прием заключается в следующих действиях:

1. Построим пример хорошей структуры.
2. Найдем минимальное изменение, такое что оно превращает хорошую структуру в плохую.

3. Посмотрим, что изменилось и как эти изменения можно компенсировать.

Применим описанный прием к нашей задаче.

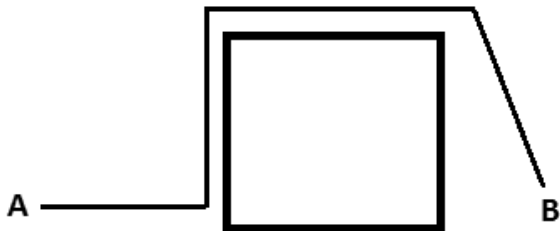


Рис. 6.10. Шаг первый. Очень хорошая структура

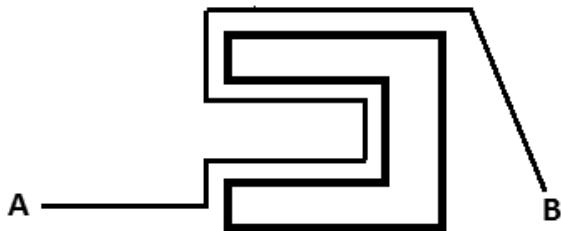


Рис. 6.11. Шаг второй. Добавляем к квадрату только одну вогнутость

Что изменилось в траектории движения одинокого путника? Ответ очевиден. Если его траектория, как уже говорилось, характеризуется последовательностью выпуклых и вогнутых углов, то можно заметить, что в данной траектории появились два вогнутых и два выпуклых угла. Прием дал продуктивную идею: если на препятствии появляется некоторое количество вогнутых углов, то они компенсируются таким же количеством выпуклых. Мы больше не будем приводить примеров, а вы можете попробовать проверить это утверждение на более сложных фигурах.

Проблема для самостоятельного исследования

Выше было дано определение выпуклого и вогнутого углов без учета возможного значения этих углов. Более того, пока мы рассматривали только прямые углы. Подумайте, что изменится, если попытаться учесть величину угла. Можно ли от учета величины полностью отказаться?

Теперь можно сформулировать и стратегию поведения путника. Путник движется, исходя из того, что препятствие перед ним хорошее, то есть полностью выпуклое. Из этого соображения путник после каждого выпуклого поворота пытается начать движение по компасу, если это не получается, то он движется вдоль стенки до следующего выпуклого поворота. Если же ему встречается некоторое количество вогнутых поворотов, то он не делает никаких попыток движения по компасу, пока не пройдет столько же выпуклых поворотов, сколько он прошел вогнутых.

Проблема для самостоятельного исследования

Во всех рассуждениях мы полагали, что путник пользуется (при обходе препятствия) либо стратегией правой руки (держится за стенку правой рукой), либо стратегией левой руки. А что изменится, если он от препятствия к препятствию будет менять стратегию? В качестве подсказки попробуйте ответить на второй вопрос.

Проблема для самостоятельного исследования

Подумайте, а есть ли у нашего путника возможность отличить только что пройденное препятствие от нового.

Мы создали стратегию, опираясь на специальный прием построения препятствий, и при этом утверждаем, что она будет работать для любого типа препятствий. Это, конечно, не очевидно, для полной уверенности нужно еще доказать следующее утверждение: *любое цельное препятствие можно построить из прямоугольника последовательными вырезами из него прямоугольников*, но справедливость этого утверждения лежит на поверхности и тратить время на его доказательство не будем.

Еще одна фундаментальная проблема – необходимо выяснить, в каких ситуациях можно найти путь из A в B , а в каких нельзя. Этим надо бы заняться сейчас, но мы поступим наоборот. Интуиция подсказывает, что ситуаций, в которых можно найти путь, очень много. Поэтому проблему существования отложим на потом. А сейчас займемся техническими деталями.

Главный алгоритм оформим в виде цикла, работающего до тех пор, пока не достигнут пункт B . Тело цикла составим из двух процедур: путь по компасу и обход препятствия. Естественное условие завершения пути по компасу – встреча препятствия, а условие завершения обхода препятствия – это отрыв от стенки. Оба условия можно проверять внутри самих процедур, это даже удобно, тогда текст главного алгоритма очень короток:

Пока не достигнут пункт назначения, делать

Выполнить процедуру движения по компасу.

Выполнить процедуру обхода препятствия.

Процедура движения по компасу заключается в прорисовке линии в направлении пункта назначения. Для чего в начале процедуры рассчитывается вектор, начальная точка которого – текущее положение путника, а конечная – пункт назначения. Затем полученный вектор преобразовывается в единичный, и тогда шаг движения путника – это смещение его на величину полученного единичного вектора до тех пор, пока не достигнут пункт назначения или не встретится препятствие. Единичный вектор необходим, так как путник видит впереди себя только на точку, и нет никакой гарантии, что на следующей точке он не столкнется с препятствием.

Процедура обхода препятствия технически не так очевидна. Ключевая сложность – в моделировании пространственной ориентировки путника. Не вполне ясно, как обозначить, где у него левая рука, а где правая. Обдумаем эту проблему. В процессе обхода путник выполняет различные повороты, в результате которых меняется его ориентация, и это надо как-то учитывать. Договоримся путника считать точкой на экране монитора с координатами x, y . Тогда его соприкосновение со стенкой препятствия возможно точками, имеющими следующие координаты: $(x+1, y)$, $(x, y+1)$, $(x-1, y)$, $(x, y-1)$. Если эти четыре точки пронумеровать произвольным образом, то номер точки, которой путник соприкоснулся с препятствием, можно считать числовым обозначением его ориентации. Затем в процессе обхода, каждый раз, когда путник совершает поворот, значение его ориентации в пространстве пересчитывается в соответствии с тем, какой поворот он совершает.

Движение вдоль препятствия – это последовательность шагов, но шагов не одинаковых. Существует несколько ситуаций.

1. В направлении точки соприкосновения путника со стенкой нет больше препятствия. В этом случае путник совершает выпуклый поворот и продолжает движение, соприкасаясь со стенкой уже другой точкой, номер которой перевычисляется.
2. Стенка не пройдена, тогда путник продолжает движение вдоль стены в соответствии со своей ориентацией.
3. Путник зашел в угол и должен совершить поворот. Поворот сводится к смене ориентации, или, что то же самое, к смене точки соприкосновения с препятствием. Наш путник совершает повороты против часовой стрелки, точки касания пронуме-

рованы также против часовой стрелки, поэтому смена номера точки сводится к его увеличению, если он меньше 4 (максимальный номер). Если же текущий номер = 4, то он становится равен 1.

В первых двух случаях соответствующим образом изменяются координаты путника, в третьем изменяется ориентация, то есть пересчитывается номер точки соприкосновения путника со стенкой препятствия.

Еще раз напомним, что ключевой момент для обработки любого движения – это ориентация путника, то есть номер точки, которой он в данный момент касается стенки препятствия. Все операции движения жестко привязаны к этой точке, поэтому они организованы в виде выбора из описанных альтернатив.

Мы потратили достаточно много времени на разработку алгоритма, полученное решение выглядит красиво, и если окажется, что оно применимо к небольшому количеству задач, работа будет напрасной. Поэтому сейчас сделаем небольшую, но важную работу по выявлению условий, при которых одинокий путник, пользуясь разработанным алгоритмом, сможет найти путь из точки *A* в точку *B*.

Критерий будем искать, анализируя алгоритм движения путника. Обратим внимание на два утверждения:

- любое цельное препятствие можно построить из большого прямоугольника, последовательно вырезая из него меньшие прямоугольники;
- путник сможет корректно обойти любое препятствие, построенное по только что сформулированному правилу, пользуясь созданным алгоритмом.

Эти два утверждения позволяют нам сформулировать очень серьезную **теорему**: если вся система препятствий, которую путнику предстоит преодолеть, находится внутри прямоугольника, и путник находится за его пределами, то, пользуясь построенным алгоритмом, путник сможет преодолеть эту систему препятствий.

Теорема выглядит правдоподобной, но не очевидной. Попробуем ее хотя бы нестрого обосновать.

Обоснование. Если система препятствий представляет собой цельное препятствие, то теорема очевидна из сказанного ранее. Предположим теперь, что система состоит из нескольких не связанных между собой препятствий. Для доказательства достаточно показать, что в процессе движения путник никогда не приблизится к

той стенке прямоугольника, охватывающего систему препятствий, с которой он начал свой путь. Очевидно, что если путник никогда не приблизится, то, следовательно, он будет удаляться и, следовательно, рано или поздно должен достичь отдаленной точки охватывающего прямоугольника.

Весь маршрут путника внутри охватывающего прямоугольника состоит из отрезков, вдоль которых он движется по компасу, и из ломаных линий обхода препятствий. При движении по компасу он, очевидно, приближается к удаленной точке. Следовательно, необходимо показать, что в ходе движения по линии обхода (вдоль стенок) он также приближается к удаленной точке охватывающего прямоугольника.

Рассмотрим текущее препятствие, с которым столкнулся путник. Для него существует охватывающий прямоугольник (назовем его малый охватывающий), ориентированный так же, как охватывающий прямоугольник для всей системы препятствий (назовем его большой охватывающий). Проходя малый охватывающий прямоугольник, путник начнет движение от точки *A* и оторвется от него в точке *B*. Ясно, что точка *B* ближе к удаленной стенке большого охватывающего прямоугольника, нежели точка *A*, а это, в свою очередь, означает, что, двигаясь по ломаной линии, он также приближается к точке назначения.

Эти рассуждения, конечно, нельзя считать строгим доказательством, скорее, это более-менее убедительное обоснование, но если к этому обоснованию добавить некоторое количество сложных тестовых примеров, то можно быть уверенным, что полученное решение если и не охватывает все возможные ситуации, то, во всяком случае, область его применения велика.

В заключение

Очень и очень часто неопытные программисты опускают обоснование своего алгоритма. Необходимость доказательства игнорируется по причинам, далеким от логики. Разработчик, придумавший красивый алгоритм, как правило, не готов согласиться с возможностью ошибки. Действительно эффективный алгоритм просто обязан быть работающим, особенно если на него потрачено много сил. В иное не хочется верить, в этом все дело, но такие рассуждения далеки от разумного поведения. Сколько бы разработчик не потратил времени и сил, его работа вполне может оказаться ошибочной. Это нужно помнить, и если алгоритм сложен, его дополнительное обоснование, если не доказательство, никогда не будет лишним.

Задача. Закраска односвязного контура

Условие задачи. На плоскости дан произвольный односвязный контур. Требуется его закрасить.

Контуром будем называть ломаную линию, конец которой совпадает с началом. Контур имеет внутреннюю область. Внутренняя область отличается от внешней тем, что любые две точки, одна из которых принадлежит внутренней области, а другая – внешней, разделяются линией контура и пройти из одной точки в другую, не пересекая линии контура, нельзя. Односвязный замкнутый контур – это такой контур, для которого любые две точки его внутренней области можно соединить линией (не обязательно прямой), не пересекая линии контура.

Задача достаточно сложная. Есть в ней трудности и идейного, и технического характера. Приступая к такой проблеме, полезно решить похожую, но более простую задачу. Это, во-первых, поможет в выработке идеи и уж во всяком случае даст полезный опыт в отработке необходимых технических навыков. Конечно, может получиться и так, что решение простого аналога окажется бесполезным, но опыт говорит, что похожие задачи часто имеют похожие решения.

Можно, конечно, возразить и так: решая похожую задачу мы теряем время, возможно впустую, а решать большую задачу все равно придется с нуля. Это правильное возражение, если у вас уже есть хорошая идея, и вы четко представляете, как ее реализовать, но ведь у нас пока нет никаких идей и ясного представления, что делать!

Поэтому подумаем об упрощении. В условии используются три содержательных термина: «контур», «произвольный», «односвязный». Упрощение условия должно быть как-то связано с изменением смысла этих терминов. Термин «контур», конечно, изменять нельзя, он включает в себе главный смысл задачи. «Односвязный» можно заменить, изменив связность контура, но это не облегчит, а усложнит задачу. Следовательно, остается только одна возможность – это сузить понятие «произвольный». И сразу запишем новое правило.

Если упрощение задачи – это изменение области применения искомого алгоритма, то следует в условии выделить существенные термины, определить область их значений и подумать, как можно уменьшить области значений без критического искажения смысла задачи.

Ограничение задачи контурами определенной формы, например прямоугольниками, будет слишком сильным сужением, поэтому попробуем поработать с произвольными выпуклыми контурами. Выпуклый контур можно определить как контур, любые две внутренние точки которого можно соединить прямой линией, не пересекающей с линией контура. Хорошим примером выпуклого контура может быть любой параллелограмм или треугольник. Более сложный пример – на следующем рисунке.

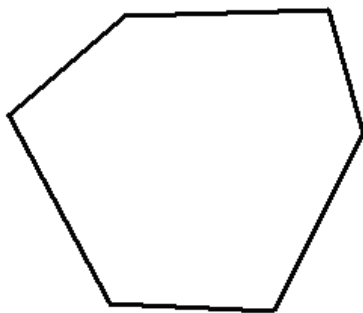


Рис. 6.12. Выпуклый контур

Обсудим некоторые алгоритмические идеи

Идея 1. Заливка. Представьте себе, как растекается вода, если ее вылить на поверхность. Если поверхность идеально гладкая и ровная, то граница водной поверхности будет, очевидно, окружностью (так как все направления равноправны) и остановится вода, только достигнув контура (а контур представьте себе как некий бордюрчик). Для реализации такого способа заливки достаточно одной точки, гарантированно находящейся во внутренней области. Затем запускаем процесс, в котором для каждой уже закрашенной точки просматриваются все соседи, для каждой соседней, незакрашенной точки выполняется закрашка, после чего она рассматривается в качестве следующего источника краски и т. д.

Описанный процесс явно рекуррентный, и если у вас появилась идея решить задачу рекурсивно, то эта мысль вполне естественна. Но против идеи рекурсивного решения есть сильное возражение. Рекурсия иногда требует серьезных ресурсов памяти, а процесс будет очень

сильно ветвиться, и мы рискуем сорвать закраску из-за переполнения стека. Но это будет хороший пример необходимости учитывать возможности машины и системы программирования. Иначе говоря, необходимо помнить, что алгоритм не висит в воздухе, он выполняется на конкретном компьютере, располагающем конкретными аппаратными возможностями.

Впрочем, можно поступить иначе. Если точки, являющиеся источником закраски, заносить в специальный массив записей координат, то без рекурсивных процедур можно обойтись. При очень длинном фронте разлива краски этот массив может растянуться на тысячи точек, что не слишком страшно. Если несколько тысяч одновременно работающих процедур – это много, то несколько тысяч элементов массива – это не очень много. Но тогда возникнет существенная логическая сложность. Каждая конкретная точка может являться точкой разлива краски только на один шаг цикла закраски. Это означает, что массив необходимо постоянно перестраивать. Одни точки в него будут записываться, другие уходить, длина массива окажется переменной величиной, что существенно усложнит логику алгоритма.

Может быть, упомянутые логические сложности и не так страшны, просто необходимо учесть их существование. Кроме того, худо без добра не бывает, и если удастся преодолеть все преграды, мы будем существенно вознаграждены.

Продолжим. Каждая точка-источник видит вокруг себя только на одну точку, то есть каждому источнику совершенно безразлична форма закрашиваемого контура. Это означает, что мы попали на идею, которая поможет решить исходную задачу.

Но у этого подхода есть существенный минус. Если закрашиваемая фигура велика, то фронт разливаемой краски может оказаться очень длинным, следовательно, массив, предназначенный для хранения координат точек-источников, также может оказаться слишком большим. Это влечет за собой расход памяти и неизбежную потерю скорости при перестройке массива.

Поэтому мы попробуем решить задачу двумя способами – изложим только что и методом, не требующим хранения больших массивов. Такой метод пока неизвестен, его мы еще поищем, а пока решим задачу закраски выпуклого контура.

Идея 2. Закраска параллельными линиями. Идея решения не сложна. Допустим, известна одна точка, гарантированно находящаяся внутри закрашиваемого контура. Пройдем по горизонтали влево от нее и вправо с целью найти, соответственно, левую и правую точ-

ки контура. Определив координаты искомых точек, проведем между ними линию. Затем поднимемся на один пиксел вверх и повторим описанные операции. Ясно, что при подъеме вверх внутренняя часть контура будет закрашиваться. Также поступим и для части контура лежащей ниже исходной точки. В построенном способе осталась одна неточность: говорится о подъеме вверх и о спуске вниз и не говорится о том, до каких пор. Каково условие завершения подъема и соответственно спуска? Ответ на этот вопрос не так прост. Предположим, мы будем выполнять подъем до тех пор, пока не встретим точку контура. Тогда возникает проблема, проиллюстрированная на следующем рисунке.

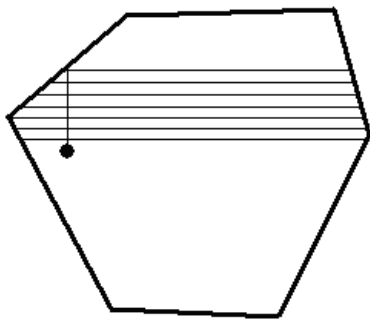


Рис. 6.13. Проблема закрашки

Здесь вертикальный отрезок – это линия движения от изначально известной точки до касания с контуром, а горизонтальные линии показывают закрашку. Видно, что вертикальный отрезок в своем движении вверх встретил линию контура до того, как контур был закрашен в части, расположенной выше исходной точки.

Проблема, впрочем, несложная. Ясно, что исходную точку надо просто смещать от границ контура. Делать это можно по-разному, мы используем красивый прием, позволяющий не выяснять, с какой стороны находится ближайшая линия границы.

На каждом шаге смещения вверх для определения координат закрашивающей линии необходимо вычислять координаты X левой и правой границ контура. Обозначим их через rx и lx . Необходимо обеспечить выполнение неравенства $lx \leq X \leq rx$. Заметим, что среднее арифметическое величин lx и rx всегда находится между ними. И если организовать пересчет координаты X на каждом шаге подъема (спуска) как среднее арифметическое крайних точек, то точка всегда

будет стремиться при подъеме к самой верхней вершине и при спуске к самой нижней. Отсюда сразу видно и условие завершения подъема (спуска). Подъем (спуск) возможен только до тех пор, пока $rx - lx > 0$, то есть до тех пор, пока левая и правая точки не совпадут.

Для разработки программы заметим, что спуск и подъем отличаются только направлением, следовательно, эти два процесса возможно реализовать одной процедурой, в которую в качестве параметров передаются координаты исходной точки и направление смещения. Операции поиска левой и правой точек отличаются только направлением, поэтому их также можно оформить одной функцией, получающей в качестве аргументов координаты точки, исходной для поиска и возвращающей координату X найденной точки.

Если вы не сочтете за труд написать и как следует протестировать программу, то попробуйте залить контуры, в которых хотя бы одна сторона сильно наклонена к горизонтали. Вполне возможно, вам удастся найти пример, в котором краска выльется за пределы контура. Если такой пример встретится, посмотрите внимательно на сильно наклоненную сторону. Хорошо видно, что фактически это не линия, а лесенка, между ступеньками которой могут быть пустые точки. Это небольшая техническая проблема, не имеющая принципиального значения, попробуйте решить ее самостоятельно.

Идем далее. Имея средство заливки выпуклых фигур, можно подумать о произвольном контуре, так как любой контур можно представить как объединение нескольких выпуклых. Пример такого объединения приведен на следующем рисунке.

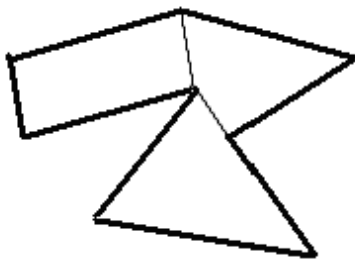


Рис. 6.14. Пример сложного контура

На рисунке тонкими линиями показано деление сложной фигуры на несколько выпуклых. Таким образом, задача закраски произвольного односвязного замкнутого контура свелась к задаче разбиения произвольной фигуры на несколько выпуклых. Такое разбиение мож-

но выполнить, если знать координаты вершин контура. Но в этом случае метод закрашки теряет универсальность, намного интереснее было бы найти метод, выполняющий свою работу и ничего не знающий о контуре, окружающем исходную точку. Кроме того, использование координат вершин, скорее всего, потребует достаточно трудоемких расчетов.

Конечно, было бы замечательно, если бы произвольный контур удалось закрасить, располагая только координатами одной внутренней точки. Однако логика и здравый смысл подсказывают, что резкое усложнение условия требует дополнительной информации. Это как золотое правило механики: выигрываем в расстоянии, – проигрываем в силе, и наоборот. Так же и здесь, выигрываем в условии (делаем его более сложным), – проигрываем в известной информации (больше требований к известному). Однако золотое правило можно направить в другую сторону. Можно попробовать не требовать дополнительных условий, а попробовать разработать более сложный алгоритм.

Более сложный алгоритм, используя только текущую информацию, должен каким-то образом определять появление новой выпуклой фигуры. Если в качестве основы для рассуждений используем алгоритм закрашки выпуклой фигуры, то текущая информация – это координаты левой и правой точек контура относительно точки спуска (подъема). Рассмотрим более внимательно процесс спуска на фигуре, разбитой на выпуклые, чуть выше.

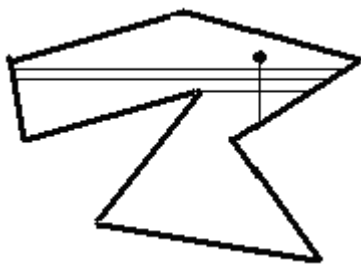


Рис. 6.15. Попытка закрашки

Исходная точка обозначена черным кружочком. Вертикальный отрезок показывает направление спуска. Направление показано без пересчета координаты точки спуска через среднее арифметическое левой и правой точек контура. Мы опустили пересчет для упрощения анализа. Итак, видим, что в некоторый момент горизонтальная линия резко изменяет свою длину, в данном случае она становится существ-

венно короче, одновременно с этим событием слева от точки спуска появляется треугольник, выпадающий из процесса заливки. Следовательно, резкое изменение длины можно связать с появлением новых фигур (точнее, с выпадением части фигуры из процесса заливки).

Нас, впрочем, интересует не только факт появления новой фигуры, но и то, где она находится. Поэтому рассмотрим не просто изменение длины горизонтальной линии, а изменение координат X левой и правой точек контура. Это действительно даст больше информации. Применительно к рассмотренному примеру можно заметить, что:

- выполнялось движение вниз;
- сильно изменилось значение левой координаты, она стала меньше.

Из этого делаем вывод, что координату X новой точки закрашки можно определить как среднее арифметическое координаты X предыдущей левой точки и координаты X текущей левой точки. При этом новая точка закрашки будет точкой спуска. Для лучшего понимания рассмотрим немного иной пример:

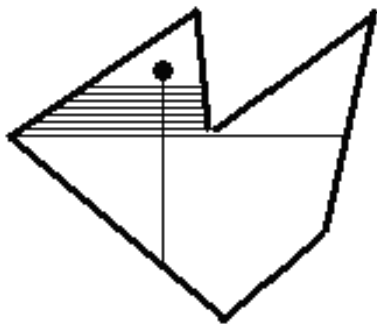


Рис. 6.16. Еще одна попытка закрашки

Здесь точка закрашки также спускается вниз, но изменения происходят не с левой, а с правой точкой, и ее координата не уменьшается, а увеличивается, при этом новая фигура расположена сверху, поэтому новая точка закрашки будет уже точкой подъема. Двух приведенных примеров, наверное, достаточно, чтобы понять, как принципиально решается задача:

- отслеживаем изменения координат левой и правой точек;
- если координата X одной из них претерпела резкое изменение, а слишком резким изменением будем считать изменение на 2

точки (по причине различного наклона линий контура), то формируем новую точку закраски, определяем новое направление и запускаем новый процесс закраски относительно определенной точки.

Единственное, что осталось сделать, — это определить возможные ситуации и для каждой определить координаты новой точки закраски и направление закраски. Возможно несколько ситуаций.

Левая граница стала больше при движении вниз.

Координата X новой точки закраски есть среднее арифметическое координат X предыдущей и текущей левой точки. Новое направление закраски — вверх.

Левая граница стала больше при движении вверх.

Координата X новой точки закраски есть среднее арифметическое координат X предыдущей и текущей левой точки. Новое направление закраски — вниз.

Левая граница стала меньше при движении вверх.

Координата X новой точки закраски есть среднее арифметическое координат X предыдущей и текущей левой точки. Новое направление закраски — вверх.

Левая граница стала меньше при движении вниз.

Координата X новой точки закраски есть среднее арифметическое координат X предыдущей и текущей левой точки. Новое направление закраски — вниз.

Четыре случая определяются и для возможных изменений правой точки. Мы их рассматривать не будем, так как аналогия практически полная. Программное решение можно оформить в виде рекурсивной процедуры закраски. Каждый новый вызов закрашивающей процедуры выполняется при обнаружении одной из восьми возможных ситуаций скачка длины.

Полученное решение обладает теми же недостатками, которые были указаны для алгоритма закраски выпуклого контура. Достоинством решения является малая требовательность к памяти, необходимой для хранения промежуточных данных. Достаточно трудно прорисовать контур настолько сложный, что потребуются работа большого количества рекурсивных процедур закраски. Наверное, это решение можно назвать оптимальным, если исправить имеющиеся недоработки, но тем не менее мы поищем второе решение.

Второе решение основано на идее, обсуждавшейся в самом начале: назовем исходную точку закраски источником. Смысл этого названия в том, что точка является источником краски для четырех со-



седних точек. А процесс можно построить так: создаем массив точек источников. На каждом шаге закрашки просматриваем весь массив и для каждой точки источника (из массива) выполняем поиск еще не закрашенных соседей (на поле). Найденный незакрашенный сосед закрашивается и становится новым источником. Точка, соседи которой проверялись, перестает быть источником и убирается из массива. Самый первый источник – это исходная точка, в которую льется краска. Таков принцип. Но, конечно же, принцип – это далеко не все, еще придется сражаться с техническими проблемами реализации.

Процесс закрашки, на первый взгляд, совершенно понятен. У каждой точки только четыре соседа (соседей по диагонали рассматривать не будем). Проверить их цвет сложности не составляет. Интуитивно ясно, что наиболее сложное место в программе – это массив точек-источников. Источники в массив должны заноситься и удаляться, соответственно, длина массива постоянно меняется, а характер этих изменений зависит от формы контура.

Конечно, мы можем построить алгоритм так, чтобы точки-источники на обработку выбирались не хаотично, а в определенном порядке, например с начала массива или с конца, но предсказать, сколько новых источников породит данный, практически невозможно. Известно только то, что как минимум их может вообще не быть, а как максимум – четыре. Правда, четыре соседа у источника могут быть только на первом шаге, когда внутри контура есть только один источник – исходный.

Предположим, что точки-источники, кандидаты на обработку выбираются с начала массива. Это означает, что, взяв из массива один источник, мы создали в массиве одну дырку, которую можно заполнить новым источником. Если взятый источник породит только один новый источник, то мы его запишем на место использованного, а если он породит несколько источников? Из сказанного ясно, что если и возможно брать источники на обработку только с начала, то записывать новые лишь в начало уже не получится. Возникшую проблему можно обойти, если источники на обработку брать не сначала, а с конца. Тогда разрывов в массиве источников не будет. Но пока выберем другую стратегию поведения. Введем для каждого элемента массива дополнительное описательное поле – флаг, который сыграет роль метки. Если элемент массива использован, то флаг имеет значение «истина», иначе флаг имеет значение «ложь». Тогда для выбора из массива источников кандидата на обработку достаточно найти в массиве источник с флагом, равным «истина». Данный прием сразу дает

критерий завершения процесса – процесс завершается тогда, когда не нашлось ни одного источника с истинным флагом.

Но можно для определения момента завершения работы использовать и косвенный критерий. Ясно, что если с каждым новым источником длину массива источников увеличивать, а с каждым использованным – уменьшать, то к концу процесса длина массива окажется равной нулю. Это были очевидные трудности, а есть одна неочевидная. Рассмотрим следующий рисунок:

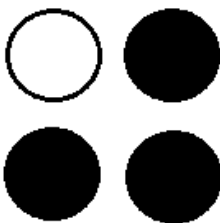


Рис. 6.17. Неочевидная проблема

Пусть черные точки – это источники, уже занесенные в массив, а белая – нет. В этой ситуации две черные точки могут объявить белую точку следующим источником, что даст удвоение потребности в размере массива в данной ситуации, а фактически размер массива начнет расти почти неограниченно, так как ситуации такого типа не редкость. В нашей стратегии записи (исключения) источников исключить источник можно только один раз, следовательно, если он окажется записанным несколько раз, балласт из лишних записей будет накапливаться быстрее, чем сбрасываться. Программа, написанная без учета озвученной проблемы, покажет, что удается закрасить только незначительную поверхность и существенное увеличение длины массива источников не приводит к значительному увеличению закрашенной области.

Решение проблемы заключается в ответе на вопрос, в каком порядке выполнять действия: записать точку как источник, а потом ее закрасить (когда она будет обнаружена в поиске источника) или наоборот.

Дело в том, что после занесения в массив источников точка может достаточно долго не попасть на обработку, если она была записана в конец массива, а это как раз и означает появление реальной возможности ее повторной и даже многократной записи. Чтобы этого избежать, необходимо после того, как точка попадет в массив, ее за-

красить, тогда для всех прочих ее соседей она уже не сможет быть источником. Поэтому ответ на поставленный вопрос такой: точка должна быть закрашена тогда же, когда ее координаты записываются в массив источников.

Проблема с закраской/занесением делает программу не совсем тривиальной, поэтому запишем ее алгоритм:

```
Длина массива источников = 1
Записываем в массив источников исходную точку
Пока длина массива источников больше нуля, делать
    Найти первый неиспользованный источник и для него
        Объявить его использованным
        Длину массива увеличить на 1
        Провести анализ соседей следующим образом
            Если сосед - незакрашенная точка, То
                Найти неиспользованный элемент
                массива источников.
                Сохранить в нем информацию о
                точке-источнике.
                Закрасить точку
            Уменьшить длину массива источников на 1
Перейти в начало массива.
```

Обратим еще раз внимание на логику изменения длины массива источников. Закрашенная точка полностью выпадает из дальнейшего анализа, и ее нужно убрать из массива, в результате чего длина массива, конечно, уменьшится, но мы, внося новый источник в массив, в тот же момент уменьшаем длину массива, и наоборот, убираем точку из массива, а длину увеличиваем.

Причина уже объяснена ранее. Точка-источник закрашивается в момент занесения в массив источников. Но, конечно, такая последовательность действий в алгоритме выглядит странно, чем осложняет понимание.

Обратите внимание на последовательность, в которой программа закрашивает точки (если вы её напишете, конечно). Минусы программы – сложность логики и затраты памяти. А сейчас третий вариант решения. Попробуем сделать логику работы с массивом источников более естественной. Для этого будем брать на обработку источники с конца массива и выполнять запись новых также в конец.

Логика изменения массива резко упрощается. Он начинает работать, как стек. Источник, последним попавший в массив, первый попадает на обработку. В массиве нет дырок и нет балласта. Логика программы, реализующая идею, настолько прозрачна, что мы не будем записывать алгоритм.

Задача для самостоятельного решения

Мы выше описали три способа закраски контура. Два из них отличаются способом обработки массива источников краски, и один построен на ином принципе. Реализуйте хотя бы один вариант в виде законченной программы.

Задача. Живая группа Го

В игре Го, правила которой мы сейчас изучать не будем, есть понятие живой группы. В задаче требуется установить, имея конкретную позицию и координаты одного камня, является ли этот камень представителем живой группы.

Возможно, вы незнакомы с правилами этой игры. Если так, то ничего страшного. Фактически сама игра для формулировки задачи не нужна, необходимо только определение живой группы. Игра ведется на квадратной доске размером 19×19 . Это классическая доска Го. Фигуры игры называются *камнями*. Ход игрока заключается в установке камня на перекрестие линий (правда, мы в решении будем устанавливать камень в клетку). *Группой камней* называется множество, в котором для каждой пары камней из множества можно пройти от одного камня пары до другого, выполняя на каждом шаге переход по соседним камням.

Соседом называется камень, стоящий на соседнем перекрестии по вертикали или горизонтали. Камни, стоящие рядом по диагонали, соседями не являются. *Живой группой* называется такая группа, в которой есть хотя бы один камень, такой что хотя бы одно соседнее с ним перекрестие пусто. Ниже на рисунке примеры живой и неживой групп с учетом нашего нарушения правил (ставим камни в клетки, а не на перекрестия).

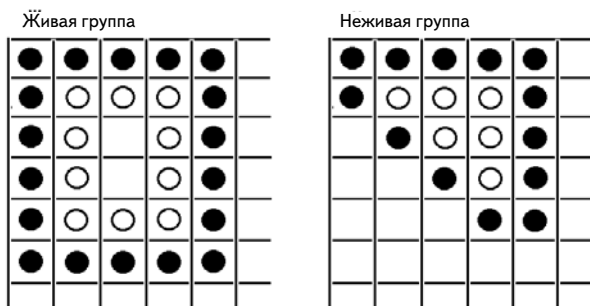


Рис. 6.18. Примеры живой и неживой групп Го

На примере этой задачи мы еще раз продемонстрируем различие между рекурсивным и нерекурсивным решениями. Вспомним, что в «программисткой» науке есть (правда, нестрогое) утверждение о том, что каждую задачу можно решить как рекурсивно, так и нерекурсивно. Вопрос только в осмысленности и эффективности. Поиск рекурсивных решений иногда довольно сложен, но бывает и наоборот. Но очень часто рекурсивный вариант решения использует большой объем ресурсов, рекурсия – очень требовательная дама.

Задача, к которой мы сейчас приступаем, имеет хорошее рекурсивное и хорошее нерекурсивное решения. Более того, различие между этими двумя типами решений в задаче о живой группе Го можно свести к минимуму. Минимальное различие заключается в организации памяти. Для рекурсии нужен стек. Следовательно, чтобы организовать рекурсию, необходимо создать конструкцию, моделирующую стековую память, например посредством массива. Первым шагом мы получим рекурсивное решение, обсудим его и затем получим второе, нерекурсивное.

Первый шаг в построении рекурсивной процедуры – надо увидеть рекуррентное определение процесса. В нашей задаче рекуррентный характер происходящего налицо. В процессе поиска происходит переход от камня к камню. Если пустое поле найдено, то группа живая, иначе нет. Общая схема рекурсии может быть такова:

- если рассматриваемое поле пустое, то поиск прекращается и возвращается сообщение об удачном его завершении;
- если рассматриваемое поле занято камнем противника, то поиск прекращается и возвращается сообщение о неудаче рекурсивного поиска;
- если рассматриваемое поле занято своим камнем, то выполняется новый вызов рекурсивной процедуры.

Это самая общая схема, не лишенная проблем. Первая и самая грубая ошибка заключается в том, что текущий вызов рассматривает только одно поле, а мы знаем, что у каждого камня может быть четыре соседа. Технически это означает, что либо должно быть четыре вызова процедуры, либо процедура должна рассматривать четыре ситуации. Как именно построить работу рекурсивной процедуры – вопрос вкуса и личного стиля. Мы выберем второй вариант – в процедуру передаются координаты поля, на котором находится свой камень, и процедура рассматривает четырех его соседей.

Пусть процедура получает на вход две координаты X , Y , тогда ее внешний вид может быть такой:

СИТУАЦИЯ = НЕУДАЧА

Рассматривается сосед слева

Если пустое поле, то СИТУАЦИЯ = УДАЧА

Если камень противника, то СИТУАЦИЯ = НЕУДАЧА

Если камень свой, то вызов процедуры с новыми координатами

Если СИТУАЦИЯ = НЕУДАЧА, то рассматривается сосед сверху

Если пустое поле, то СИТУАЦИЯ = УДАЧА

Если камень противника, то СИТУАЦИЯ = НЕУДАЧА

Если камень свой, то вызов процедуры с новыми координатами

И точно так же проработаем соседей справа и снизу. Сразу исправим небольшую ошибку (или просто недоговоренность). Ранее везде говорилось о рекурсивной процедуре, и по инерции в алгоритме речь идет о процедуре. В то же время говорится о некоторой переменной под именем СИТУАЦИЯ. Очевидно, данная переменная должна сообщать свое значение предыдущим вызовам, а для этого либо она должна быть глобальной величиной, либо рекурсию необходимо оформить в виде функции, возвращающей значения СИТУАЦИИ.

Следующая более тонкая ошибка нам уже встречалась в задаче о закраске контура, имеются в виду решения, основанные на понятии точки – источнике закраски. Вспомните, там мы столкнулись с тем, что одна и та же точка может оказаться соседом нескольких точек, в результате чего она попадет в массив источников многократно, кроме того, соседом текущей точки является и та точка, которая была источником для текущей, что уже грозит движением по кругу.

Клетки доски Го также можно рассматривать как точки, а следовательно, проблемы предыдущей задачи полностью переходят и на данную. Но это, в общем-то, плюс, так как описанные проблемы уже решены, а их полная идентичность означает, что и решать их можно тем же методом. То есть, чтобы вторично не попасть в клетку, занятую своим камнем, этот камень после проведенного анализа, независимо от результата, убирается из клетки. Технически это можно осуществить так: пусть доска – это двумерный массив:

- ноль – клетка пуста;
- единица – клетка занята собственным камнем;
- двойка – клетка занята камнем противника.

Тогда, для того чтобы убрать камень из клетки, достаточно единицу заменить на любое другое число, кроме нуля. Свою работу рекурсивная функция начнет с указанного камня, каждый вызов породит столько вызовов, сколько у текущего камня окажется соседей. Возвращать функция будет истину, если она получит хотя бы раз истину от порожденных вызовов функций, и ложь, если все порожденные

вызовы функций вернут ложь. Таким образом, сам вызов функции вернет ложное значение только в том случае, если все порождаемые вызовы последующего уровня вернут значение ложь.

Примечание. После того как было указано на сходство двух разных задач, оставшиеся проблемы имеют характер технических вопросов, а не принципиальных, но, конечно, нужно согласиться, что задача о живой группе Го и задача о закраске контура выглядят слишком по-разному, чтобы искать в них аналогию. Однако ничего странного нет. Аналогию мы увидели совсем не в тексте условий, а в процессе поиска нужной точки. В обеих задачах ищется точка с определенными свойствами. Более тонкая аналогия в том, как она ищется. И это в обеих задачах делается одинаково. А именно для каждой точки просматриваются соседи, и для каждого соседа проводится анализ его свойств. Различие в задачах следующее. В задаче о закраске нужны все пустые точки, а в задаче о живой группе – хотя бы одна, но для технологии поиска это различие не имеет принципиального значения.

Обратите внимание, что в программе обрабатываются четыре направления, несмотря на то что одно из направлений – это направление-источник (клетка, из которой мы попали в данную), его рассматривать в качестве кандидата даже нет смысла. Но такой подход позволяет не запоминать клетку-источник и избавляет нас от существенных технических сложностей.

Только что мы рассмотрели рекурсивный вариант решения. Его соль и самый главный пункт – координаты текущего камня, порождающего рекурсивные вызовы для анализа соседей. В рекурсивном варианте эти координаты хранятся в стеке. Для создания нерекурсивного алгоритма необходимо продумать организацию стека из координат. Наиболее естественной структурой данных для этих целей, конечно, будет массив. А так как координат две, то речь пойдет о массиве записей из двух компонентов x и y .

Для того чтобы массив работал как стек, введем специальную переменную величину (назовем ее вершиной стека), всегда равную длине массива (точнее, его части, заполненной значимыми величинами). Она (эта переменная) обеспечит доступ к верхушке стека. В процессе обхода позиции при обнаружении камня занесение его координат в массив может быть выполнено двумя операциями:

- вершина стека увеличивается на 1;
- координаты камня записываются в элемент массива с индексом, равным вершине стека.

Очередной камень для обработки всегда берется с вершины стека. После обработки длина массива (стека), соответственно, уменьшается на 1. Анализ позиции прекращается в двух случаях.

1. На каком-то шаге обработки было обнаружено пустое соседнее поле. Даже одноразового такого события достаточно для завершения анализа. В таком случае можно сделать вывод, что исследуемая группа живая.
2. Длина массива (стека) стала равна нулю. Это означает, что исследованы все камни группы и ни разу не было обнаружено пустое поле. Следовательно, в этом случае вполне можно сделать вывод, что исследуемая группа не есть живая.

Задача для самостоятельного решения

Алгоритм живой группы Го разобран достаточно детально. Напишите самостоятельно его реализацию.

В заключение

Обсуждение в данном тексте в основном посвящено сравнению рекурсивного и нерекурсивного решений. И пример наиболее ярко показывает проблему номер один – разработку конструкции стека. Но в этой задаче мы столкнулись и с не менее интересной находкой. Мы увидели неожиданную аналогию с задачей о закраске контура. Этот пример показывает, что аналогия не есть похожесть условий.

Задача о черных пятнах на белой шкуре

Условие задачи. На белой шкуре есть некоторое количество черных пятен разного размера. Необходимо найти хотя бы одно самое большое.

Разумеется, мы не умеем работать с такой структурой данных, как черные пятна на белой шкуре. Поэтому условие сразу нуждается в формальном уточнении. Дадим более приемлемую формулировку.

Условие задачи. Двумерный массив заполнен нулями и единицами. Пятном из единиц называется такое множество элементов массива, для которого верно следующее утверждение: для любой пары элементов A и B пятна существует путь по точкам от A до B , такой что все точки пути также принадлежат пятну. Понятие пути мы определили в задаче о живой группе Го. Найти пятно, состоящее из наибольшего количества единиц.

Сразу заметим очень важную вещь. Из определения пути следует, что пятно однозначно определяется любым своим элементом, а именно пятно, порожденное некоторой единицей, есть множество элементов массива, до которых существует путь от данной единицы. Отсюда же следует идея алгоритма в первом приближении. Можно пробежать по всему массиву, для каждой найденной единицы построить порожаемое ей пятно и вычислить количество единиц, в него входящих. Требуемое уточнение идеи также лежит на поверхности. Совершенно очевидно, что в реализации каждое пятно будет повторяться столько раз, сколько в нем единиц. Поэтому необходимо для построения эффективной программы научиться игнорировать уже построенное пятно. Сделать это проще всего стиранием пятна в процессе его построения.

Задача для самостоятельного решения

Подробный анализ задачи и её реализации оставим на самостоятельную работу. Отметим только, что некоторые важные вещи, полезные для понимания техники реализации, уже использовались в задачах на закраску контура и в задаче о живой группе Го.

В заключение

Мы рассмотрели небольшое количество задач, но с детальным их анализом, чтобы показать возможные пути исследования, методы, приемы, возникающие трудности и способы их разрешения. Если вы прочитали главу внимательно, то должны были увидеть главный, ключевой момент – в любой задаче необходимо увидеть неопределенность в понимании условия, процесса, структуры данных, задать себе грамотный вопрос и ответить на него. Именно таким путем, пошагово, решая одну проблему за другой, и выстраивается общее решение.



ГЛАВА 7.

Практикум

Содержание последней главы – некоторое количество задач разной степени сложности по различным темам. Для каждой задачи, кроме условий, даны небольшие подсказки, которые помогут вам сформулировать идею решения. Сразу предупредим, что подсказки не надо воспринимать как прямое указание к действию. Они, скорее, задают возможное направление исследования, и не более того. Практикум не ставит своей целью дать систематический охват какой-либо темы или детальную отработку тех или иных методов решения задач. Просто набор задач для небольшой тренировки.

Задача 1. Дано натуральное число $n > 1$. Определить длину периода десятичной записи дроби $1/n$.

Подсказка. Первая цифра дробной части равна целой части от деления $10^k/n$ при минимально возможном k . Следующая цифра получается при делении на n уже остатка от предыдущего деления, умноженного на некоторую степень десятки. Из чего следует, что цифры десятичной записи определяются через последовательность остатков от деления и можно попробовать доказать, что период десятичной записи равен периоду в последовательности остатков.

Задача 2. Даны два многочлена A и B степени N и M соответственно. Оба многочлена заданы массивом своих коэффициентов. Необходимо определить массив коэффициентов многочлена C , являющегося их произведением, $C = A \times B$.

Подсказка. Степень многочлена C , очевидно, равна $M + N$. При перемножении многочленов A и B будет получено $M \times N$ слагаемых различных степеней от 0 до $M + N$. Степени некоторых слагаемых будут одинаковы. Сумма коэффициентов при таких слагаемых равна соответствующему коэффициенту (при слагаемом такой же степени) многочлена – произведения.

Задача 3. Даны два возрастающих числовых массива X и Y . Объединить их в возрастающий массив Z .

Подсказка. Процесс можно проиллюстрировать так. Пусть у нас есть две стопки карточек, на каждой из которых написано слово, отсортированных по алфавиту. Мы соединяем их в одну стопку, выбирая каждый раз ту из верхних карточек обеих стопок, которая приоритетна в алфавитном порядке.

Задача 4. Некоторое число содержится в каждом из трех целочисленных неубывающих массивов X , Y , Z . Найти это число хотя бы в одном массиве.

Подсказка. Принципиально достаточно просмотреть один массив, что гарантированно даст положение искомого числа. Изюминка задачи в том, что неизвестно, в каком массиве число стоит ближе к началу (просмотр массива от начала, наиболее естественное решение), а это значит, что выбор массива, весьма вероятно, может оказаться неудачным. И быть может более разумно организовать обход всех трех массивов одновременно.

Задача 5. Более сложный вариант предыдущей задачи. Заранее неизвестно, существует ли общий элемент в трех неубывающих массивах, требуется это выяснить, и конечно же, найти его хотя бы в одном из массивов.

Подсказка. Ко всему тому, чтобы было сказано в предыдущей задаче, добавим, что теперь нужен простой критерий, позволяющий быстро выяснять, есть ли смысл продолжать поиск в конкретном массиве.

Задача 6. Дана возрастающая последовательность целых чисел и число a . Выяснить, содержится ли число в этой последовательности.

Подсказка. Суть задачи в том, чтобы найти возможность уйти от простого перебора элементов массива. Как это возможно, было детально рассмотрено в четвертой главе данной книги.

Задача 7. Дан двумерный целочисленный массив, упорядоченный как по строкам, так и по столбцам. Требуется выяснить, встречается ли хотя бы один раз в этом массиве заданное число.

Подсказка. Отличие от предыдущей задачи – в двумерности массива. Поэтому если для поиска числа в одномерном массиве мы должны определить отрезок, в котором разумно искать число и определить метод сжатия отрезка, то здесь такой же подход необходимо применить к прямоугольнику внутри массива.

Задача 8. Дан неубывающий массив положительных целых чисел. Найти наименьшее целое положительное число, не представимое в виде суммы нескольких элементов этого массива (каждый элемент массива может быть использован не более одного раза).

Подсказка. Пусть известно, что числа, представимые в виде суммы элементов $a[1], \dots, a[k]$, заполняют отрезок от 1 до некоторого N . Если $a[k+1] > N+1$, то $N+1$ и будет минимальным числом, не представимым в виде суммы элементов массива. Если же $a[k+1] \leq N+1$, то числа, представимые в виде суммы элементов $a[1] \dots a[k+1]$, заполняют отрезок от 1 до $N+a[k+1]$.

Задача 9. Дан целочисленный массив и число b . Переставить числа в массиве таким образом, чтобы слева от некоторой границы стояли числа, меньшие или равные b , а справа от границы – большие b . Из операций, изменяющих порядок элементов в массиве, разрешена только операция перестановки двух элементов.

Подсказка. Можно выполнить только один проход массива, но одновременно в двух направлениях: от середины к началу и от середины к концу массива. Если слева будет встречен элемент, больший b , а справа на этом же шаге – меньший, чем b , то эти два элемента – очевидные кандидаты на перестановку. Но в этой идее есть существенный пробел. Здесь, очевидно, предполагается, что слева и справа кандидатов на перестановку будет одинаковое количество, что маловероятно.

Задача 10. В массиве стоят числа 0, 1, 2. Переставить их в порядке возрастания, если единственной разрешенной операцией, изменяющей порядок элементов, является операция перестановки двух элементов.

Подсказка. Очевидно, это вариант предыдущей задачи, если задать не одну границу, а две.

Задача 11. Даны массив $a[1] \dots a[n]$ и число $m \leq n$. Для каждой группы из m стоящих рядом членов (таких групп, очевидно, $n-m+1$) вычислить ее сумму за минимальное число действий.

Подсказка. Для этой задачи существует простое, лобовое решение. Если же поставить цель максимально уменьшить количество операций, то можно подумать о том, как от уже по-

считанной суммы перейти к следующей, не считая учтенных элементов заново. А именно новая сумма образуется вычитанием ровно одного элемента и добавлением также ровно одного элемента.

Задача 12. Даны квадратная целочисленная таблица размером $N \times N$ и число $L \leq N$. Для каждого квадрата размера $L \times L$ в этой таблице вычислить сумму стоящих в нем чисел. Общее число действий должно быть минимально.

Подсказка. Задача от предыдущей отличается только размерностью массива, поэтому общая идея будет той же, только добавляется и вычитается столбец или строка элементов.

Задача 13. Даны две последовательности целых чисел. Найти максимальную длину последовательности, являющейся подпоследовательностью обеих последовательностей.

Подсказка. Если две последовательности ограничить небольшим числом элементов, то составить множество всех подпоследовательностей и затем определить длину наибольшей труда бы не составило. Если так и сделать, то затем можно добавлять к исходным последовательностям по одному элементу и пересчитывать длины во множестве подпоследовательностей.

Задача 14. Дана последовательность целых чисел. Найти максимальную длину ее возрастающей подпоследовательности.

Подсказка. К задаче применим подход, предложенный в предыдущей задаче. Выделим из исходной последовательности небольшую, например некоторое количество первых членов. Построим множество возрастающих последовательностей, затем начнем наращивать исходную и пересчитывать возрастающие.

Задача 15. Перечислить все разбиения целого положительного числа n на целые положительные слагаемые (разбиения, отличающиеся лишь порядком слагаемых, считаются за одно).

Подсказка. Получить хотя бы одно представление не составляет труда. Например, любое число представимо суммой единиц. Затем остается придумать, каким образом, имея очередное представление, перейти к следующему. Это можно сделать так: найдем первое слагаемое, которое можно увеличить на единицу. Прибавим к нему единицу, при этом сумма превысит наше число на 1. Осталось придумать, как скомпенсировать эту новую единицу всеми возможными способами, используя слагаемые, стоящие правее увеличенного.

Задача 16. Перечислить все последовательности длины n из чисел $1\dots k$ в таком порядке, чтобы каждая следующая отличалась от предыдущей в единственной цифре, причем не более, чем на 1.

Подсказка. Рассмотрим прямоугольную доску ширины n и высоты k . На каждой вертикали поставим шашку. Таким образом, положения шашек соответствуют последовательностям из чисел $1\dots k$ длины n (s -ый член последовательности соответствует высоте шашки на s -ой горизонтали). На каждой шашке нарисуем стрелочку, которая может быть направлена вверх или вниз. Вначале все шашки поставим на нижнюю горизонталь стрелочкой вверх. Далее двигаем шашки по такому правилу: найдя самую правую шашку, которую можно подвинуть в направлении нарисованной на ней стрелки, двигаем ее на одну клетку в этом направлении, а все стоящие правее ее шашки (они уперлись в край) разворачиваем кругом. Ясно, что на каждом шаге только одна шашка сдвигается, то есть один член последовательности меняется на 1.

Задача 17. Напечатать все перестановки чисел $1\dots n$ так, чтобы каждая следующая получалась из предыдущей перестановкой двух соседних чисел.

Подсказка. Задача почти очевидно сводима к предыдущей. Действительно построим первую перестановку как возрастающую или, наоборот, убывающую последовательность. Получить из неё новую последовательность можно, поменяв любые два элемента.

Задача 18. Перечислить все последовательности длины $2n$, составленные из n единиц и n минус единиц, у которых сумма любого начального отрезка положительна.

Подсказка. Изображая единицу вектором $(1, 1)$, а минус единицу вектором $(1, -1)$, можно сказать, что мы ищем пути из точки $(0, 0)$ в точку $(n, 0)$, не опускающиеся ниже оси абсцисс.

Задача 19. На окружности задано $2n$ точек, пронумерованных от 1 до $2n$. Перечислить все способы провести n непересекающихся хорд с вершинами в этих точках. Из каждой точки может выходить не более одной линии хорды.

Подсказка. Очевидно, что любая хорда множество точек разбивает на два множества, назовем их A и B . Для того чтобы следующая хорда не пересекалась с уже построенной, она

должна соединять две точки, принадлежащие только одному из множеств, либо A , либо B . Таким образом, в любом из названных множеств для любой точки должна найтись пара, а это возможно только в том случае, если в каждом из множеств будет четное число точек.

Задача 20. Перечислить все расстановки скобок в произведении n сомножителей. Порядок сомножителей не меняется, скобки полностью определяют порядок действий.

Подсказка. Задача становится очень похожа на предыдущую, если представить себе, что сомножители – это точки на окружности, а пара скобок, – это концы хорды. Есть здесь, правда, важный нюанс. Количество множителей не обязательно четно.

Задача 21. Дано n точек на плоскости. Указать $(n-1)$ -звенную несамопересекающуюся незамкнутую ломаную, проходящую через все эти точки. (Соседним отрезкам ломаной разрешается лежать на одной прямой.)

Подсказка. Упорядочим точки по x -координате, а при равных x -координатах – по y -координате. В таком порядке и можно проводить ломаную.

Задача 22. Дано n точек на плоскости. Построить их выпуклую оболочку – минимальную выпуклую фигуру, их содержащую. (Форму выпуклой оболочки примет резиновое колечко, если его натянуть на гвозди, вбитые в точках.)

Подсказка. Упорядочим точки – годится любой из порядков, использованных в двух предыдущих задачах. Затем, рассматривая точки по очереди, будем строить выпуклую оболочку уже рассмотренных точек.

Задача 23. Даны n чисел и функция f , принимающая (на них) значения $1...m$. Требуется переставить числа в таком порядке, чтобы значения функции f не убывали (сохраняя порядок внутри каждой из групп).

Подсказка. Попробуйте завести m списков суммарной длины n и помещать в i -й список числа, для которых значение функции f равно i . Вариант: посчитать для всех i , сколько имеется чисел x с $f(x)=i$, после чего легко определить, с какого места нужно начинать размещать числа с $f(x)=i$.

Задача 24. Имеется n одинаковых на вид камней, некоторые из которых на самом деле различны по весу. Имеется прибор, позволяющий по двум камням определить, одинаковы они

или различны (но не говорящий, какой тяжелее). Известно, что среди этих камней большинство (более $n/2$) одинаковых. Сделав не более n взвешиваний, найти хотя бы один камень из этого большинства.

Подсказка. Если найдены два различных камня, то их оба можно выбросить – хотя бы один из них плохой, и большинство останется большинством.

Задача 25. Дан связный ориентированный граф. Также известно, что из каждой вершины выходит столько же ребер, сколько входит. Доказать, что существует замкнутый цикл, проходящий по каждому ребру ровно один раз. Составить алгоритм поиска такого цикла.

Подсказка. Представьте себе змею, которая ползет по графу, поедая его вершины и увеличиваясь в длине. Если она перекусит сама, себя это и будет означать, что обнаружен цикл.

Задача 26. Доказать, что для всякого n существует последовательность нулей и единиц длины 2^n со следующим свойством: если «свернуть ее в кольцо» и рассмотреть все фрагменты длины n (их число равно 2^n), то мы получим все возможные последовательности нулей и единиц длины n . Построить алгоритм отыскания такой последовательности.

Подсказка. Рассмотрим граф, вершинами которого являются последовательности нулей и единиц длины $(n - 1)$. Будем считать, что из вершины x ведет ребро в вершину y , если x может быть началом, а y – концом некоторой последовательности длины n . Тогда из каждой вершины входят и выходят два ребра. Цикл, проходящий по всем ребрам, и даст требуемую последовательность.

Задача 27. Написать программу подсчета числа вершин в дереве.

Подсказка. Задача, очевидно, имеет хорошее рекурсивное решение, которое можно получить из следующего утверждения: любое поддерево, порожденное некоторой вершиной, имеет количество вершин, равное сумме вершин, порожденных её сыновьями плюс она сама.

Задача 28. Требуется составить график соревнований по виду спорта, предполагающего парные состязания. Вид спорта роли не играет, необходимо выполнить только следующие условия:

- в каждом бою участвуют два спортсмена;
- проигравший спортсмен выбывает из соревнования;
- в любом поединке встречаются спортсмены, имеющие или равное количество боев, или у одного из них на один бой меньше.

Подсказка. Предложим следующий способ определения пар бойцов: выстроим спортсменов в ряд. Пусть пары образуют спортсмены, являющиеся соседями. Пары можно создавать, проходя ряд слева направо (назовем это направление прямым) и, наоборот, проходя ряд справа налево (назовем это направление обратным). Прямое и обратное направления не равнозначны, в том смысле, что в прямом направлении боец без пары может оказаться на правом краю и в обратном на левом. Начнем построение пар с прямого прохода. Построение пар на очередном шаге может оставить без пары только одного бойца. Если на следующем шаге направление построения пар оставить тем же, то боец, оставшийся без пары на предыдущем шаге, опять может остаться без соперника.

Задача 29. Даны два положительных целых числа N и M . Вычислить N^M быстрее, чем за M умножений.

Подсказка. Обдумайте, как можно свести степень к более простому выражению на основании информации о четности или нечетности степени.

Задача 30. На плоскости задана целочисленная решетка (координаты узлов решетки – целые числа). На плоскости также задан многоугольник, координаты которого – рациональные числа. Определить количество узлов решетки, лежащих в вершинах и на сторонах многоугольника.

Подсказка. Задача сводится к определению точек, которые лежат на прямой, заданной двумя точками.

Книги издательства «ДМК Пресс» можно заказать в торгово-издательском холдинге «АЛЬЯНС БУКС» наложенным платежом, выслав открытку или письмо по почтовому адресу: **123242, Москва, а/я 20** или по электронному адресу: **orders@alians-kniga.ru**.

При оформлении заказа следует указать адрес (полностью), по которому должны быть высланы книги; фамилию, имя и отчество получателя. Желательно также указать свой телефон и электронный адрес.

Эти книги вы можете заказать и в интернет-магазине: **www.alians-kniga.ru**.

Оптовые закупки: тел. **(499) 782-38-89**

Электронный адрес: **books@alians-kniga.ru**.

Потопахин Виталий Валерьевич

Искусство поиска решения в нестандартной задаче

Главный редактор *Мовчан Д. А.*
dmkpress@gmail.com

Корректор *Синяева Г. И.*

Верстка *Паранская Н. В.*

Дизайн обложки *Мовчан А. Г.*

Формат 60×90 ¹/₁₆. Гарнитура «Петербург».

Печать офсетная. Усл. печ. л. 10,17.

Тираж 100 экз.

Веб-сайт издательства: www.dmk.ru